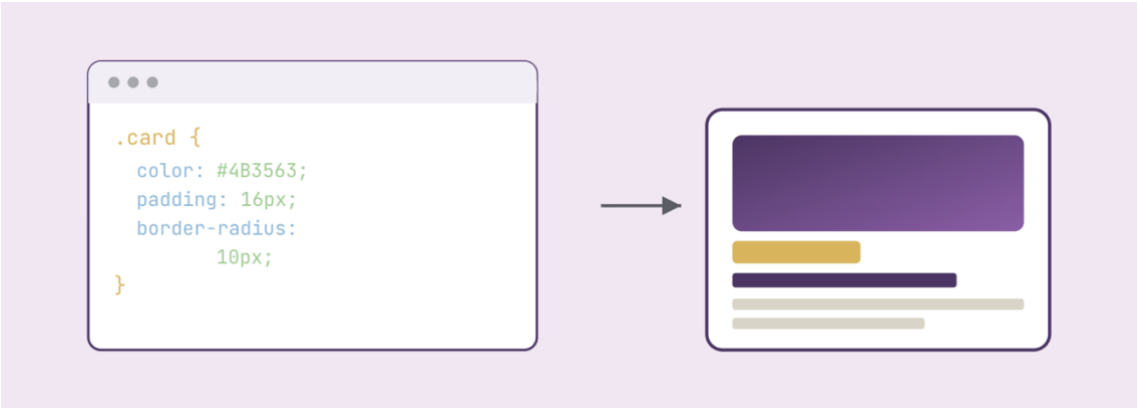
Tema 3
Fundamentos de CSS



Es el tema más largo del módulo, y con motivo: aquí se sientan las bases sobre las que se apoyará todo lo que viene después (Flexbox, Grid, CSS, animaciones). Tómatelo con calma, prueba cada ejemplo en tu propio editor, y no pases al siguiente apartado hasta que el anterior te resulte cómodo.

1. HTML semántico como base del estilado

1.1 Por qué HTML semántico antes de CSS

El HTML semántico consiste en elegir, para cada bloque de contenido, la etiqueta que describe qué es ese contenido, no solo cómo debería ordenarse en pantalla. Un menú de navegación se marca con <nav> porque es una navegación, no porque "quede colocado arriba".

Podría parecer un asunto puramente técnico, pero tiene consecuencias muy concretas:

- **Accesibilidad**: un lector de pantalla anuncia "navegación" o "contenido principal" a partir de estas etiquetas, permitiendo saltar directamente entre secciones. Sin ellas, una persona que no puede ver la pantalla se enfrenta a un bloque de texto indiferenciado.
- **Posicionamiento en buscadores (SEO):** los motores de búsqueda usan estas etiquetas para entender qué parte de la página es el contenido relevante y cuál es navegación repetida en todas las páginas.
- **Mantenibilidad del CSS:** seleccionar article h2 es más estable y legible en el tiempo que depender de una clase como class="titulo-azul-grande", que deja de tener sentido en cuanto cambia el diseño.
- **Estructura reutilizable:** las etiquetas semánticas de estructura de página son encabezado, navegación, contenido principal, barra lateral, pie.

Nota: Antes de HTML5 (2014), era habitual maquetar una página entera a base de con nombres de clase como class="header" o class="nav". Funcionaba visualmente, pero no aportaba ningún significado real al navegador ni a las tecnologías de asistencia: un div es siempre, semánticamente, "un bloque genérico sin significado propio", lo diga su clase o no.

1.2 Etiquetas de estructura de página

Estas son las etiquetas que definen las grandes regiones de una página:

Etiqueta	Región que representa
`<header>`	Cabecera de la página o de una sección: logotipo, título, navegación introductoria. Puede repetirse dentro de un `<article>`.
`<nav>`	Un bloque de enlaces de navegación principal. No todo grupo de enlaces necesita `<nav>`: se reserva para los bloques de navegación realmente relevantes.

`<main>`	El contenido principal y único de la página. Solo puede haber un `<main>` visible por página.
`<article>`	Contenido autónomo que tendría sentido por sí solo fuera de contexto: una entrada de blog, una noticia, un comentario.
`<section>`	Una agrupación temática de contenido, normalmente con su propio encabezado. A diferencia de `<article>`, no tiene por qué tener sentido de forma aislada.
`<aside>`	Contenido relacionado pero secundario respecto al contenido principal: enlaces relacionados, publicidad, una biografía del autor.
`<footer>`	Pie de página o de sección: información legal, enlaces secundarios, datos de contacto.



Importante: <section> es, con diferencia, la etiqueta que más se usa mal. La regla práctica: si el bloque no tiene un encabezado propio (<h2>, <h3>...) y solo lo estás usando para poder aplicarle un estilo, probablemente lo que necesitas es un , no un <section>.

1.3 **Div y span: los comodines**

 y son las dos únicas etiquetas de HTML sin ningún significado semántico propio: son contenedores genéricos que existen para agrupar contenido cuando ninguna etiqueta semántica encaja, casi siempre con el único fin de aplicarles estilos o comportamiento. es un contenedor de bloque; , uno de línea (dentro de un párrafo, por ejemplo).

Usarlos no es un error: lo es usarlos cuando existe una alternativa semántica mejor. La regla práctica es sencilla: antes de escribir , pregúntate si existe una etiqueta que describa mejor ese contenido.



 sigue siendo perfectamente legítimo dentro de una frase, por ejemplo para aplicar un color a una sola palabra sin crear un salto de línea: Esta palabra está resaltada.

Actividad · WebCompleta 1
Esta página ya tiene todo el contenido de la landing page de Nimbus, pero maquetada solo con . Tu tarea es sustituir los que sí tienen una etiqueta semántica mejor, sin tocar el contenido de dentro:
1. El div que envuelve la cabecera → <header>.
2. El div de los tres enlaces principales → <nav>.
3. El div que envuelve todo el contenido central → <main> (recuerda: solo puede haber un <main> por página).
4. Los cuatro bloques de contenido (hero, producto, precios, testimonios) → <section>, conservando su mismo id para que los enlaces del menú sigan funcionando.
5. El div que envuelve cada testimonio → <blockquote>, y el div con el nombre de la persona → <cite>.
6. El div del pie de página → <footer>.

Deja como el resto de bloques (las tarjetas de características, las tarjetas de precio, las columnas del pie): no tienen una etiqueta semántica mejor..

2. Qué es CSS

CSS (Cascading Style Sheets, hojas de estilo en cascada) es el lenguaje que define la presentación visual de un documento HTML: colores, tipografías, espaciados, tamaños y posición de cada elemento. Su ventaja fundamental es separar el contenido de la presentación: el HTML dice qué es cada cosa, el CSS dice cómo se ve.

Esa separación, hoy evidente, fue una revolución cuando se generalizó a finales de los 90: antes, el aspecto visual se controlaba con atributos y etiquetas de presentación

mezcladas directamente en el HTML (<font color="red">, ...), hoy obsoletas y eliminadas del estándar. Separar ambas capas trae ventajas muy concretas:

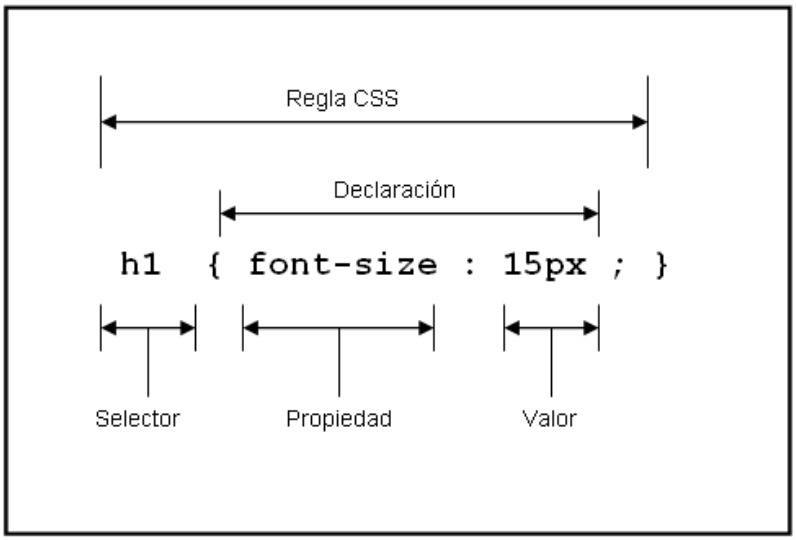
- **Mantenimiento**: cambiar el aspecto de todo un sitio modificando un único archivo.
- **HTML más ligero**, sin marcado repetido de presentación en cada página.
- **Consistencia**: la misma hoja de estilos aplicada a cientos de páginas garantiza que todas usen la misma guía de estilo.
- **Soporte prácticamente universal**: todos los navegadores modernos implementan CSS3.

Nota: CSS es un estándar mantenido por el W3C (World Wide Web Consortium), organización que también define HTML y las pautas WCAG que veremos más adelante. CSS3 no es una versión única y cerrada, sino un conjunto de módulos independientes (selectores, colores, animaciones, Grid...) que avanzan y se actualizan cada uno a su propio ritmo.

3. Sintaxis CSS

3.1 Sintaxis de una regla y comentarios

Una regla CSS asocia un selector (qué elementos se ven afectados) con un bloque de declaraciones (propiedad y valor, separados por dos puntos y terminados en punto y coma).




```
.card {
  color: #24262B;
  padding: 16px;
  border-radius: 10px;
}
```

Los comentarios en CSS se escriben entre /* */ (nunca con la sintaxis de comentario HTML <!-- -->, que en un archivo .css no significa nada).

```
/* Estilo de la tarjeta de artículo */
.card {
```

```
  padding: 16px; /* espacio interior */
}
```

3.2 Formas de incorporar CSS

Hay tres formas de aplicar CSS a un documento HTML, y solo una de ellas es la recomendable para un proyecto real:

- **En línea (inline)**

Con el atributo style, directamente sobre la etiqueta:

```
<p style="color: red; font-weight: bold;">Aviso importante</p>
```

- **Interna**

Dentro de una etiqueta <style> en el <head> del documento:

```
<head>
  <style>
    p { color: red; }
  </style>
</head>
```

- **Externa (recomendada)**

En un archivo .css aparte, vinculado con <link>:

```
<link rel="stylesheet" href="estilos.css">
```

También existe la regla @import, que permite importar una hoja de estilos desde dentro de otra:

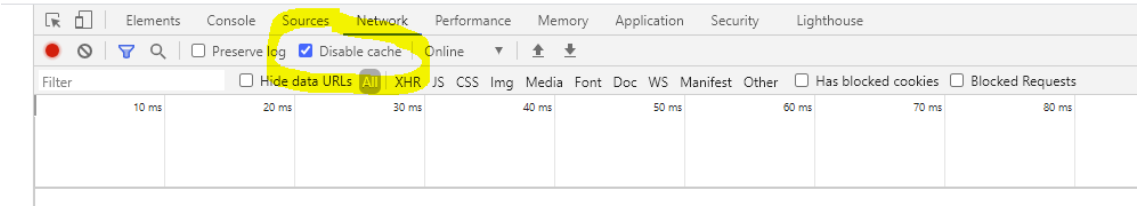
```
/* Al principio de otro archivo .css */
@import url("base.css");
```

Método	Cuando usarlo
En línea	Casi nunca en producción, solo para pruebas rápidas o estilos generados dinámicamente por JavaScript.
Interna	Páginas de un único uso, correos HTML, o prototipos muy pequeños.
Externa	Cualquier proyecto real: se cachea por separado, se reutiliza entre páginas y mantiene el HTML limpio.

3.3 Evitar la caché del navegador

A veces nos puede pasar que el navegador almacena en caché nuestro fichero css (o js, …) y aunque modifiquemos dicho fichero, el navegador no coge los nuevos cambios porque ya tiene ese fichero en la caché.

Para evitar esto, a veces podemos usar Ctrl+F5 para forzar una recarga completa o también podemos pulsar F12 (o botón derecho → Inspeccionar), ir a la sección 'Network' y desactivar la caché:



3.4 Elementos de bloque y en línea

Antes de ver todos los valores de display, conviene saber de dónde viene esta propiedad. Cada etiqueta HTML trae, por defecto, un comportamiento ya decidido por el navegador: es un elemento de bloque o un elemento en línea. Un elemento de bloque genera un salto de línea antes y después de sí mismo, y ocupa todo el ancho disponible; un elemento en línea no genera ningún salto de línea, y ocupa solo el espacio justo de su contenido, encajando en medio de una frase.

p es un elemento de bloque; strong es un elemento en línea. Es la razón por la que dos párrafos seguidos siempre aparecen uno debajo del otro, mientras que resaltar una palabra con no rompe la frase en la que está:

```
<p>Este texto tiene una strong>palabra importante</strong> en medio.</p>

<p>Y este es un segundo párrafo distinto.</p>
```

Esta clasificación no es fija ni definitiva: con CSS puedes cambiarla en cualquier etiqueta, con la propiedad display y, entre otros, los valores block e inline. El resto de esta sección trata, precisamente, de esa propiedad y de todos sus valores:

Valor de display	Comportamiento
block	Ocupa todo el ancho disponible y empieza en una línea nueva (div, p, article...).
inline	Ocupa solo el espacio de su contenido, en la misma línea que lo que le rodea (span, a...). No admite width/height/margen vertical.
inline-block	Se comporta como inline en el flujo, pero admite width, height y márgenes verticales como block.
none	Elimina el elemento del renderizado: no ocupa espacio, como si no existiera.
flex / grid	Convierte al elemento en contenedor flexible o de rejilla.

3.5 Selectores

El selector decide a qué elementos se aplica una regla. Estos son los básicos:

Selector	Sintaxis	Selecciona
Tipo (etiqueta)	article	Todos los elementos <article>.

Clase	.card	Todos los elementos con class="card" (una clase puede repetirse en muchos elementos).
ID	#main-nav	El único elemento con id="main-nav" (un id no debe repetirse en la página).
Universal	*	Todos los elementos.
Atributo	[type="email"]	Elementos con ese atributo y valor exactos.

Los selectores se combinan para describir relaciones entre elementos:

Combinador	**Sintaxis**	**Selecciona**
Descendiente	.card p	Cualquier <p> dentro de .card, a cualquier profundidad.
Hijo directo	.card > p	Solo los <p> que sean hijos inmediatos de .card.
Hermano adyacente	h3 + p	El <p> que va justo después de un <h3>.
Hermano general	h3 ~ p	Todos los <p> que compartan padre y vayan después del <h3>.
Múltiple	h1, h2, h3	Varios elementos con las mismas reglas de estilo

Descendente (p span): selecciona cualquier dentro de <p>, sin importar a qué profundidad esté anidado:

```
.p span {
  color: #5B5D63;
}
```

Por ejemplo, en el siguiente ejemplo, ¿que texto saldría rojo?

```
<p>
  bla bla bla <span>texto1</span> bla bla bla<br/>
  <a href="">bla bla bla <span>texto2</span></a>
  ...
</p>
<span>otro</span>
```

¿Cómo saldrían ahora los textos del ejemplo anterior?

```
/* selecciona todos los span que estén dentro de un elemento (cualquiera), que a su vez esté dentro de un párrafo */
p * span { color: yellow; }
```

Hijo directo (.card > p): solo selecciona los <p> que sean hijos *inmediatos* de .card, saltándose los que estén envueltos en otro elemento intermedio:

```
.card > p {
  color: #5B5D63;
}
```

Ejemplo:

```
p > span { color: green; }
```

En el siguiente HTML, ¿qué textos estarían en verde?

```
<p><span>Texto1</span></p>
<p><a href="#"><span>Texto2</span></a></p>
```

Hermano adyacente (h3 + p): selecciona el elemento que va **justo después**, compartiendo el mismo padre.

```
h3 + p {
  font-weight: 600;
}
```

Para que se cumpla:

- elemento1 y elemento2 deben ser elementos hermanos (mismo padre)
- elemento2 debe aparecer inmediatamente después de elemento1 en el código HTML.

Dado el siguiente código, ¿qué encabezados h2 aparecen en rojo y cuales en verde?:

```
h2 { color: green; }
h1 + h2 { color: red }
```

```
<h1>Titulo H1</h1>
<h2>Subtítulo</h2>
<h2>Otro subtítulo 1</h2>
<h2>Otro subtítulo 2</h2>
<h2>Otro subtítulo 3</h2>
```

El siguiente código, ¿cómo funcionará?

```
p + p { color: blue; }
```

```
<p>Bla bla bla</p>
<p>Bla bla bla</p>
<p>Bla bla bla</p>
<p>Bla bla bla</p>
<p>Bla bla bla</p>
```

Hermano general (h3 ~ p): selecciona **todos** los elementos que compartan padre con el <h3> y vengan después de él, no solo el inmediato. Es decir, tienes que ser hermanos que estén a continuación, pero no hace falta que esté justo detrás.

```
h3 ~ p {
  font-weight: 600;
}
```

Dado el siguiente código, ¿qué encabezados h2 aparecen en rojo y cuales en verde?:

```
h2 { color: green; }
h1 ~ h2 { color: red }
h1 ~ h3 { color: green }

<h1>Titulo H1</h1>
<h2>Subtítulo</h2>
<h2>Otro subtítulo 1</h2>
<h3>Otro subtítulo 3</h3>
<h2>Otro subtítulo 2</h2>
```

Otra forma de posicionar selectores

Si tenemos el siguiente código HTML:

```
<p class="destacado">Bla bla bla...</p>
<p>Bla bla bla <a href="#" class="destacado">bla bla bla</a> bla bla bla...</p>
<p>Bla bla bla <em class="destacado">bla bla bla</em> bla bla bla...</p>
```

Este selector pone en rojo todas las clases “destacado”:

```
.destacado{ color: red; }
```

Y si quiero aplicar el color rojo, solamente al primer párrafo, ¿cómo lo puedo hacer? Esto se consigue, combinando el selector de tipo y de clase:

```
p.destacado { color: red; }
```

OJO !!!, cuidado con la sintaxis:

```
/* Todos los elementos de tipo "p" con atributo class="aviso" */
p.aviso { ... }

/* Todos los elementos con atributo class="aviso" que estén dentro
   de cualquier elemento de tipo "p"(OJO! Al espacio entre p y .aviso */
p .aviso { ... }

/* Todos los elementos "p" de la página y todos los elementos con
   atributo class="aviso" de la página */
p, .aviso { ... }
```

Comentar también que HTML permite asignar varias clases a un mismo elemento:

```
<p class="especial destacado error">Párrafo de texto...</p>
```

```
.error { color: red; }
.destacado { font-size: 15px; }
.especial  { font-weight: bold; }
```

Esto puede llegar a ser muy útil en el diseño, estructurando bien los estilos y ahorrando código.
Para rizar el rizo, también podemos combinar dos selectores clases:

```
/* aquellos elementos de la página que dispongan de un atributo class con al menos los valores error y destacado */
.error.destacado { color: blue; }
```

Ejercicio 1· Selectores
A partir del código HTML y CSS que se encuentra AQUÍ, añadir los selectores CSS que faltan para aplicar los estilos deseados. Cada regla CSS incluye un comentario en el que se explica los elementos a los que debe aplicarse:

3.6 Cascada, especificidad y herencia

Cuando varias reglas CSS afectan al mismo elemento con la misma propiedad, el navegador decide cuál gana siguiendo, en este orden, tres criterios: el origen (una regla tuya frente a la del navegador, y !important por encima de todo), la especificidad del selector, y, si empatan, el orden en el que aparecen (la última gana).

Padres, hijos y hermanos
Los elementos de un documento HTML forman una estructura equivalente a un árbol genealógico. Un párrafo es hijo de <body> si está contenido directamente en él, y <body> es, por tanto, su padre. Si hay varios párrafos contenidos directamente en <body>, esos párrafos son hermanos entre sí: comparten el mismo padre.
Entender esta relación es la base de dos cosas que ya conoces (los selectores, que literalmente navegan estas relaciones) y de una que viene ahora: la herencia.

Cómo se calcula la especificidad
Lo que más confunde al empezar es pensar que la especificidad es un número único que se calcula sumando. No lo es. Son tres recuentos independientes, uno por cada categoría de selector, y se comparan de uno en uno, empezando por el más importante, exactamente como compararías las cifras de 101 con 012 una a una, no sumándolas: el primer dígito ya decide.

Categoría	Qué cuenta	Vale
ID	#id	lo más
Clase	.clase, [atributo], :pseudo-clase (como :hover, :nth-child()...)	lo de en medio
Tipo	etiqueta, ::pseudo-elemento (como ::before)	lo menos

Fíjate en algo que suele quedar sin decir: los selectores de atributo y las pseudo-clases cuentan en la misma categoría que las clases, no por separado. Y los pseudo-elementos (::before, ::marker...) cuentan como si fueran una etiqueta más.

El procedimiento, paso a paso

1. Cuenta cuántos id tiene cada selector. Si uno tiene más que el otro, **ya ha ganado**: no hace falta mirar nada más.
2. Si empatan en id (lo normal es que ambos tengan 0), cuenta clases + atributos + pseudo-clases. El que tenga más, gana.
3. Si también empatan ahí, cuenta etiquetas + pseudo-elementos. El que tenga más, gana.
4. Si empatan en los tres recuentos, no hay ganador por especificidad: decide el orden de aparición (la última regla escrita).



El selector `article .card h3` tiene especificidad (0 id, 1 clase, 2 etiquetas). Frente a él, `#post-42 h3` (1 id, 0 clases, 1 etiqueta) gana siempre, sin importar el orden en que se escriban ambas reglas.

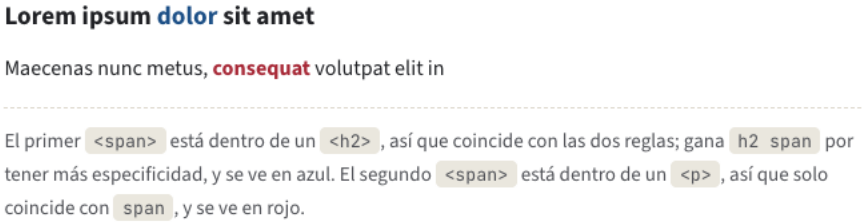
Por qué esto importa: mil clases no le ganan nunca a un id

.a.b.c.d.e → 0 id, 5 clases, 0 etiquetas
#x → 1 id, 0 clases, 0 etiquetas

Dos reglas más para fijar la idea, antes de seguir. Primero, cuando dos selectores de **distinta** especificidad compiten, gana el más específico, sin importar cuál esté escrita primero:

```
<h2>Lorem ipsum <span>dolor</span> sit amet</h2>
<p>Maecenas nunc metus, <span>consequat</span> volutpat elit in</p>
```

```
h2 span { color: blue; }  /* 2 etiquetas */
span { color: red; }      /* 1 etiqueta */
```



El primer `<span>` está dentro de un `<h2>`, así que coincide con las dos reglas; gana `h2 span` por tener más especificidad, y se ve en azul. El segundo `<span>` está dentro de un `<p>`, así que solo coincide con `span`, y se ve en rojo.

Segundo, cuando dos selectores tienen **la misma** especificidad, no hay empate real: gana la que esté escrita más abajo en la hoja de estilos.

```
<h2>Este titular decide su color por el orden, no por la especificidad</h2>
```

```
h2 { color: blue; }
```

```
h2 { color: red; }
```

Este titular decide su color por el orden, no por la especificidad

Las dos reglas son idénticas en especificidad (una sola etiqueta). Gana `red`, la segunda: en caso de empate, siempre se impone la declaración que aparece más tarde.

Origen (de menor a mayor prioridad)	Ejemplo
Estilos por defecto del navegador	El azul y subrayado por defecto de un enlace.
Tus reglas en hoja externa/interna, por especificidad	article .card h3 vs. #post-42 h3
Estilo en línea	style="color: red;"
!important	color: red !important; (evítalo: rompe la cascada y complica el mantenimiento)

Dos ejemplos resueltos

El primero esconde una sorpresa real, no solo teórica: pruébalo pasando el cursor por el enlace.

```
<nav class="nav" id="main-nav">
  <a href="#">Inicio</a>
</nav>
```

```
.nav a:hover {
  color: orange;
}
#main-nav a {
  color: blue;
}
```

Selector	ID	Clase	Tipo
.nav a:hover	0	2 (.nav + :hover)	1 (a)
#main-nav a	**1**	**0**	**1 (a)**

El enlace se ve azul en reposo, porque solo coincide #main-nav a. Pasa el cursor por encima: **sigue azul**. #main-nav a tiene un id, así que gana en el paso 1, sin mirar que la otra regla tenga una pseudo-clase. El efecto :hover queda enterrado, aunque sí está coincidiendo en ese momento: no hay ningún error visible en el código, simplemente

esa regla nunca puede ganar. Es, en la práctica, la causa más habitual de un :hover que "no funciona".

```
<a href="https://ejemplo.com" class="externo">Visita la web</a>
a[href^="https"] {
  color: green;
}
.externo {
  color: purple;
}
```

Selector	ID	Clase	Tipo
a[href^="https"]	**0**	**1 (atributo)**	**1 (a)**
.externo	0	1	0

Empatan en id (0) y en la columna de clase (1 contra 1: el selector de atributo cuenta igual que una clase). El desempate pasa a la columna de etiqueta: a[href^="https"] suma 1 por el a, .externo suma 0. Gana a[href^="https"], y el enlace se ve verde.

Herencia
La herencia decide qué propiedades heredan los elementos hijos de su padre, y cuáles no. Tiene sentido que color y font-family se hereden: así defines la tipografía base una sola vez en body, y toda la página la adopta, sin tener que repetirla elemento a elemento. Y tiene sentido que margin, padding, border o background no se hereden nunca por defecto: imagina el caos si el margen de un contenedor se copiara automáticamente en cada uno de sus hijos, y en los hijos de sus hijos.

Qué se hereda y qué no responde, en general, al sentido común, pero si alguna vez tienes dudas sobre una propiedad concreta, la referencia de CSS de MDN lo indica explícitamente: cada propiedad tiene una ficha con una tabla resumen que incluye el campo Inherited (se hereda / no se hereda).

Puedes forzar el comportamiento de cualquier propiedad con cuatro valores especiales, válidos en todas ellas:

Valor	Efecto
inherit	Toma el mismo valor que tiene calculado su elemento padre, se herede esa propiedad por defecto o no.
initial	Vuelve al valor inicial que define **la especificación de CSS** para esa propiedad (el mismo en todos los navegadores), no al que aplique el navegador para esa etiqueta en concreto.
unset	Actúa como inherit si la propiedad se hereda de forma natural, o como initial si no.

revert	Vuelve al valor que le daría la hoja de estilos propia del navegador a esa etiqueta, como si tu regla no existiera.

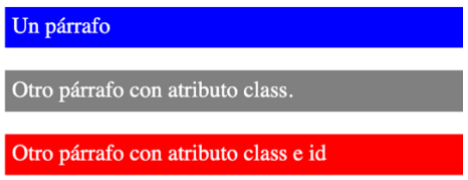
Importante: Antes de añadir !important o inflar un selector con más clases "para que gane", pregúntate si el problema real es de organización: casi cualquier conflicto de especificidad se resuelve mejor reordenando o simplificando las reglas que forzando la cascada.

Para practicar: varias reglas sobre el mismo elemento

Un último ejemplo que combina todo lo visto en este apartado: especificidad, !important y herencia a la vez. Antes de mirar el resultado, intenta predecir qué fondo, borde y color tendrá cada párrafo.

```
<p>Un párrafo</p>
<p class="better">Otro párrafo con atributo class.</p>
<p class="better" id="winning">Otro párrafo con atributo class e id</p>
```

```
#winning {
  background-color: red;
  border: 1px solid black;
}
.better {
  background-color: gray;
  border: none !important;
}
p {
  background-color: blue;
  color: white;
  padding: 5px;
}
```



El primer párrafo solo coincide con p: fondo azul. El segundo coincide con p y .better; .better gana el fondo (más especificidad) y, por el !important, también el borde, pero el color y el padding siguen viniendo de p, porque .better no los toca. El tercero coincide con las tres reglas: para el fondo gana #winning (el id tiene más especificidad que la clase), pero para el borde gana igualmente .better, porque un !important vence a cualquier especificidad normal, por alta que sea.

Ejercicio 2· Selectores

Crea un fichero HTML con un h1 y un párrafo. A continuación, añade los estilos necesarios utilizando "clases" para que el texto de h1 se muestran en verde con fondo azul, y el texto del párrafo esté en azul con fondo verde. Resuelve de dos maneras:
- Utilizando 4 clases: texto_azul, texto_verde, fondo_azul y fondo_verde
- Utilizando 2 clases: texto_azul_con_fondo_verde y texto_verde_con_fondo_azul

Ejercicio 3· Selectores

Interpreta el siguiente código. ¿De qué color y tamaño aparecerán los diferentes elementos?

```
<head>
  <style>
    i+b {
      color: red
    }
    .nivel1 {
      color: green
    }
    .nivel2 {
      color: blue
    }
    ul ul .nivel2 {
      font-size: x-small
    }
    ul ul li {
      font-size: x-small;
      color: red
    }
  </style>
</head>
<body>
  <h1>
    <p> <i>Título</i>: <b> Anidamiento y agrupamiento </b> de selectores </p>
  </h1>
  <ul>
    <li class="nivel1"> Agrupamiento</1i>
    <li class="nivell"> Anidamiento </li>
    <ul>
      <li class="nivel2"> Anidamiento de hijos</li>
      <li class="nivel2"> Anidamiento de adyacentes </li>
      <li> Anidamiento común</li>
    </ul>
```

```
  </ul>
</body>
```

3.7 **Unidades de medida**

Las medidas en CSS se emplean, entre otras, para definir la altura, anchura y márgenes de los elementos y para establecer el tamaño de letra del texto. Todas las medidas se indican como un valor numérico entero o decimal seguido de una unidad de medida (sin ningún espacio en blanco entre el número y la unidad de medida).

Unidad	Tipo	Relativa a
px	Absoluta	Un tamaño fijo en píxeles.
%	Relativa	Al tamaño del elemento contenedor.
em	Relativa	Al tamaño de fuente del elemento **padre** (se multiplica en anidamientos).
rem	Relativa	Al tamaño de fuente del elemento **raíz** (html). Recomendada para tipografía.
vw / vh	Relativa	Al ancho o alto del **viewport** (1vw = 1% del ancho de la ventana).

Como norma práctica: usa rem para tipografía y espaciados que deban escalar con las preferencias de accesibilidad del navegador, % para anchos fluidos dentro de un contenedor, y px solo para detalles que de verdad deben ser fijos (un borde de 1px, por ejemplo).
Ejemplo de uso de “em”:

```
html {
        font-size: 32px;
    }
    p {
        font-size: 2em;
    }
    #p2 {
        font-size: 16px;
    }
    .article {
        font-size: 2em;
        /* Ahora 1em = 32px en elementos anidados */
    }
    .article p {
        /* Se hereda el font-size, así que aquí 1em = 32px */
        font-size: 0.5em;
        /*Si cambiamos a rem, cogeria el tamaño del html*/
    }
```

```
<p>Elemento p con tamaño em</p>
<p id="p2">Elemento p con tamaño 16px</p>
<div class="article">
   <p>Lorem ipsum dolor sit amet.</p>
</div>
```

3.8 Colores

Notación	Ejemplo
Palabra clave	red, transparent, currentColor
Hexadecimal	#4B3563 (o la forma corta #48F)
rgb() / rgba()	rgb(75, 53, 99) / rgba(75, 53, 99, 0.5)
hsl() / hsla()	hsl(269, 30%, 30%)

Como ya vimos, HSL es el más práctico para diseñar variaciones de un mismo tono. La sintaxis moderna de CSS incluso permite el canal alfa sin necesidad de la "a" extra: rgb(75 53 99 / 50%).

Si solo indicamos 3 caracteres, es como escribir cada número 2 vece:
p { color: #476; } <==> p { color: #447766; }

currentColor merece una mención especial: hace referencia al valor actual de color en ese mismo elemento, útil para que un borde o un icono SVG hereden automáticamente el color del texto que los acompaña.
Sin currentColor, tendrías que sincronizar a mano el color del texto y el del icono:

```
.alerta {
  color: #B23A3A;
}
.alerta svg {
  fill: #B23A3A; /* si cambias el color de arriba, hay que acordarse de cambiar esto también */
}
```

Con currentColor, el icono simplemente seguiría lo que diga color, sin repetir el valor:

```
.alerta {
  color: #B23A3A;
}
.alerta svg {
  fill: currentColor;
}
```

Si mañana cambias .alerta a un tono distinto, el icono se actualiza solo. De hecho, currentColor es el valor **por defecto** de fill y stroke en SVG cuando no se especifica nada, precisamente por esto.

Otro ejemplo:

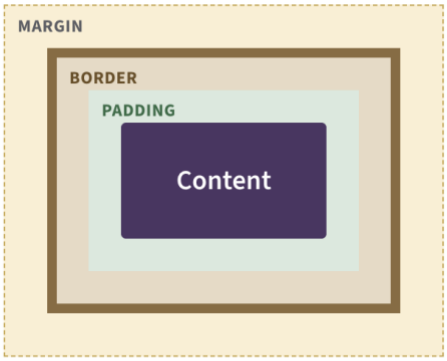
```
.btn-outline {
  color: #4B3563;
  background: transparent;
  border: 2px solid currentColor;
}

.btn-outline:hover {
  color: #FFFFFF;
  background: #4B3563;
}
```

Fíjate en el :hover: solo cambiamos color y background. Como el borde usa currentColor, pasa automáticamente de morado a blanco junto con el texto, sin necesitar una segunda declaración de border-color dentro del :hover.

3.9 **El modelo de caja**

Todo elemento HTML se representa internamente como una caja rectangular formada por cuatro capas concéntricas: contenido, relleno (padding), borde y margen.



El margen no tiene color propio (dejará ver el fondo de la página); aquí se ha coloreado solo para que las cuatro capas se distingan con claridad.

Hay un detalle del dibujo que conviene fijar: el fondo (background-color y background-image) se pinta por detrás del contenido, del relleno y también del borde, pero nunca en el margen. El margen es siempre transparente: es solo espacio vacío alrededor de la caja. Si hay imagen y color de fondo a la vez, la imagen se dibuja por encima del color.

```
<div class="caja">Contenido de la caja</div>
```

```
body { background: #F1EEE6; }
```

```css
.caja {
  background: #EFE7F2;
  border: 8px dashed #4B3563;
  padding: 16px;
  margin: 16px;
}
```



El lila del fondo se ve a través de los huecos del borde discontinuo (el fondo llega por debajo del borde) y rellena el padding, pero el margen de 16px deja ver el color de la página: el fondo no lo invade.

3.10 Operadores del selector de atributo

El selector de atributo con = ([type="email"] del apartado anterior) exige una coincidencia exacta. Para coincidencias parciales existen operadores adicionales, muy usados para distinguir enlaces según su destino sin tener que añadirle una clase a mano:

Operador	Sintaxis	Selecciona un atributo que...
Nombre atributo	[nombre_atributo]	elementos con el atributo “nombre_atributo”, da igual el valor
Nombre y valor	[nombre_atributo=valor]	elementos con el atributo “nombre_atributo”, y su valor es “valor”
Empieza por	[href^="https"]	empieza exactamente por ese texto.
Termina en	[href$=".pdf"]	termina exactamente en ese texto.
Contiene	[class*="card"]	contiene ese texto en cualquier posición.
Palabra exacta	[class~="destacado"]	contiene esa palabra completa, entre espacios (útil cuando un elemento tiene varias clases).

```html
<div class="attr-demo">
  <a href="/productos">Ver catálogo de productos</a>
  <a href="https://ejemplo-proveedor.com">Sitio de un proveedor externo</a>
  <a href="/guia-instalacion.pdf">Descargar guía en PDF</a>
</div>
```

```css
a[href^="https"] { color: red; font-weight: 700; }
a[href$=".pdf"] { background: blue; }
```

Ver catálogo de productos **Sitio de un proveedor externo** Descargar guía en PDF

Los tres enlaces comparten la misma etiqueta y no tienen ninguna clase: el segundo se resalta en rojo por empezar su href por https, y el tercero recibe un fondo por terminar en .pdf. El propio destino del enlace ya dice lo que hace falta saber.

Nota: En la práctica profesional, las clases son el selector que más se usa con diferencia: son reutilizables (a diferencia del id) y no dependen de la posición del elemento en el árbol. Reserva los id para anclas de navegación interna o para JavaScript, y los selectores de atributo para casos como el anterior, donde el propio HTML ya contiene la información que necesitas.

3.11 Pseudo-clases y pseudo-elementos

Una **pseudo-clase** selecciona un elemento según un estado o una posición que no aparece como atributo en el HTML (con solo dos puntos :hover). Un **pseudo-elemento** selecciona una parte concreta de un elemento, no el elemento completo.

Pseudo-clase	Selecciona
:hover	Mientras el cursor está encima.
:focus	Mientras el elemento tiene el foco del teclado.
:active	Mientras se está pulsando.
:first-child / :last-child	El primer o el último hijo de su contenedor.
:nth-child(n)	El hijo en la posición *n* (admite fórmulas: :nth-child(2n) selecciona los pares).
:nth-of-type(n)	El hijo en la posición *n* de un mismo tipo
:not	Selecciona elementos que NO cumplen

Estados de un enlace: el orden importa
Un enlace tiene, en realidad, cuatro posibles estados, no dos. A :hover y :active se suman otros dos, específicos de la etiqueta <a>:

Pseudo-clase	Selecciona
:link	Un enlace con href que la persona todavía no ha visitado.
:visited	Un enlace que la persona ya ha visitado antes, según el historial del navegador.
:hover	Mientras el cursor está encima (ya visto en la tarjeta anterior).
:active	Mientras se está pulsando, entre el clic y soltar el botón.

```
<a href="#">Enlace de ejemplo</a>
```

```
a:link    { color: #1E5B96; }
a:visited { color: #8B5FA6; }
a:hover   { color: #A9812E; text-decoration: underline; }
a:active  { color: #B23A3A; }
```

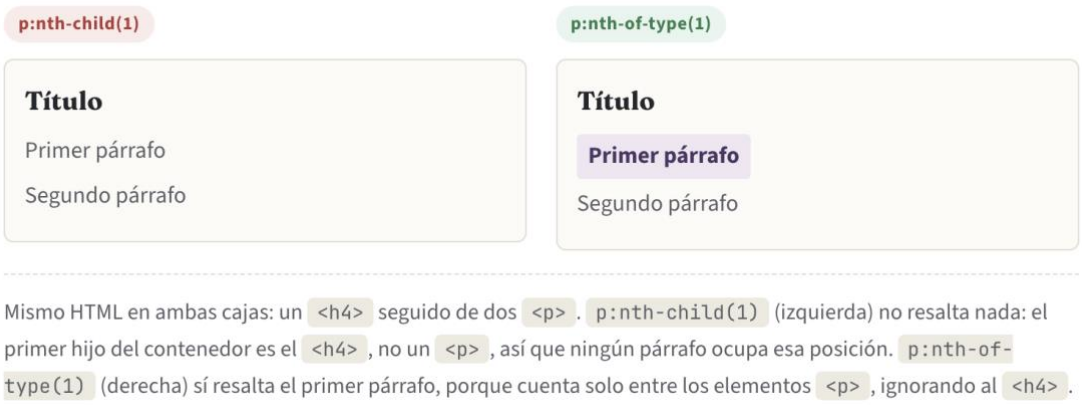
Las cuatro reglas compiten por el mismo elemento con la misma especificidad (una sola pseudo-clase cada una), así que gana la que esté escrita más abajo. Por eso el orden no es arbitrario: si :hover fuera la primera regla y :visited la última, un enlace visitado nunca podría ponerse del color de :hover, porque :visited siempre ganaría. El orden que evita este problema se recuerda con la regla mnemotécnica **LoVe HAte**: :link, :visited, :hover, :active.

:nth-child() frente a :nth-of-type()

:nth-child(n) cuenta la posición de un elemento entre **todos** sus hermanos, sean de la etiqueta que sean. :nth-of-type(n) cuenta solo entre los hermanos de su **mismo tipo de etiqueta**. Mientras todos los hermanos comparten etiqueta, da igual cuál uses; en cuanto se mezclan etiquetas distintas, es fácil llevarse una sorpresa:

```
<div>
  <h4>Título</h4>
  <p>Primer párrafo</p>
  <p>Segundo párrafo</p>
</div>
```

```
p:nth-child(1) { ... }     /* Imagen izquierda: no selecciona nada */
p:nth-of-type(1) { ... }   /* Imagen derecha: selecciona "Primer párrafo" */
```



Mismo HTML en ambas cajas: un `<h4>` seguido de dos `<p>`. `p:nth-child(1)` (izquierda) no resalta nada: el primer hijo del contenedor es el `<h4>`, no un `<p>`, así que ningún párrafo ocupa esa posición. `p:nth-of-type(1)` (derecha) sí resalta el primer párrafo, porque cuenta solo entre los elementos `<p>`, ignorando al `<h4>`.

:not(): todo menos esto

:not() es la pseudo-clase de negación: lleva dentro un selector, entre paréntesis, y selecciona los elementos que *no* lo cumplen. Sirve para expresar "todos menos...", sin tener que escribir una regla general y otra que la deshaga.

El caso clásico es una lista con una línea separadora entre elementos, pero no después del último. La forma antigua necesita dos reglas, la segunda para deshacer lo que hizo la primera:

```
<ul class="pasos">
```

```
  <li>Elige fecha</li>
  <li>Elige hora</li>
  <li>Confirma tus datos</li>
  <li>Recibe el correo</li>
</ul>
/* Sin :not(): dos reglas, la segunda deshace la primera */
.pasos li { border-bottom: 1px solid #C9BFD3; }
.pasos li:last-child { border-bottom: none; }   /* hay que deshacer la regla anterior */

/* Con :not(): una sola regla, que se lee tal cual: "todos los li menos el último" */
.pasos li:not(:last-child) { border-bottom: 1px solid #C9BFD3; }
```

El resultado es idéntico. La diferencia está en el código: con :not() no hay nada que deshacer, y si mañana añades otro elemento a la lista, la regla sigue valiendo sin tocarla. Dentro del paréntesis puedes poner casi cualquier selector sencillo. Estas son las formas más habituales:

Escribes	Selecciona
li:not(:last-child)	Todos los li que no son el último hijo.
.producto:not(.agotado)	Los productos que no llevan la clase agotado.
a:not([href^="https"])	Los enlaces cuyo href no empieza por https (por ejemplo, los internos).
input:not([type="checkbox"])	Los campos input que no son casillas.
button:not(:disabled):hover	Los botones con el cursor encima, solo si están habilitados.
p:not(.nota, .aviso)	Los párrafos que no son ni .nota *ni* .aviso.

Otro ejemplo realista: un catálogo en el que solo los productos disponibles reaccionan al pasar el cursor:

```
<div class="producto">Teclado mecánico</div>
<div class="producto agotado">Ratón inalámbrico (agotado)</div>
<div class="producto">Monitor de 27 pulgadas</div>
```

```
.producto {
  padding: 10px 14px;
  margin-bottom: 8px;
  border: 1px solid #E2DDD2;
  border-radius: 6px;
}
.producto.agotado { opacity: .5; text-decoration: line-through; }
```

```
/* Solo reaccionan los productos que NO están agotados */
.producto:not(.agotado):hover {
  border-color: #4B3563;
  background: #EFE7F2;
  cursor: pointer;
}
```

Pseudo-clases de estado en formularios

Pseudo-clase	Selecciona
:checked	Una casilla o un botón de radio marcado.
:disabled	Un campo deshabilitado.
:required	Un campo con el atributo required.
:placeholder-shown	Un campo vacío, mientras todavía se ve su placeholder.

Son la base para dar estilo a controles nativos sin una sola línea de JavaScript, por ejemplo para que la etiqueta de una casilla cambie de aspecto cuando se marca (combinando el hermano adyacente con :checked):

```
<input type="checkbox" id="c1"><label for="c1">Marca esta casilla</label>
<input type="checkbox" id="c2" disabled><label for="c2">Casilla deshabilitada</label>
```

```
input:checked + label {
  color: green;
  font-weight: 700;
}
input:disabled + label {
  opacity: 0.5;
}
```

:enabled y :disabled en detalle

:disabled selecciona los controles de formulario que no se pueden usar: aquellos con el atributo disabled.

:enabled es justo lo contrario: los que sí se pueden usar. Solo existen para controles (input, select, textarea, button, fieldset): div:enabled no selecciona nunca nada. Se usan para dos cosas: atenuar visualmente lo que no se puede usar y evitar que reaccione a :hover o al foco.

Un formulario con un campo y un botón deshabilitados:

```
<form class="reserva">
  <label for="mail">Correo</label>
  <input type="email" id="mail" value="ana@ejemplo.com">
```

```
  <label for="cod">Código promocional (caducado)</label>
  <input type="text" id="cod" value="VERANO25" disabled>

  <button type="submit">Reservar</button>
  <button type="submit" disabled>Reservar (sin plazas)</button>
</form>
```

```
.reserva input {
    display: block;
    width: 100%;
    padding: 8px 10px;
    margin: 4px 0 12px;
    border: 1px solid #4B3563;
    border-radius: 6px;
}
.reserva button {
    padding: 8px 14px;
    border: 0;
    border-radius: 6px;
    background: #4B3563;
    color: #fff;
}
/* Controles que no se pueden usar */
.reserva input:disabled {
    background: #EEE;
    color: #777;
    border-color: #CCC;
    cursor: not-allowed;
}
.reserva button:disabled {
    opacity: .45;
    cursor: not-allowed;
}
/* El efecto hover solo en los controles habilitados */
.reserva button:enabled:hover {
    background: #A9812E;
    cursor: pointer;
}
```

:checked en detalle

:checked selecciona las casillas (type="checkbox"), los botones de radio (type="radio") y las opciones de una lista desplegable (<option>) que están **marcados en este momento**. Lo importante es esa última parte: no mira el atributo checked que escribiste en el HTML, sino el estado actual del control, que cambia cuando la persona hace clic. Por eso el CSS reacciona al instante, sin ningún código de programación.

La técnica de base es siempre la misma: el input va *antes* que su label en el HTML, y se usa el hermano adyacente (+) para dar estilo a la etiqueta según el estado del control. Con ella se construyen controles completamente personalizados. Primero, tarjetas para elegir una opción (botones de radio):

```
<div class="planes">
  <div class="plan">
    <input type="radio" name="plan" id="p-basico" checked>
    <label for="p-basico">Básico <span>5 GB · 4 € al mes</span></label>
  </div>
  <div class="plan">
    <input type="radio" name="plan" id="p-pro">
    <label for="p-pro">Pro <span>100 GB · 9 € al mes</span></label>
  </div>
  <div class="plan">
    <input type="radio" name="plan" id="p-max">
    <label for="p-max">Máximo <span>1 TB · 19 € al mes</span></label>
  </div>
</div>
```

```
.plan { position: relative; margin-bottom: 8px; }

/* El control nativo se oculta visualmente, pero sigue ahí */
.plan input { position: absolute; opacity: 0; }

.plan label {
  display: block;
  padding: 10px 14px;
  border: 2px solid #E2DDD2;
  border-radius: 8px;
  cursor: pointer;
}
.plan label span { display: block; font-size: .85em; color: #5B5D63; }

.plan input:checked + label {          /* la tarjeta del plan elegido */
  border-color: #4B3563;
  background: #EFE7F2;
```

```
}
```

Segundo, un interruptor (*switch*) hecho con una casilla. Aquí usamos dos pseudo-elementos (::before y ::after, vistos antes en este apartado) para dibujar el carril y la bolita, y :checked decide su color y su posición:

```
<div class="interruptor">
  <input type="checkbox" id="notif">
  <label for="notif">Recibir avisos por correo</label>
</div>
```

```
.interruptor { position: relative; }
.interruptor input { position: absolute; opacity: 0; }

.interruptor label { position: relative; display: inline-block; padding-left: 54px; line-height: 24px; cursor:
pointer; }

/* El carril del interruptor */
.interruptor label::before {
  content: "";
  position: absolute; left: 0; top: 0;
  width: 44px; height: 24px;
  border-radius: 12px;
  background: #BDB4C7;
}
/* La bolita */
.interruptor label::after {
  content: "";
  position: absolute; left: 3px; top: 3px;
  width: 18px; height: 18px;
  border-radius: 50%;
  background: #fff;
}

.interruptor input:checked + label::before { background: #4B3563; }
.interruptor input:checked + label::after { left: 23px; }
```

Importante: Para ocultar el control nativo usa position: absolute; opacity: 0; nunca display: none. Un elemento con display: none desaparece también para el teclado y para los lectores de pantalla: la persona que no usa el ratón ya no podría marcar la casilla. Con opacity: 0 el control sigue existiendo, recibe el foco y responde al teclado.

:target: el elemento al que apunta la URL
Un enlace como <a href="#paso2"> no lleva a otra página: lleva al elemento de esta página cuyo id sea paso2, y añade #paso2 al final de la dirección (esa parte se llama fragmento). Mientras la dirección termine en #paso2, ese elemento es el destino (target) y la pseudo-clase :target lo selecciona. Cuando se pulsa otro enlace, el destino cambia y la regla pasa al nuevo elemento. Así de simple: no hace falta ninguna clase ni ningún código para marcar "dónde estoy".

El primer uso, el más habitual, es resaltar la sección a la que se ha saltado desde un índice. Este es el HTML (los id de las secciones coinciden con los fragmentos de los enlaces):

```
<nav class="indice">
  <a href="#paso1">Paso 1</a>
  <a href="#paso2">Paso 2</a>
  <a href="#paso3">Paso 3</a>
</nav>

<section id="paso1" class="paso">
  <h4>Paso 1: Preparar</h4>
  <p>Reúne los ingredientes.</p>
</section>
<section id="paso2" class="paso">
  <h4>Paso 2: Mezclar</h4>
  <p>Combina todo en un bol.</p>
</section>
<section id="paso3" class="paso">
  <h4>Paso 3: Hornear</h4>
  <p>Cuarenta minutos a 180 grados.</p>
</section>
```

```
.paso {
  padding: 8px 14px;
  margin: 8px 0;
  border-left: 4px solid #E2DDD2;
  background: #fff;
}
.paso:target {                  /* el paso al que apunta la URL */
  border-left-color: #A9812E;
  background: #EFE7F2;
}
```

El segundo uso es mostrar y ocultar contenido sin ningún código de programación: un panel solo se ve cuando es el destino. Es el truco de las "pestañas con CSS puro":

```html
<div class="pestanas">
  <nav>
    <a href="#descripcion">Descripción</a>
    <a href="#ingredientes">Ingredientes</a>
    <a href="#opiniones">Opiniones</a>
  </nav>

  <p class="inicio">Elige una pestaña.</p>

  <div id="descripcion" class="panel">Una tarta sencilla, ideal para merendar.</div>
  <div id="ingredientes" class="panel">Harina, huevos, azúcar y mantequilla.</div>
  <div id="opiniones" class="panel">Valoración media: 4,7 sobre 5.</div>
</div>
```

```css
.panel { display: none; }            /* ninguno se ve por defecto */
.panel:target { display: block; }    /* solo el que es destino de la URL */

/* Mientras haya un destino dentro, oculta el mensaje inicial */
.pestanas:has(:target) .inicio { display: none; }
```

Pseudo-elementos: las partes de un elemento

Hasta aquí hemos usado pseudo-clases, que seleccionan un elemento *real* cuando está en cierto estado (a:hover, input:checked). Un pseudo-elemento es otra cosa: da estilo a una parte de un elemento (su primera letra, su viñeta, el texto seleccionado) o a una caja generada por CSS que no existe en el HTML. Se escribe con dos dos puntos (::) para que no se confunda con una pseudo-clase.

	Pseudo-clase (:)	**Pseudo-elemento (::)**
Qué selecciona	Un elemento real en un estado o posición concretos	Una parte de un elemento, o contenido que genera CSS
Ejemplo	a:hover, li:first-child	p::first-letter, li::marker
¿Existe en el HTML?	Sí, es una etiqueta del documento	No: lo crea o lo delimita el navegador
Especificidad	Como una clase (0,1,0)	Como una etiqueta (0,0,1)

Estos son los nueve pseudo-elementos que debes dominar. Los dos primeros se explican a fondo justo a continuación; los demás, en los apartados que siguen:

Pseudo-elemento	**Qué estiliza**
::before	Una caja generada **al principio** del contenido del elemento

Pseudo-elemento	Qué estiliza
::after	Una caja generada **al final** del contenido del elemento
::placeholder	El texto de ayuda de un campo de formulario vacío
::selection	El texto que el usuario selecciona con el ratón
::marker	La viñeta o el número de un elemento de lista
::first-line	La primera línea de texto de un bloque
::first-letter	La primera letra del texto de un bloque
::file-selector-button	El botón de un <input type="file">

Nota: CSS tiene más pseudo-elementos de los que verás en este curso: unos para componentes web (::slotted(), ::part()), otros para subtítulos de vídeo (::cue), para transiciones entre páginas (::view-transition) o para carruseles hechos solo con CSS (::scroll-marker), y algunos con prefijo de navegador, como ::-webkit-scrollbar. Muchos son recientes y su soporte varía: comprueba siempre caniuse.com antes de usar uno en un proyecto real.

::before y ::after: contenido generado por CSS
Son los pseudo-elementos más potentes porque **crean una caja nueva** dentro del elemento sin tocar el HTML. ::before se inserta como *primer hijo* del elemento, antes de su contenido; ::after, como *último hijo*, después. Para el navegador, un párrafo con los dos queda así:

```
<p class="aviso">
  ::before      <!-- caja generada por CSS -->
  Texto del párrafo
  ::after       <!-- caja generada por CSS -->
</p>
```

Fíjate que van **dentro** del elemento, no a su lado: quedan rodeados por su padding y su border y heredan su color, su fuente y su tamaño de letra. Lo vemos pintando cada caja de un color distinto:

```
<p class="aviso">Texto del párrafo</p>
```

```
.aviso {
  padding: 12px;
  border: 2px solid #4B3563;
  background: #fff;
}
```

```css
.aviso::before {
  content: "::before";
  margin-right: 8px;
  padding: 2px 6px;
  background: #EFE7F2;
  color: #4B3563;
  font-size: .8rem;
}

.aviso::after {
  content: "::after";
  margin-left: 8px;
  padding: 2px 6px;
  background: #F4E9C9;
  color: #6B4F12;
  font-size: .8rem;
}
```

Sin content, el pseudo-elemento no existe
Esta es la regla de oro y la causa de la mayoría de errores. Un ::before o un ::after solo se genera si tiene la propiedad content, aunque sea una cadena vacía (content: ""). Sin ella no se dibuja nada, por mucho fondo, tamaño o posición que le des.
Qué valores puede tener content

Valor	Qué genera	Ejemplo
Cadena de texto	Ese texto	content: "Nuevo: "
Cadena vacía	Una caja sin texto, para decorarla	content: ""
attr(nombre)	El valor de un atributo del propio elemento	content: attr(href)
counter(nombre)	El valor de un contador CSS (numeración automática)	content: counter(paso)
open-quote, close-quote	Las comillas definidas con la propiedad quotes	content: open-quote
url(...)	Una imagen (no se puede controlar su tamaño)	content: url(icono.svg)
none	Nada: el pseudo-elemento no se genera	content: none

Su caja es inline por defecto

Un pseudo-elemento generado se comporta como un elemento de línea: ignora width y height y no admite márgenes verticales. Para darle tamaño hay que cambiar su display (inline-block o block) o posicionarlo con position: absolute, que lo convierte en bloque automáticamente.

Ejemplo: subrayado animado
Aquí el ::after es una línea de 2 px que ocupa todo el ancho del enlace pero está encogida a cero con scaleX(0); al pasar el cursor, la transición la hace crecer desde la izquierda. Sin JavaScript y sin etiquetas extra:

```
<nav class="menu">
  <a href="#inicio">Inicio</a>
  <a href="#cursos">Cursos</a>
  <a href="#contacto">Contacto</a>
</nav>
```

```
.menu a {
  position: relative;
  margin-right: 24px;
  color: #4B3563;
  text-decoration: none;
}

.menu a::after {
  content: "";
  position: absolute;
  left: 0;
  bottom: -3px;
  width: 100%;
  height: 2px;
  background: #A9812E;
  transform: scaleX(0);          /* línea encogida */
  transform-origin: left;        /* crece desde la izquierda */
  transition: transform .3s ease;
}

.menu a:hover::after,
.menu a:focus-visible::after {
  transform: scaleX(1);          /* línea completa */
}
```

::placeholder y ::selection
::placeholder da estilo al texto de ayuda de un campo vacío.

::selection, al fragmento de texto que la persona usuaria selecciona con el ratón, en lugar del azul por defecto del navegador.

```
<input type="text" class="ph-demo" placeholder="Escribe tu correo electrónico">
<p class="sel-demo">Selecciona parte de este texto...</p>
```

```
.ph-demo::placeholder { color: var(--gold); font-style: italic; }
.sel-demo::selection { background: var(--accent); color: #fff; }
```

::marker

Da estilo a la propia viñeta o número de un elemento de lista, algo que hasta hace poco exigía rodeos con list-style-image o con ::before sobre cada <li>:

```
<ul>
  <li>Primer punto de la lista</li>
  <li>Segundo punto de la lista</li>
</ul>
```

```
li::marker {
  color: var(--gold);
  font-weight: 700;
}
```

::first-line y ::first-letter

Dos pseudo-elementos pensados específicamente para texto. ::first-line selecciona la primera línea renderizada de un elemento de bloque: no cuenta caracteres, sino lo que ocupe esa primera línea en pantalla, así que cambia sola si el ancho del contenedor cambia. ::first-letter selecciona solo el primer carácter.

```
<p class="fl-demo">
  El buen diseño parece obvio una vez hecho, pero exige muchas decisiones invisibles antes de llegar ahí.
</p>
```

```
.fl-demo::first-line {
  font-weight: 700;
}
.fl-demo::first-letter {
  font-size: 2.2em;
  color: var(--accent);
  font-weight: 700;
}
```

::file-selector-button

El campo <input type="file"> es famoso por su aspecto feo y por ser casi imposible de personalizar. Hoy, el pseudo-elemento ::file-selector-button da estilo al **botón** que lleva dentro (el que abre el explorador de archivos), sin trucos de JavaScript ni ocultar el campo. Antes se escribía con un prefijo propio (::-webkit-file-upload-button), pero ya no hace falta.

```
<label for="foto">Sube tu foto de perfil</label><br>
<input type="file" id="foto" class="subida" accept="image/*">
```

```
.subida {
  font: inherit;
  color: #2B2233;
}

.subida::file-selector-button {
  margin-right: 12px;
  padding: 8px 16px;
  border: 0;
  border-radius: 6px;
  background: #4B3563;
  color: #fff;
  font: inherit;
  font-weight: 700;
  cursor: pointer;
  transition: background .2s;
}

.subida::file-selector-button:hover {
  background: #A9812E;
}
```

::backdrop

Cuando un elemento se muestra en la **capa superior** (*top layer*) del navegador, por encima de todo lo demás, CSS genera tras él una caja que cubre toda la ventana: el ::backdrop. El caso típico es el cuadro de diálogo modal, <dialog> abierto con showModal(). Por defecto el fondo es un velo gris casi transparente; con ::backdrop lo personalizamos, incluso con desenfoque (backdrop-filter):

```
<button id="abrir">Abrir diálogo</button>

<dialog id="dialogo">
  <p><strong>¿Eliminar el curso?</strong></p>
  <p>Esta acción no se puede deshacer.</p>
  <form method="dialog">
    <button>Cancelar</button>
    <button value="ok">Eliminar</button>
  </form>
</dialog>
```

```
<script>
  document.querySelector("#abrir").addEventListener("click", () => {
    document.querySelector("#dialogo").showModal();
  });
</script>
```

```
dialog {
  padding: 24px;
  border: 0;
  border-radius: 10px;
  box-shadow: 0 10px 40px rgba(0, 0, 0, .35);
}

dialog::backdrop {
  background: rgba(75, 53, 99, .6);
  backdrop-filter: blur(3px);
}
```

Veamos tres usos reales, de menor a mayor complejidad.

1. Un acento decorativo antes de un título

Una barra de color a la izquierda, generada por CSS, sin necesitar un extra en el HTML.

```
h2::before {
  content: "";
  display: inline-block;
  width: 4px;
  height: 1em;
  background: #4B3563;
  margin-right: 8px;
  vertical-align: -2px;
}
```

Aquí content: "" no añade texto, solo "activa" el pseudo-elemento para poder darle forma de barrita con width/height/background.

2. Un asterisco automático en campos obligatorios

```
label.required::after {
    content: " *";
    color: #B23A3A;
    font-weight: 700;
```

```
}
```

Ventaja real: si mañana cambias de "obligatorio con asterisco" a "obligatorio con la palabra (requerido)", cambias una sola regla CSS y se actualiza en todos los formularios del sitio, sin tocar el HTML.

3. Comillas automáticas alrededor de una cita

```
blockquote::before {
    content: "“";
    font-size: 2rem;
    color: #A9812E;
}

blockquote::after {
    content: "”";
    font-size: 2rem;
    color: #A9812E;
}
```

Ejemplo completo:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        /* Ejemplo 1: acento decorativo antes de un título */
        h2::before {
            content: "";
            display: inline-block;
            width: 4px;
            height: 1em;
            background: #4B3563;
            margin-right: 8px;
            vertical-align: -2px;
        }

        /* Ejemplo 2: asterisco automático en campos obligatorios */
label.required::after {
```

```
    content: " *";
    color: #B23A3A;
    font-weight: 700;
}
        /* Ejemplo 3: comillas automáticas alrededor de una cita */
blockquote::before {
    content: "“";
    font-size: 2rem;
    color: #A9812E;
}

blockquote::after {
    content: "”";
    font-size: 2rem;
    color: #A9812E;
}
    </style>
</head>

<body>
    <h2>Cómo elegir una retícula</h2>

    <form>
        <label class="required" for="email">Correo electrónico</label><br>
        <input type="email" id="email">
    </form>

    <blockquote>
        El buen diseño es tan poco diseño como sea posible.
    </blockquote>
</body>

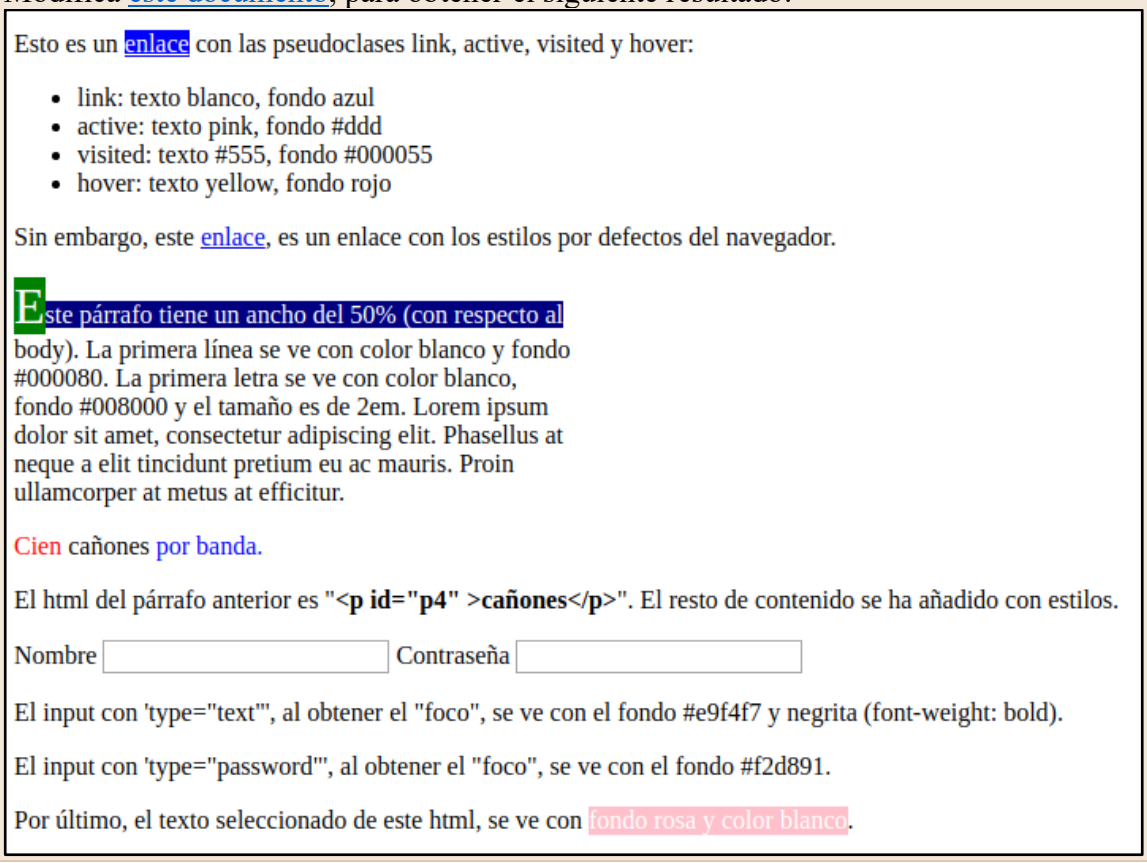
</html>
```

Cosas importantes:

- **Son inline por defecto**, como un . Si quieres darles un tamaño concreto (como en el ejemplo de la barrita), necesitas cambiar display a inline-block o block.
- **No funcionan en elementos sin contenido**: <img>, <input>,
... En estos elementos el navegador no tiene un "contenido" interno al que anteponer o añadir nada, así que ::before/::after se ignoran silenciosamente.

Ejercicio 4· Selectores

Modifica este documento, para obtener el siguiente resultado:



Estos tres son selectores modernos, con muy buen soporte en navegadores actuales, que resuelven problemas muy concretos de los selectores "clásicos" que ya conoces.

:is(): agrupa varios selectores en uno

Sin :is(), si quieres aplicar el mismo estilo a un mismo elemento dentro de varios contenedores distintos, tienes que repetir la regla entera:

```
header a,
nav a,
footer a {
  text-decoration: none;
}
```

Con :is(), escribes la parte que varía una sola vez:

```
:is(header, nav, footer) a {
  text-decoration: none;
}
```

Ambas reglas hacen exactamente lo mismo. La ventaja crece con selectores más largos o anidados:

```
/* Sin :is() */
article h2, article h3, article h4,
section h2, section h3, section h4 {
```

```
  color: #4B3563;
}

/* Con :is() */
:is(article, section) :is(h2, h3, h4) {
  color: #4B3563;
}
```

Especificidad: :is() toma la especificidad del selector *más específico* de su lista. Si escribes :is(#sidebar, .card) p, todo el selector se calcula como si llevara un id (aunque .card por sí sola valga menos), lo cual puede sorprenderte al depurar conflictos de cascada.

:where(): igual que :is(), pero con especificidad cero
:where() selecciona exactamente los mismos elementos que :is(). La única diferencia es que, sea cual sea su contenido, **su especificidad siempre vale cero**:

```
:where(header, nav, footer) a {
  text-decoration: none;
}
```

Esta regla se comporta, a efectos de especificidad, como si no existiera ningún selector: cualquier otra regla, por simple que sea (incluso a { } a secas, si aparece después), la sobrescribe sin esfuerzo.

¿Cuándo usar cuál? :where() es la opción habitual para estilos base o de librería, pensados para que cualquiera pueda sobreescribirlos después sin pelearse con la especificidad. :is() es la opción por defecto para el resto de casos, donde sí quieres que el selector "cuente" con su peso normal.

:has(): selecciona según lo que un elemento contiene
Este es el más nuevo de los tres, y durante años fue imposible en CSS: seleccionar un elemento padre en función de lo que hay dentro de él (por eso se le llamó informalmente el "selector padre").

```
/* Selecciona solo las tarjetas que incluyen una imagen */
.card:has(img) {
  padding-top: 0;
}
```

```
/* Selecciona solo los formularios que tienen algún campo inválido */
form:has(:invalid) {
  border: 1px solid #B23A3A;
}
```

```
/* Resalta la etiqueta de un checkbox marcado, sin usar JavaScript */
label:has(input:checked) {
  background: #E9F5EE;
```

```
  font-weight: 600;
}
```

Ese último ejemplo es el más revelador del poder de :has(): antes hacía falta JavaScript (o el hack de CSS de poner el <input> antes que el <label> y usar el combinador hermano adyacente +) para conseguir algo así. Con :has(), la relación se expresa de forma directa y funciona sin importar el orden del HTML.

:empty: selecciona un elemento que no tiene absolutamente nada dentro: ni texto, ni otros elementos.

```
<p class="aviso"></p>
```

```
.aviso:empty {
  display: none;
}
```

El uso típico es ocultar un contenedor que a veces llega vacío (un aviso, un bloque de resultados, un párrafo generado por un gestor de contenidos) para que no deje un hueco de margen o relleno sin nada dentro.

La trampa: qué cuenta como "vacío"
El resultado no es el que mucha gente espera:

HTML	¿Coincide con :empty?
	Sí
<!-- comentario -->	Sí (los comentarios no cuentan)
<img>, 	Sí (no pueden tener contenido, así que siempre están vacíos)
(un espacio)	No
+ salto de línea +	No
	No (tiene un hijo, aunque el hijo esté vacío)
texto	No

3.12 Repasando propiedades

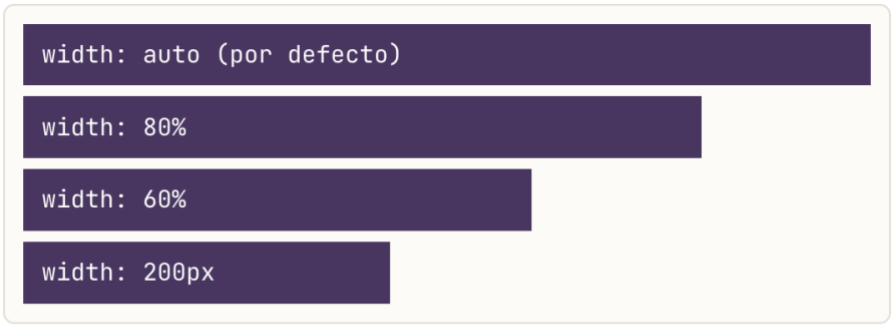
Anchura y altura: width y height
Fijan el tamaño del área de contenido. Comparten las mismas reglas:

Aspecto	Qué hay que saber
Valores	Una unidad de medida (200px, 20rem), un porcentaje o auto.

Valor inicial	auto: el navegador calcula el tamaño según el contenido y el espacio disponible.
Negativos	No se admiten.
Porcentajes	Se calculan sobre el ancho (o el alto) del contenedor, no sobre el propio elemento.
A quién se aplican	A todos los elementos salvo los elementos en línea que no sean imágenes: un ignora su width.

```
<div class="barra">width: auto (por defecto)</div>
<div class="barra ancho-80">width: 80%</div>
<div class="barra ancho-60">width: 60%</div>
<div class="barra ancho-200">width: 200px</div>
```

```
* { box-sizing: border-box; }
.barra { background: #4B3563; color: #fff; padding: 6px 10px; margin-bottom: 6px; }
.ancho-80 { width: 80%; }
.ancho-60 { width: 60%; }
.ancho-200 { width: 200px; }
```



Un elemento de bloque con `auto` ocupa todo el ancho de su contenedor. Los porcentajes se reparten sobre ese contenedor y cambian si lo hace su ancho (redimensiona la ventana y compruébalo); los 200px se quedan en 200px siempre.

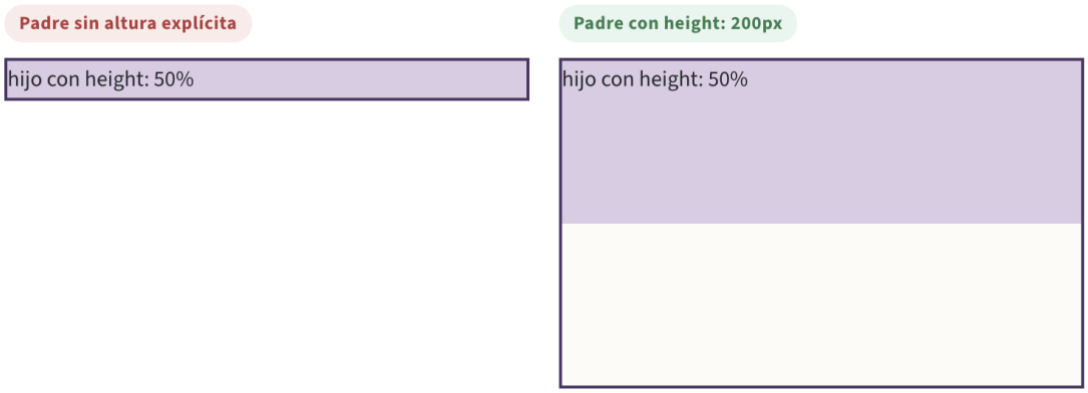
La altura en porcentaje: un clásico

Aquí está la trampa más habitual de height: cuando se da en porcentaje, la referencia es la altura del contenedor, pero si ese contenedor no tiene una altura explícita (es decir, su height es auto), el navegador ignora el porcentaje y usa auto también.

```
<div class="padre">
  <div class="hijo">hijo con height: 50%</div>
</div>
```

```html
<div class="padre padre-alto">
  <div class="hijo">hijo con height: 50%</div>
</div>
```

```css
.padre { border: 2px solid #4B3563; }
.padre-alto { height: 200px; }
.hijo { height: 50%; background: #D9CBE3; }
```



A la izquierda el hijo mide lo que mide su texto: el 50% de "auto" no se puede calcular. A la derecha mide exactamente 100px, la mitad de los 200px del padre.

El caso más frecuente es el de un dentro de <body> con height: 50% o 100%: no obtiene la mitad (ni el total) de la ventana, porque body y html tampoco tienen altura explícita. Para que funcione, toda la cadena de contenedores hasta html necesita una altura (html, body { height: 100%; }), aunque hoy se prefiere min-height: 100vh o 100dvh.

Límites de tamaño: min-width, max-width, min-height y max-height
width y height fijan un tamaño. Pero una página web no tiene un tamaño fijo: se ve en una ventana de escritorio, en una tableta o en un móvil, y su contenido cambia. Lo que normalmente quieres decir no es "esta caja mide 600 px", sino "esta caja puede crecer o encogerse *dentro de unos límites*". Para eso existen cuatro propiedades que ponen un suelo y un techo al tamaño:

Propiedad	Qué hace	Valor inicial
min-width	Ancho **mínimo**: la caja nunca será más estrecha, aunque width o el contenido pidan menos.	0 (sin mínimo)
max-width	Ancho **máximo**: la caja nunca será más ancha, aunque width pida más.	none (sin máximo)
min-height	Altura **mínima**: la caja nunca será más baja, pero puede crecer si el contenido lo necesita.	0 (sin mínimo)

max-height	Altura **máxima**: la caja nunca será más alta; si el contenido no cabe, se sale (apartado 8.13) a menos que añadas overflow.	none (sin máximo)

Aceptan las mismas unidades que width y height (píxeles, porcentajes sobre el contenedor, rem, vw, vh...) y, como ellas, no admiten valores negativos. Y se calculan sobre el mismo área que width: el contenido con content-box, la caja entera con border-box.

Cómo se resuelve cuando hay varios valores

El navegador calcula el tamaño en tres pasos, siempre en este orden: primero toma el width (o el que resulte de auto); después, si ese valor supera el max-width, lo reduce a ese máximo y por último, si el resultado es menor que el min-width, lo sube a ese mínimo. Una consecuencia importante: **el mínimo gana al máximo**. Si escribes min-width: 200px; max-width: 150px, la caja mide 200 px. Lo mismo ocurre con la altura.

Veámoslo con cuatro barras dentro de un escenario cuya anchura puedes cambiar tú. Este es el HTML y el CSS:

```
<div class="escenario">
  <div class="tira libre">width: 50%</div>
  <div class="tira con-min">width: 50% y min-width: 200px</div>
  <div class="tira con-max">width: 50% y max-width: 240px</div>
  <div class="tira fluida">width: 100% y max-width: 420px</div>
</div>
```

```
.tira {
  background: #4B3563;
  color: #fff;
  padding: 8px 10px;
  margin-bottom: 8px;
}
.libre    { width: 50%; }
.con-min  { width: 50%; min-width: 200px; }
.con-max  { width: 50%; max-width: 240px; }
.fluida   { width: 100%; max-width: 420px; }
```

El escenario es un contenedor redimensionable, su CSS es solo para poder arrastrarlo (la propiedad resize añade una esquina que se puede arrastrar con el ratón:

```
.escenario {
  border: 2px dashed #A9812E;
  padding: 10px;
  overflow: hidden;
  resize: horizontal;      /* permite arrastrar la esquina inferior derecha */
  box-sizing: border-box;
  width: 100%;
  min-width: 160px;
}
```

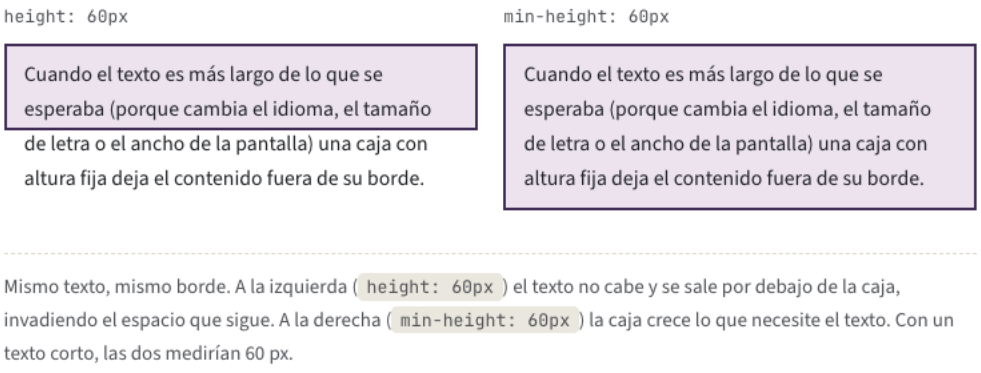
Importante: La combinación width: 100%; max-width: 600px; (casi siempre con margin: 0 auto) es el patrón más usado de todo el CSS: una columna de contenido que se adapta a pantallas pequeñas y no se estira de forma absurda en pantallas grandes. Compárala con width: 600px a secas, que se sale de la pantalla en cualquier móvil de menos de 600 px de ancho.

min-height: una altura que puede crecer

Dar un height fijo a una caja con texto es una apuesta: funciona hasta que el texto es más largo de lo previsto, y entonces se sale de su borde. min-height expresa la intención correcta: "al menos tan alta como esto, pero crece si hace falta". Dos avisos con el mismo texto largo:

```
<div class="aviso aviso-alto">Este aviso tiene height: 60px.</div>
<div class="aviso aviso-min">Este aviso tiene min-height: 60px.</div>
```

```
.aviso {
  padding: 8px 12px;
  border: 2px solid #4B3563;
  background: #EFE7F2;
  font-size: .9rem;
}
.aviso-alto { height: 60px; }       /* altura fija: el contenido puede salirse */
.aviso-min  { min-height: 60px; }   /* altura mínima: la caja crece si hace falta */
```



max-height: un techo con scroll

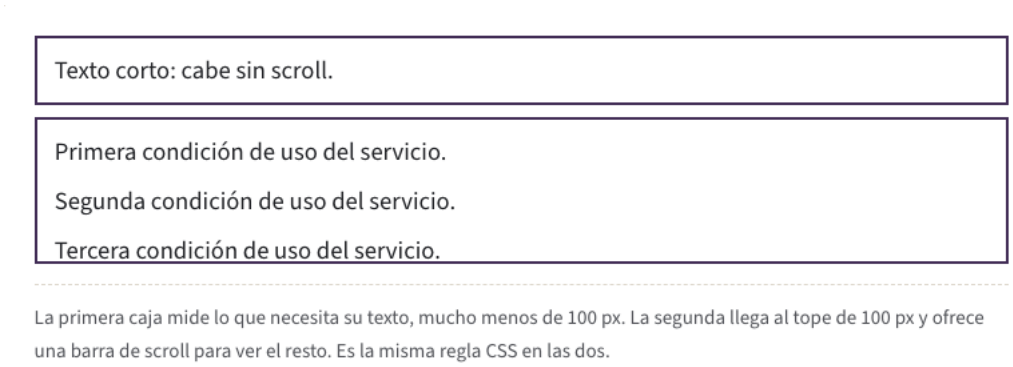
max-height se combina casi siempre con overflow: auto para hacer paneles con altura limitada: un cuadro de condiciones de uso, una lista larga dentro de un menú, un registro de mensajes. La caja solo llega al máximo cuando el contenido lo supera, si es corto, la caja es corta (algo que un height fijo no consigue):

```
<div class="terminos">Texto corto: cabe sin scroll.</div>


<div class="terminos">
  <p>Primera condición de uso del servicio.</p>
  <p>Segunda condición de uso del servicio.</p>
  <p>Tercera condición de uso del servicio.</p>
```

```
  <p>Cuarta condición de uso del servicio.</p>
  <p>Quinta condición de uso del servicio.</p>
  <p>Sexta condición de uso del servicio.</p>
</div>
```

```
.terminos {
  max-height: 100px;     /* tope: nunca más alta de 100px */
  overflow: auto;        /* si el contenido no cabe, aparece una barra de scroll */
  padding: 8px 12px;
  border: 2px solid #4B3563;
  margin-bottom: 8px;
}
.terminos p { margin: 0 0 6px; }
```



min-height: 100vh: la página entera

Un contenedor con height: 100% no llega hasta abajo si sus padres no tienen altura (la trampa del apartado anterior). La alternativa fiable es min-height con la unidad vh (1 vh es el 1 % del alto de la ventana):

```
body { min-height: 100vh; }        /* el body llega al menos hasta el pie de la ventana */
```

Con min-height y no height, el body ocupa al menos toda la ventana (así el pie de página puede quedar abajo en páginas cortas), pero crece sin problema cuando el contenido es más largo.

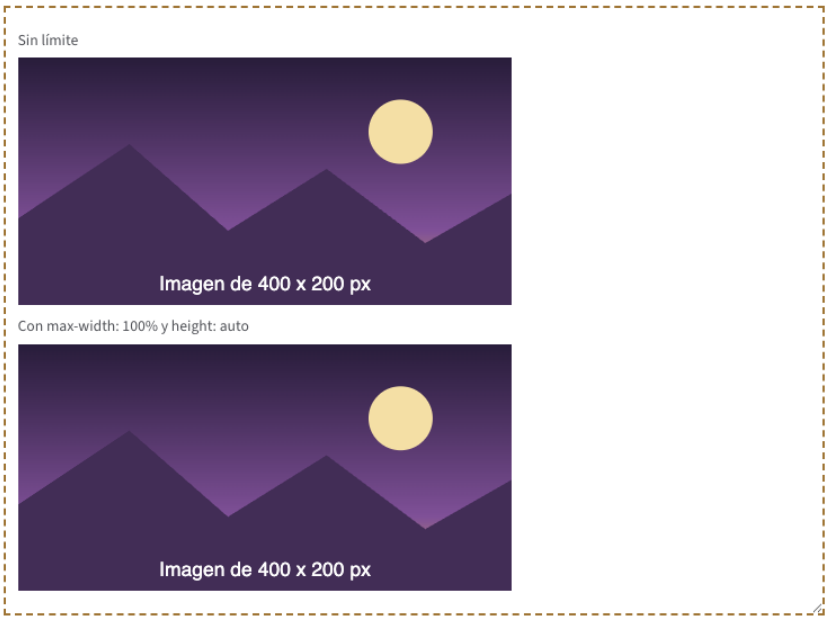
max-width: 100% en las imágenes

Una imagen mide, por defecto, su tamaño real, y no se adapta al contenedor: si el contenedor es más estrecho, la imagen se sale. La regla max-width: 100% la limita al ancho del contenedor, y height: auto hace que la altura siga la proporción original para que no se deforme. Se diferencia de width: 100% en que esta última agrandaría también las imágenes pequeñas; max-width solo las encoge cuando hace falta. Este es el HTML y el CSS:

```
<img class="sin-limite" src="paisaje.svg" alt="Paisaje de 400 por 200 píxeles">
<img class="con-limite" src="paisaje.svg" alt="Paisaje de 400 por 200 píxeles">
```

```
.con-limite {
```

```
  max-width: 100%;      /* nunca más ancha que su contenedor... */
  height: auto;         /* ...y la altura sigue la proporción */
}
```



Nota: Es habitual encontrar, al principio de las hojas de estilo profesionales, una regla como img { max-width: 100%; height: auto; } que aplica este comportamiento a todas las imágenes de la web. Cuando veamos responsive verás cómo encaja en una estrategia completa para pantallas de cualquier tamaño.

El tamaño real de una caja

Aquí está el error de cálculo más común del modelo de caja. Por defecto (box-sizing: content-box), width y height miden solo el contenido. El relleno, el borde y el margen se suman por fuera, de modo que el ancho final de la caja no son los 300px que escribiste:

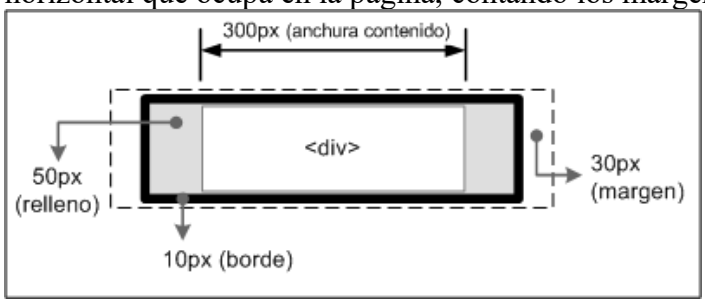
```
<div class="caja-contenido">Contenido</div>
```

```
.caja-contenido {
  box-sizing: content-box;   /* es el valor por defecto */
  width: 300px;
  padding: 0 50px;
  margin: 0 30px;
  border: 10px solid #4B3563;
}
```

Margen	Borde	Relleno	Contenido	Relleno	Borde	Margen

30px	10px	50px	**300px**	50px	10px	30px

La caja visible (de borde a borde) mide 10 + 50 + 300 + 50 + 10 = 420px. El espacio horizontal que ocupa en la página, contando los márgenes, es 30 + 420 + 30 = 480px.



box-sizing: content-box vs. border-box
Con box-sizing: border-box, el relleno y el borde se descuentan del ancho que declaras, así que width pasa a ser el ancho visible de la caja. El mismo ejemplo anterior, cambiando solo esa propiedad:

```
<div class="caja-borde">Contenido</div>
```

```
.caja-borde {
  box-sizing: border-box;
  width: 300px;
  padding: 0 50px;
  margin: 0 30px;
  border: 10px solid #A9812E;
}
```

Contenido (width: 300px, ahora es el ancho visible)

La caja visible mide ahora exactamente 300px (180px de contenido + 100px de relleno + 20px de borde) y, con los márgenes, ocupa 360px. Es lo que esperas casi siempre cuando escribes `width: 300px`.

Por eso se suele activar de forma global, en todos los elementos, al principio de la hoja de estilos:

```
/* Práctica estándar en cualquier proyecto moderno */
* {
  box-sizing: border-box;
}
```

Importante: Aplica box-sizing: border-box de forma global al principio de cualquier proyecto. Con content-box (el valor por defecto), añadir un padding a un elemento con

ancho fijo desajusta la maquetación de forma poco intuitiva; con border-box, el tamaño que declaras es, siempre, el tamaño real.

3.13 Colocar elementos sin Flexbox: bloque y en línea

Flexbox y Grid llegaron para resolver la colocación de elementos con pocas líneas, pero CSS ya tenía herramientas para hacerlo antes, y siguen funcionando y apareciendo en millones de webs. Conocerlas tiene dos ventajas: sabrás leer (y arreglar) código antiguo, y entenderás por qué Flexbox y Grid existen. Esta sección recorre, caso por caso, cómo se coloca cada combinación de elementos de bloque y en línea con CSS "puro": sin Flexbox, sin Grid y sin JavaScript.

Los ejemplos de esta sección parten de este CSS base. Añádelo al principio de tu hoja de estilos antes de probarlos:

```
* {
  box-sizing: border-box;
}
body {
  font: 16px sans-serif;
  margin: 20px;
}
```

Mapa rápido: qué técnica para cada caso

Quiero...	Técnica	Apartado
Centrar un bloque horizontalmente	width + margin-left y margin-right en auto	Un elemento de bloque
Llevar un bloque a la derecha	margin-left: auto	Un elemento de bloque
Alinear lo que hay *dentro* de un bloque	text-align	Un elemento en línea
Alinear en vertical **elementos en línea** entre sí	vertical-align	Dos elementos en línea
Poner dos bloques uno junto a otro	inline-block o position: absolute	Dos elementos de bloque
Anclar algo a una esquina, o cubrir un contenedor	position: relative + absolute	Colocar con position
Mantener algo visible al desplazarse	position: sticky o fixed	Colocar con position
Centrar en horizontal *y* en vertical	absolute + transform, margin: auto o line-height	Centrar en los dos ejes

Punto de partida: el flujo normal

Si no escribes ninguna regla de posición, el navegador coloca los elementos según su tipo. Los elementos de **bloque** se apilan de arriba abajo, cada uno en su propia línea y ocupando todo el ancho de su contenedor. Los elementos **en línea** se colocan uno tras otro dentro de las líneas de texto del bloque que los contiene, y pasan a la línea siguiente cuando no caben. Todo lo que sigue consiste en modificar ese comportamiento por defecto.

Un elemento de bloque

Un bloque con width: auto (el valor por defecto) ocupa todo el ancho disponible, así que no hay nada que colocar: ya está "colocado". La cuestión aparece cuando le das un ancho menor: sobra espacio a un lado, y los márgenes laterales deciden dónde queda. Por defecto, a la izquierda. Un margen lateral en auto absorbe todo el espacio sobrante en ese lado:

```
<div class="area">
  <div class="bloque">Por defecto (izquierda)</div>
  <div class="bloque centrado">Márgenes laterales auto</div>
  <div class="bloque derecha">margin-left: auto</div>
</div>
```

```
.area { border: 1px dashed #888; padding: 8px; }
.bloque {
  width: 220px;
  padding: 8px;
  margin-bottom: 8px;
  background: #4B3563;
  color: #fff;
}
.centrado { margin-left: auto; margin-right: auto; }
.derecha { margin-left: auto; }
```



Escribimos margin-left y margin-right por separado, y no margin: 0 auto, para no anular el margin-bottom de los bloques. Un error frecuente es intentar mover un bloque con text-align. Esa propiedad alinea el contenido *en línea* que hay dentro de un elemento (texto, enlaces, imágenes), no su propia caja ni las cajas de bloque que contiene:

```
<div class="area alineada">
```

```
  <div class="bloque">La caja no se mueve; su texto sí</div>
</div>
```

```
.area { border: 1px dashed #888; padding: 8px; }
.alineada { text-align: center; }
.bloque { width: 220px; padding: 8px; background: #4B3563; color: #fff; }
```



El bloque sigue pegado a la izquierda aunque su contenedor tenga `text-align: center`: lo único que se centra es el texto que hay dentro de él.

Un elemento en línea

Un elemento en línea no tiene una caja propia que colocar: forma parte de la línea de texto de su bloque padre, y se mueve con ella. Por eso se coloca **desde el padre**, con text-align:

```
<div class="fila izq"><span class="etiqueta">en línea</span></div>
<div class="fila cen"><span class="etiqueta">en línea</span></div>
<div class="fila der"><span class="etiqueta">en línea</span></div>
```

```
.fila { border: 1px dashed #888; padding: 6px; margin-bottom: 6px; }
.izq { text-align: left; }
.cen { text-align: center; }
.der { text-align: right; }
.etiqueta { background: #A9812E; color: #fff; padding: 2px 10px; }
```



El mismo `<span>` en tres contenedores. Lo que cambia entre ellos es la propiedad `text-align` del padre, no ningún estilo del propio elemento.

Un elemento en línea también tiene limitaciones que conviene conocer.
Ignora width y height (mide lo que mida su contenido), e ignora los márgenes verticales. Sí respeta los márgenes y el relleno *laterales* (padding). El relleno vertical se dibuja, pero no empuja a las líneas vecinas. Para tener todas las propiedades de una caja sin dejar de colocarse como un elemento en línea existe display: inline-block:

```
<div class="fila">
```

```
  Antes <span class="caja-linea">en línea</span> después
</div>
<div class="fila">
  Antes <span class="caja-linea bloque-linea">inline-block</span> después
</div>
```

```
.fila { border: 1px dashed #888; padding: 6px; margin-bottom: 6px; }
.caja-linea {
  width: 160px;
  height: 50px;
  margin: 20px 30px;
  padding: 4px 8px;
  background: #A9812E;
  color: #fff;
}
.bloque-linea { display: inline-block; }
```



Los dos `<span>` tienen exactamente las mismas reglas. En la primera fila, `width` y `height` se ignoran (la caja mide lo que mide su texto) y los 20px de margen vertical no separan nada, aunque los 30px laterales sí funcionan. En la segunda, con `inline-block`, se respeta todo.

Dos elementos en línea

Dos elementos en línea se colocan uno detrás de otro, en la misma línea, mientras quepan. Entre ellos hay un hueco que no escribiste en el CSS: **el espacio en blanco del HTML** (espacios y saltos de línea) se convierte en un espacio de texto. Si no hay ni un espacio entre las dos etiquetas, quedan pegadas. Para separarlos con precisión usa márgenes laterales:

```
<div class="fila">
  <span>Uno</span> <span>Dos</span><span>Tres</span>
</div>
<div class="fila separados">
  <span>Uno</span> <span>Dos</span> <span>Tres</span>
</div>
<div class="fila estrecha">
  <span>Uno</span> <span>Dos</span> <span>Tres</span> <span>Cuatro</span>
<span>Cinco</span>
```

```
</div>
```

```
.fila { border: 1px dashed #888; padding: 6px; margin-bottom: 6px; }
.fila span { background: #4B3563; color: #fff; padding: 2px 10px; }
.separados span { margin-right: 20px; }
.estrecha { width: 200px; }
```



Primera fila: entre "Uno" y "Dos" hay un espacio en el HTML, y entre "Dos" y "Tres" no hay ninguno, por eso se tocan. Segunda: además del espacio, un margen derecho de 20px en cada uno. Tercera: cuando los elementos no caben, saltan a la línea siguiente. Para impedirlo, `white-space: nowrap` en el padre los mantiene en una sola línea (aunque se salgan del contenedor).

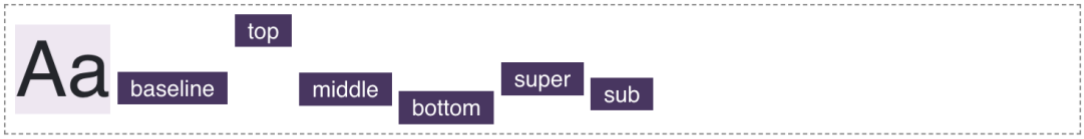
Alinear en vertical: vertical-align

Dos elementos en línea de distinto tamaño se alinean, por defecto, por su **línea base** (la línea sobre la que "se apoyan" las letras). vertical-align, que se aplica al elemento en línea (no al padre), cambia esa alineación respecto a la línea que los contiene:

```
<div class="fila">
  <span class="grande">Aa</span>
  <span class="chip">baseline</span>
  <span class="chip arriba">top</span>
  <span class="chip medio">middle</span>
  <span class="chip abajo">bottom</span>
  <span class="chip sup">super</span>
  <span class="chip inf">sub</span>
</div>
```

```
.fila { border: 1px dashed #888; padding: 6px; }
.grande { font-size: 48px; background: #EFE7F2; }
.chip {
  font-size: 14px;
  background: #4B3563;
  color: #fff;
  padding: 2px 8px;
}
.arriba { vertical-align: top; }
.medio { vertical-align: middle; }
```

```
.abajo { vertical-align: bottom; }
.sup { vertical-align: super; }
.inf { vertical-align: sub; }
```



El "Aa" grande hace la línea más alta, y deja ver cómo se alinea cada etiqueta pequeña: `baseline` (la primera, por defecto) apoya su base en la del texto; `top` y `bottom` la pegan al borde superior o inferior de la línea; `middle` la centra; `super` y `sub` la suben o la bajan, como un exponente o un subíndice.

Un caso donde vertical-align resuelve un problema real: las imágenes (<img>) son elementos en línea y se apoyan en la línea base del texto, dejando por debajo el hueco que ocuparían las letras con rabito (g, p, y). El resultado es una franja de unos píxeles bajo la imagen, que desconcierta porque no se ve ningún margen en tu CSS:

```
<div class="cuadro"><img src="foto.png" alt=""></div>
<div class="cuadro"><img class="sin-hueco" src="foto.png" alt=""></div>
```

```
.cuadro {
  display: inline-block;
  vertical-align: top;
  margin-right: 12px;
  background: #A9812E;
}
img { width: 96px; height: 64px; }
.sin-hueco { vertical-align: middle; }
```



Usa cualquier imagen en lugar de `foto.png` (aquí es un dibujo incrustado). El fondo dorado del contenedor deja ver la franja bajo la primera imagen, y desaparece en la segunda con `vertical-align: middle`. Otra forma de quitarla es `display: block` en la imagen, que la convierte en un bloque y deja de apoyarse en la línea base.

Un elemento de bloque y uno en línea

Un bloque siempre ocupa su propia línea, así que un elemento en línea que esté junto a él acaba **en otra línea**: antes o después del bloque, nunca a su lado. El navegador envuelve el contenido en línea suelto en "cajas anónimas" para poder apilarlo con el bloque, sin que escribas nada:

```
<div class="area">
  <div class="bloque">Un bloque</div>
  <span class="etiqueta">en línea, entre dos bloques</span>
```

```
  <div class="bloque">Otro bloque</div>
</div>
<div class="area">
  Texto antes <div class="bloque">bloque en medio del texto</div> texto después
</div>
```

```
.area { border: 1px dashed #888; padding: 8px; margin-bottom: 8px; }
.bloque { background: #4B3563; color: #fff; padding: 6px 10px; }
.etiqueta { background: #A9812E; color: #fff; padding: 2px 10px; }
```



En el primer contenedor, el elemento en línea queda en su propia línea, sin poder subir junto al primer bloque. En el segundo, el bloque corta el texto en dos trozos: lo de antes y lo de después quedan en líneas separadas. Para poner un bloque junto a otro elemento, el bloque tiene que dejar de comportarse como bloque (siguiente apartado).

Dos elementos de bloque

Por defecto, dos bloques se apilan, separados por sus márgenes verticales. Para colocarlos **uno junto a otro** sin Flexbox hay 2 técnicas, cada una con su truco.

1. display: inline-block

Convierte cada bloque en una caja que se coloca como un elemento en línea, pero que acepta ancho y alto. Es la más sencilla, pero tiene una trampa famosa: **el espacio en blanco del HTML entre las dos etiquetas** (el de la sección anterior) cuenta como un espacio de texto, y ocupa unos píxeles. Si los dos anchos suman el 100%, esos píxeles de más hacen que el segundo bloque no quepa y salte a la línea siguiente. Fíjate también en vertical-align: top: sin él, los dos bloques se alinean por su línea base y quedan desalineados si tienen alturas distintas.

```
<div class="pagina">
  <div class="lateral">Lateral (25%)</div>
  <div class="principal">Principal (75%)</div>
</div>
```

```
.pagina { border: 1px dashed #888; }
.lateral, .principal {
  display: inline-block;
  vertical-align: top;
  padding: 10px;
```

```
}
.lateral { width: 25%; background: #EFE7F2; }
.principal { width: 75%; background: #FDF3D0; }
```

Hay varias formas de arreglarlo. La más extendida es quitar el espacio en el HTML, pegando las etiquetas, o rellenando el hueco con un comentario HTML, que el navegador no convierte en espacio y permite mantener el código bien sangrado:

```
<div class="pagina">
  <div class="lateral">Lateral (25%)</div><!--
  --><div class="principal">Principal (75%)</div>
</div>
```

```
.pagina { border: 1px dashed #888; }
.lateral, .principal {
  display: inline-block;
  vertical-align: top;
  padding: 10px;
}
.lateral { width: 25%; background: #EFE7F2; }
.principal { width: 75%; background: #FDF3D0; }
```

3.14 Posicionamiento

Valor	Comportamiento
static	El valor por defecto: el elemento sigue el flujo normal del documento. Las propiedades top/left/etc. no tienen efecto.
relative	Sigue ocupando su espacio original en el flujo, pero puede desplazarse visualmente con top/left/etc. respecto a esa posición.
absolute	Sale del flujo normal (deja de ocupar espacio) y se posiciona respecto al ancestro **posicionado** más cercano (cualquier valor distinto de static), o respecto al documento si no hay ninguno.
fixed	Se posiciona respecto a la ventana del navegador y no se mueve al hacer scroll.
sticky	Híbrido: se comporta como relative hasta que, al hacer scroll, alcanza el umbral definido (por ejemplo, top: 0), momento en el que se vuelve fixed dentro de su contenedor.

Posicionamiento relativo
El posicionamiento relativo desplaza una caja respecto de su posición original establecida mediante el posicionamiento normal. El desplazamiento de la caja se controla con las propiedades top, right, bottom y left.

Por tanto, la propiedad top se emplea para mover las cajas de forma descendente, la propiedad bottom mueve las cajas de forma ascendente, la propiedad left se utiliza para desplazar las cajas hacia la derecha y la propiedad right mueve las cajas hacia la izquierda.

Si se utilizan valores negativos en las propiedades top, right, bottom y left, su efecto es justamente el inverso.



Como es lógico, si se indica la propiedad ‘top’, no se indicará ‘bottom’, o al revés. En el caso de que se indiquen las dos, una de las dos se ignorará.
Lo mismo pasa con left y right.

Ejemplo:

```
<div class="fila">
  <div class="caja">1</div>
  <div class="caja desplazada">2</div>
  <div class="caja">3</div>
</div>
```

```
.fila { display: flex; }
.caja {
  width: 50px;
  height: 50px;
  margin-right: 14px;
  background: #4B3563;
  color: #fff;
}
.desplazada {
  position: relative;
  top: 16px;
  left: 16px;
}
```

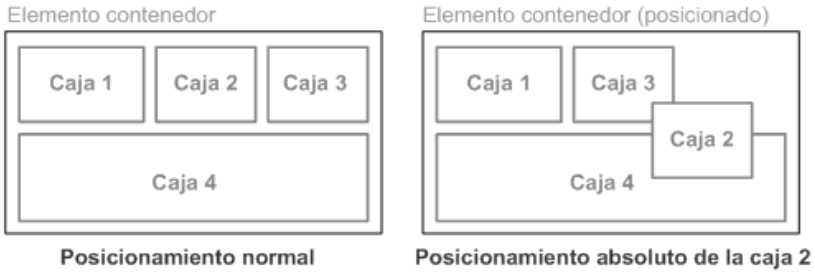


La caja **2** tiene `position: relative; top: 16px; left: 16px;`. Se desplaza visualmente respecto a su posición original (el hueco discontinuo), pero **sigue reservando ese espacio** en el flujo: por eso la caja 3 no se mueve para ocupar el hueco, a diferencia de lo que pasaría con `absolute`.

Importante: El error más común con position: absolute es olvidar poner position: relative en el contenedor. Si ningún ancestro está posicionado, el elemento se ancla al <body> completo, con resultados normalmente muy distintos de lo esperado.

Posicionamiento absoluto
El posicionamiento absoluto se emplea para establecer de forma exacta la posición en la que se muestra la caja de un elemento. La nueva posición de la caja se indica mediante las propiedades top, right, bottom y left.
La interpretación de los valores de estas propiedades es mucho más compleja que en el posicionamiento relativo, ya que en este caso dependen del posicionamiento del elemento contenedor.

Las cajas posicionadas de forma absoluta "salen del flujo normal de la página", lo que provoca que el resto de elementos de la página se muevan y en ocasiones, ocupen la posición original en la que se encontraba la caja.



Por ejemplo, en el ejemplo anterior, la poner la caja2 con una posición absoluta, la caja3 ha modificado su posición. Esto no pasaba en el posicionamiento relativo.

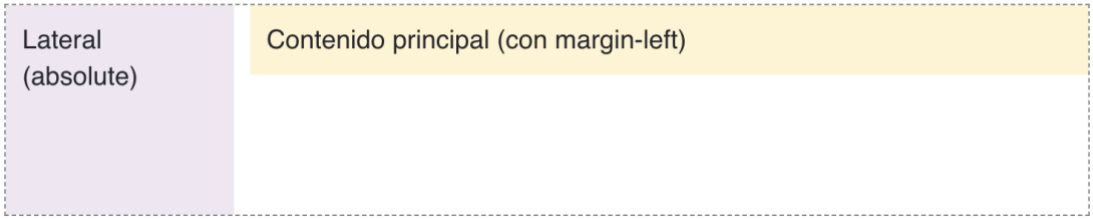
Para determinar cuál es el elemento contenedor:
- Se recorren todos los elementos contenedores empezando por el más cercano a la caja y llegando hasta el <body>
- El primer elemento contenedor que esté posicionado de cualquier forma **diferente a static**, se convierte en la referencia que determina la posición de la caja posicionada de forma absoluta.
- Si ningún elemento contenedor está posicionado, la referencia es la ventana del navegador, que no debe confundirse con el elemento <body> de la página.

Una vez visto el posicionamiento, seguimos el posicionamiento de elementos.
3. position: absolute
Un elemento con position: absolute sale del flujo: los demás se comportan como si no existiera, y se coloca con top, right, bottom y left respecto a su ancestro posicionado más cercano. Para hacer dos columnas, se saca una de ellas del flujo y se deja un hueco a la otra con un margen. La contrapartida, importante: como el elemento absoluto no

cuenta para la altura de su contenedor, **el contenedor solo mide lo que midan sus hijos en flujo normal**, y conviene darle una altura propia:

```
<div class="pagina">
  <div class="lateral">Lateral (absolute)</div>
  <div class="contenido">Contenido principal (con margin-left)</div>
</div>
```

```
.pagina { position: relative; height: 130px; border: 1px dashed #888; }
.lateral {
  position: absolute;
  top: 0;
  bottom: 0;
  left: 0;
  width: 140px;
  padding: 10px;
  background: #EFE7F2;
}
.contenido { margin-left: 150px; padding: 10px; background: #FDF3D0; }
```



El lateral se estira de arriba a abajo del contenedor con `top: 0` y `bottom: 0`, y el contenido principal se aparta 150px con `margin-left`. Si quitas el `height` del contenedor, este pasa a medir solo lo que mide el contenido principal (el lateral, al estar fuera del flujo, no le da altura) y el lateral se acorta con él. Y si el lateral no estuviera anclado también por abajo (`bottom: 0`) y tuviera más contenido que el principal, sobresaldría por debajo del contenedor.

Técnica	Ventaja	Cuidado con...
inline-block	La más simple; admite alturas distintas con vertical-align: top.	El espacio en blanco del HTML; las alturas no se igualan.
absolute	Control total de la posición.	No da altura al contenedor: conviene fijársela; no se adapta al contenido.

Colocar con position

Anteriormente se explicó qué hace cada valor de position. Aquí, los patrones prácticos.
Un elemento con position: absolute se coloca respecto al ancestro posicionado más cercano (normalmente un contenedor al que se da position: relative a propósito).

Además, el elemento absoluto pasa a comportarse como un bloque, aunque fuera :

```
<div class="referencia">
  <span class="esquina arr-izq">arriba izquierda</span>
  <span class="esquina arr-der">arriba derecha</span>
  <span class="esquina aba-izq">abajo izquierda</span>
  <span class="esquina aba-der">abajo derecha</span>
</div>
```

```
.referencia { position: relative; height: 130px; border: 1px dashed #888; }
.esquina {
  position: absolute;
  padding: 3px 10px;
  background: #4B3563;
  color: #fff;
}
.arr-izq { top: 0; left: 0; }
.arr-der { top: 0; right: 0; }
.aba-izq { bottom: 0; left: 0; }
.aba-der { bottom: 0; right: 0; }
```



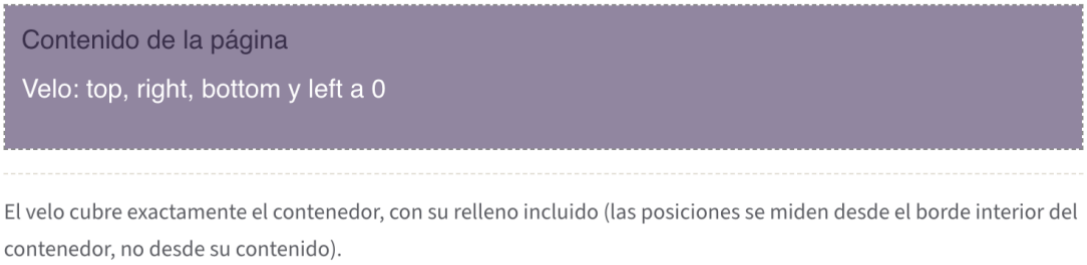
Cuatro `<span>` anclados a cada esquina del contenedor. Fíjate en que, al ser absolutos, ya aceptan `padding` vertical y se han vuelto elementos de bloque, sin que escribas `display`.

Si das valor a los cuatro lados a la vez, el elemento se estira hasta cubrir por completo a su contenedor. Es la base de los velos y superposiciones (inset: 0 hace lo mismo en una sola propiedad):

```
<div class="referencia">
  Contenido de la página
  <div class="velo">Velo: top, right, bottom y left a 0</div>
</div>
```

```
.referencia {
  position: relative;
  height: 90px;
  padding: 10px;
```

```
  border: 1px dashed #888;
}
.velo {
  position: absolute;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  padding: 40px 10px 10px;
  background: rgba(75, 53, 99, .6);
  color: #fff;
}
```



El velo cubre exactamente el contenedor, con su relleno incluido (las posiciones se miden desde el borde interior del contenedor, no desde su contenido).

Los otros dos valores no sacan del flujo de la misma manera. position: sticky se comporta como un elemento normal hasta que, al desplazar, llega a la distancia que indicas; entonces se queda "pegado" mientras su contenedor siga a la vista. Desplaza esta lista:

```
<div class="lista">
  <div class="cabecera">Cabecera pegajosa (position: sticky)</div>
  <div class="fila">Fila 1</div>
  <div class="fila">Fila 2</div>
  <div class="fila">Fila 3</div>
  <div class="fila">Fila 4</div>
  <div class="fila">Fila 5</div>
  <div class="fila">Fila 6</div>
  <div class="fila">Fila 7</div>
  <div class="fila">Fila 8</div>
</div>
```

```
.lista { height: 130px; overflow: auto; border: 1px solid #888; }
.cabecera {
  position: sticky;
  top: 0;
  padding: 6px 10px;
```

```
  background: #4B3563;
  color: #fff;
}
.fila { padding: 8px 10px; border-bottom: 1px solid #eee; }
```

position: fixed coloca el elemento respecto a la **ventana** del navegador y lo mantiene ahí aunque se desplace la página (barras de cookies, botones de "volver arriba"). Para poder enseñarlo sin que se quede pegado a tu pantalla, este ejemplo se muestra dentro de un marco con su propia ventana y su propio desplazamiento:

```
<h3>Página larga</h3>
<p>Párrafo 1 de relleno para poder desplazar la página.</p>
<p>Párrafo 2 de relleno para poder desplazar la página.</p>
<p>Párrafo 3 de relleno para poder desplazar la página.</p>
<p>Párrafo 4 de relleno para poder desplazar la página.</p>
<p>Párrafo 5 de relleno para poder desplazar la página.</p>
<p>Párrafo 6 de relleno para poder desplazar la página.</p>
<p>Párrafo 7 de relleno para poder desplazar la página.</p>
<p>Párrafo 8 de relleno para poder desplazar la página.</p>
<div class="barra">Barra fija abajo (position: fixed)</div>
```

```
.barra {
  position: fixed;
  left: 0;
  right: 0;
  bottom: 0;
  padding: 8px 12px;
  background: #4B3563;
  color: #fff;
}
```

Centrar en los dos ejes

Centrar en horizontal es sencillo (márgenes en auto o text-align). Centrar en **vertical** era, antes de Flexbox, el problema más difícil de CSS, porque los bloques no tienen una forma directa de repartir su espacio sobrante en vertical. Estas son las cuatro soluciones clásicas, todas con el mismo objetivo: un contenedor de 200 × 110px con un elemento en el centro exacto.

```
<div class="envoltorio">
  <div class="c1"><div class="objeto">transform</div></div>
</div>
<div class="envoltorio">
  <div class="c2"><div class="objeto">margin: auto</div></div>
</div>
```

```
<div class="envoltorio">
  <div class="c3">line-height</div>
</div>
```

```
.envoltorio {
  display: inline-block;
  vertical-align: top;
  margin: 0 10px 10px 0;
}
.c1, .c2, .c3 { width: 200px; height: 110px; border: 1px dashed #888; }
.objeto {
  width: 100px;
  padding: 8px 0;
  text-align: center;
  background: #4B3563;
  color: #fff;
}

/* 1. position + transform */
.c1 { position: relative; }
.c1 .objeto {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}

/* 2. position + margin: auto */
.c2 { position: relative; }
.c2 .objeto {
  position: absolute;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  height: 36px;
  margin: auto;
}
```

```
/* 3. line-height (solo una línea de texto) */
.c3 { line-height: 108px; text-align: center; }
```



Los tres métodos dan el mismo resultado visual. **1.** top: 50% y left: 50% colocan la esquina del elemento en el centro, y translate(-50%, -50%) lo retrocede la mitad de su propio tamaño: funciona sin conocer el tamaño del elemento. **2.** Con los cuatro lados a 0 y un tamaño fijo, margin: auto reparte el espacio sobrante a los cuatro lados: necesita conocer ancho y alto. **3.** Una línea de texto con un line-height igual a la altura del contenedor queda centrada; solo sirve para una única línea.

Ejercicios

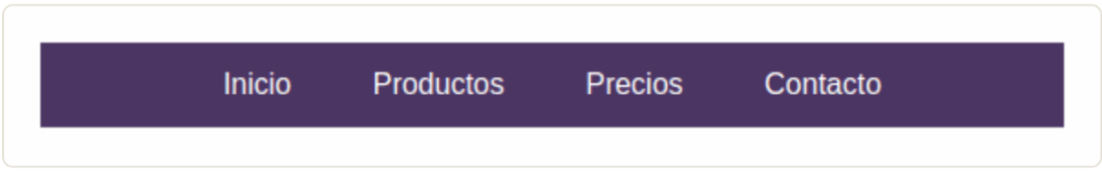
Ejercicios · Posicionamiento

Ejercicio 1: Crea una tarjeta de 320px de ancho, centrada en horizontal en la página. El título va centrado dentro de la tarjeta y el enlace "Leer más" alineado a la derecha. Sin text-align en el body y sin Flexbox.

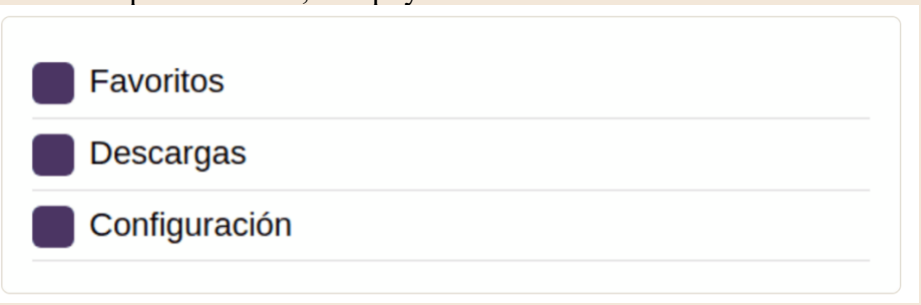


Ejercicio 2: Convierte esta lista en una barra de navegación horizontal, con fondo morado y los cuatro enlaces centrados. Cada enlace debe ser un área clicable amplia (con relleno arriba, abajo y a los lados). Sin Flexbox. Color lila: #4B3563

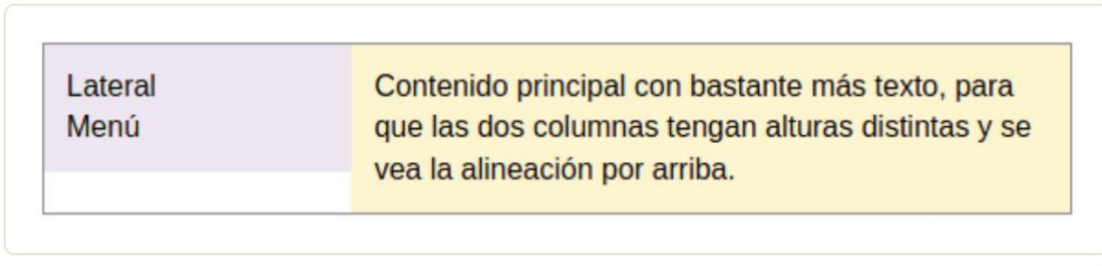
```
<ul class="menu">
  <li><a href="#">Inicio</a></li>
  <li><a href="#">Productos</a></li>
  <li><a href="#">Precios</a></li>
  <li><a href="#">Contacto</a></li>
</ul>
```



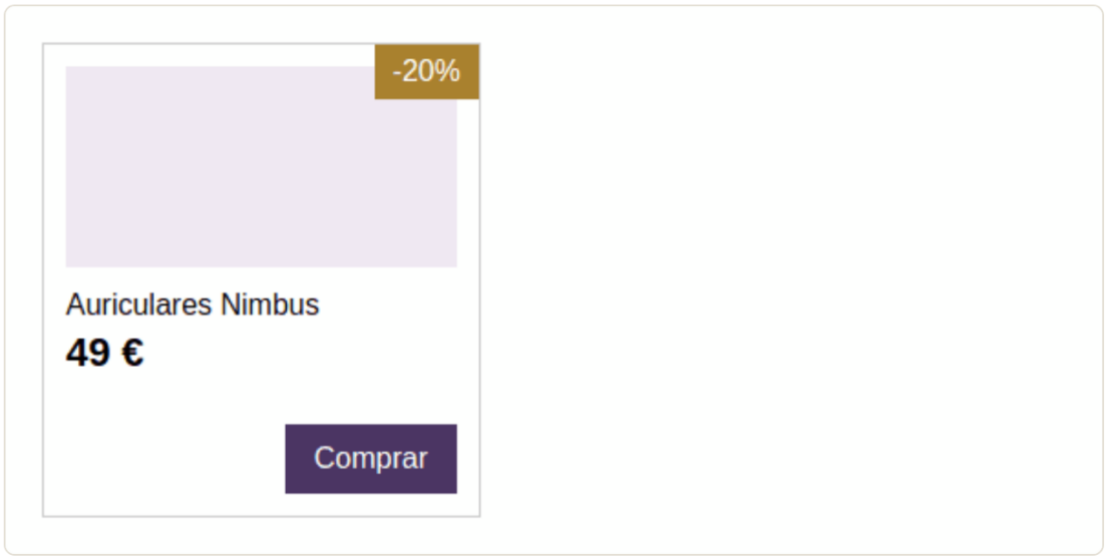
Ejercicio 3: Cada fila lleva un icono cuadrado de 28px (dibújalo con CSS, con un vacío) seguido de un texto grande. El icono tiene que quedar centrado en vertical respecto al texto, no apoyado en su línea base.



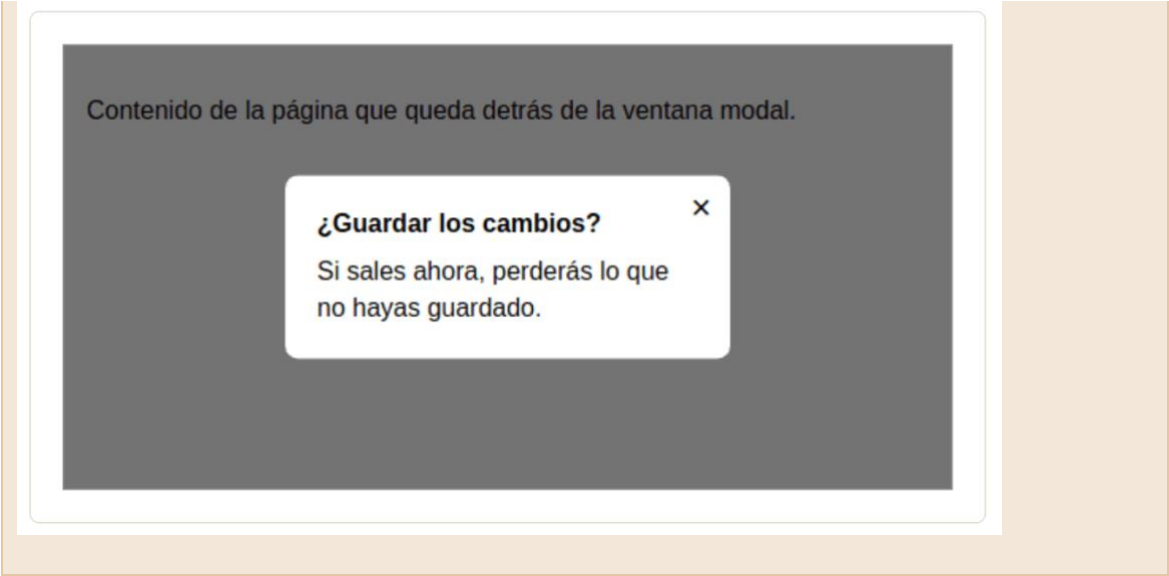
Ejercicio 4: Maqueta una página con una columna lateral (30% del ancho) y una columna principal (70%), una junto a otra y alineadas por arriba, aunque tengan alturas distintas. Sin Flexbox, sin Grid y sin que la segunda columna salte a la línea siguiente.



Ejercicio 5: Una tarjeta de 240 × 260px con una insignia "-20%" pegada a su esquina superior derecha y un botón "Comprar" anclado a la esquina inferior derecha, a 12px de los bordes. El precio y el nombre siguen el flujo normal.



Ejercicio 6: Dentro de una "página" de 280px de alto, muestra una ventana modal de 280px de ancho centrada en los dos ejes, sobre un velo oscuro que cubra toda la página, y con un botón "×" en la esquina superior derecha de la modal.



3.15 Márgenes, relleno, bordes y sombras

Las tres capas que rodean al contenido (relleno, borde y margen) se controlan con propiedades que comparten la misma lógica: una propiedad para cada lado (margin-top, margin-right, margin-bottom, margin-left, y lo mismo con padding) y un atajo (shorthand) que acepta de uno a cuatro valores.

El atajo de 1 a 4 valores

Valores	Ejemplo	Significado
1	margin: 10px;	Los cuatro lados valen 10px.
2	margin: 10px 5px;	Arriba y abajo 10px; izquierda y derecha 5px.
3	margin: 5px 2px 1px;	Arriba 5px; izquierda y derecha 2px; abajo 1px.
4	margin: 4px 3px 2px 1px;	Arriba 4px, derecha 3px, abajo 2px, izquierda 1px.

Con cuatro valores, el orden sigue el sentido de las agujas del reloj, empezando por arriba: **arriba, derecha, abajo, izquierda**. Con menos valores, los que faltan se copian del lado opuesto. Funciona igual en margin, padding, border-width, border-style y border-color.

Margen (margin)
Es el espacio **exterior** de la caja, transparente, que la separa de sus vecinas. Su valor inicial es 0 (si ves separación en un párrafo o un título sin haber escrito nada, es el margen por defecto que la hoja de estilos del navegador da a esa etiqueta). Admite unidades, porcentajes, auto y, a diferencia de width o padding, también **valores negativos**. Los márgenes verticales (margin-top y margin-bottom) no se aplican a los elementos en línea.

```
<div class="marco">
  <div class="interior">margin: 10px 40px 20px 70px</div>
</div>
```

```css
.marco { border: 1px solid #888; }
.interior {
  margin: 10px 40px 20px 70px;
  background: #4B3563;
  color: #fff;
  padding: 8px;
}
```



La caja morada no tiene ancho declarado, así que ocupa lo que queda libre dentro del marco una vez descontados sus márgenes: 70px a la izquierda y 40px a la derecha. Arriba deja 10px y abajo 20px, que se ven como huecos entre la caja y el borde del marco.

El valor auto en los márgenes laterales de un bloque con ancho fijo reparte el espacio sobrante a ambos lados, y por eso es la forma clásica de **centrar** un bloque horizontalmente:

```html
<div class="marco">
  <div class="centrada">margin: 0 auto</div>
</div>
```

```css
.centrada {
  width: 200px;
  margin: 0 auto;
}
```



Fusión de márgenes verticales (*margin collapse*)
Cuando se tocan dos márgenes verticales, no se suman: **se fusionan** en uno solo, que mide lo que mida el mayor de los dos. Pasa en dos situaciones.
1. Entre hermanos. Si una caja tiene margin-bottom: 40px y la siguiente margin-top: 20px, la separación entre ambas no es de 60px, sino de 40px:

```html
<div class="franja-a">margin-bottom: 40px</div>
<div class="franja-b">margin-top: 20px</div>
```

```css
.franja-a { margin-bottom: 40px; }
.franja-b { margin-top: 20px; }
```



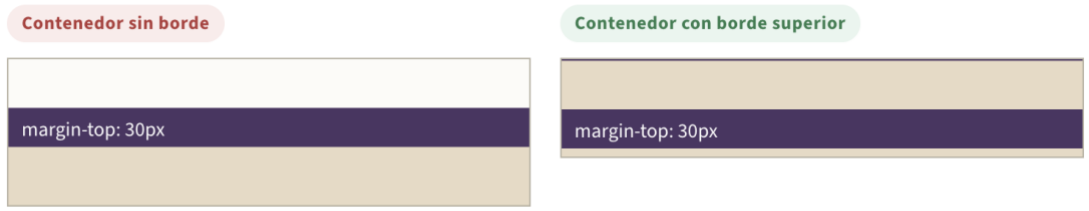
El hueco entre las dos franjas mide 40px, no 60px.

2. Entre un contenedor y su primer (o último) hijo. Si el contenedor no tiene borde, relleno ni nada que lo separe de su hijo, el margen superior del hijo "se escapa" y acaba empujando al contenedor entero, en lugar de quedarse dentro:

```
<div class="contenedor-a">
  <div class="hijo-mg">margin-top: 30px</div>
</div>

<div class="contenedor-b">
  <div class="hijo-mg">margin-top: 30px</div>
</div>
```

```
.contenedor-a,
.contenedor-b { background: #E7D9C3; height: 60px; }
.contenedor-b { border-top: 1px solid #4B3563; }
.hijo-mg {
  margin-top: 30px;
  height: 24px;
  background: #4B3563;
  color: #fff;
}
```



A la izquierda, el hueco de 30px queda *fuera* del contenedor beige: es el contenedor el que baja. A la derecha, un simple borde de 1px impide la fusión y el hueco queda *dentro*, entre el borde del contenedor y su hijo.

Para evitar la fusión no hace falta el borde: basta con cualquier cosa que separe ambos márgenes, como un padding en el contenedor o display: flow-root en él. Solo afecta a márgenes **verticales** de elementos en flujo normal; los horizontales nunca se fusionan, y los de elementos Flexbox o Grid tampoco.

Relleno (padding)
Es el espacio **interior**, entre el contenido y el borde. Tiene las mismas cuatro propiedades por lado y el mismo atajo de 1 a 4 valores que el margen, con dos

diferencias: **no admite valores negativos ni auto**. Y, a diferencia del margen, el fondo de la caja sí lo pinta (apartado 8.9). Es lo que convierte un enlace o un texto en un botón con aire alrededor:

```
<span class="boton">Sin relleno</span>
<span class="boton con-relleno">Con relleno</span>
```

```
.boton {
  display: inline-block;
  background: #4B3563;
  color: #fff;
  border-radius: 6px;
}
.con-relleno { padding: 8px 18px; }
```



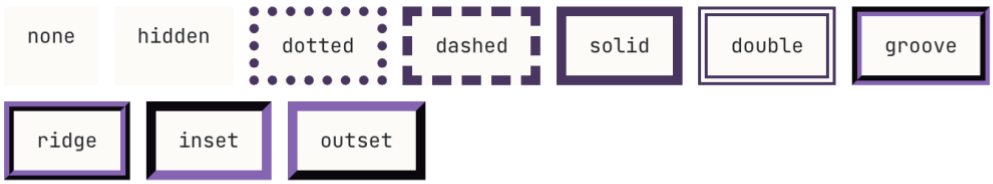
Borde (border)
Un borde tiene tres aspectos, cada uno con su propiedad (y con su versión por lado: border-top-width, border-left-color...):

Aspecto	Propiedad	Valores
Grosor	border-width	Una unidad de medida o thin, medium (por defecto), thick. Admite 1 a 4 valores.
Estilo	border-style	none (por defecto), hidden, dotted, dashed, solid, double, groove, ridge, inset, outset. Admite 1 a 4 valores.
Color	border-color	Cualquier color o transparent. Admite 1 a 4 valores.

Estos son los diez estilos, con un borde de 6px del mismo color (los efectos 3D de groove, ridge, inset y outset, y las dos líneas de double, solo se aprecian con un grosor suficiente):

```
<div class="borde b-none">none</div>
<div class="borde b-hidden">hidden</div>
<div class="borde b-dotted">dotted</div>
<div class="borde b-dashed">dashed</div>
<div class="borde b-solid">solid</div>
<div class="borde b-double">double</div>
<div class="borde b-groove">groove</div>
<div class="borde b-ridge">ridge</div>
<div class="borde b-inset">inset</div>
<div class="borde b-outset">outset</div>
```

```css
.borde {
  border-width: 6px;
  border-color: #4B3563;
  padding: 8px 14px;
}
.b-none { border-style: none; }
.b-hidden { border-style: hidden; }
.b-dotted { border-style: dotted; }
.b-dashed { border-style: dashed; }
.b-solid { border-style: solid; }
.b-double { border-style: double; }
.b-groove { border-style: groove; }
.b-ridge { border-style: ridge; }
.b-inset { border-style: inset; }
.b-outset { border-style: outset; }
```



`none` y `hidden` no dibujan nada (la diferencia solo importa en las tablas). Los demás se distinguen mejor cuanto más grueso sea el borde.

Importante: El estilo de borde por defecto es none, y con él el grosor se calcula como 0 (lo comprobamos en Chrome). Por eso border: 5px red; no dibuja nada: falta el estilo. Siempre hace falta, como mínimo, un grosor y un estilo, por ejemplo border: 5px solid red;

Lo habitual es usar el atajo border, que acepta las tres cosas en cualquier orden, y su versión por lado cuando solo quieres un borde:

```css
/* Los cuatro lados */
div { border: 1px solid red; }

/* Equivale a escribir los cuatro por separado */
div {
  border-top: 1px solid red;
  border-right: 1px solid red;
  border-bottom: 1px solid red;
  border-left: 1px solid red;
}
```

```
/* Solo uno, como separador */
.menu { border-bottom: 3px solid #004C99; }
```

Esquinas redondeadas y sombras

Propiedad	Ejemplo	Notas
border-radius	border-radius: 10px;	Redondea las esquinas; admite un valor por esquina.
box-shadow	box-shadow: 0 4px 12px rgba(0,0,0,.15);	Desplazamiento X, desplazamiento Y, difuminado, (opcional expansión), color.

La tarjeta de artículo con esas dos propiedades aplicadas:

```
.card {
  border-radius: 10px;
  box-shadow: 0 8px 20px rgba(75, 53, 99, .18);
}
```



La tarjeta del ejemplo, ya con `border-radius: 10px` y un `box-shadow` suave aplicados.

El mismo box-shadow, con distintos valores, sirve para sugerir distintos niveles de "elevación" sobre la página:

```
<div class="elev elev-1">Sutil</div>
<div class="elev elev-2">Media</div>
<div class="elev elev-3">Alta</div>
```

```
.elev { width: 90px; height: 70px; background: #fff; border-radius: 8px; }
```

```
.elev-1 { box-shadow: 0 1px 3px rgba(0,0,0,.12); }
.elev-2 { box-shadow: 0 6px 14px rgba(0,0,0,.16); }
.elev-3 { box-shadow: 0 16px 32px rgba(0,0,0,.22); }
```



Cuanto mayor el desplazamiento Y y el difuminado, más "flota" el elemento sobre la página. Usa siempre un color con transparencia (`rgba()`), nunca negro sólido: una sombra sólida se ve como un bloque pegado, no como una sombra.

border-radius en detalle: esquinas sueltas y esquinas elípticas

Redondear una esquina significa recortarla con un cuarto de círculo. El **radio** es el tamaño de ese círculo: cuanto mayor, más redondeada la esquina. Hasta ahora lo escribimos como border-radius: 10px, que redondea las cuatro esquinas por igual, pero la propiedad es mucho más flexible. Es una abreviatura de **cuatro propiedades largas**, una por esquina:

Propiedad larga	Esquina
border-top-left-radius	Superior izquierda
border-top-right-radius	Superior derecha
border-bottom-right-radius	Inferior derecha
border-bottom-left-radius	Inferior izquierda

Este es el mismo caso escrito en versión larga y en versión corta, el resultado es idéntico. En la corta, los valores siguen el orden de las agujas del reloj empezando por arriba a la izquierda:

```
<div class="larga">Versión larga</div>
```

```
.larga {
  border-top-left-radius: 40px;
  border-top-right-radius: 0;
  border-bottom-right-radius: 40px;
  border-bottom-left-radius: 10px;
}

/* Es exactamente lo mismo que la versión corta, en el orden de las agujas del reloj */
.corta { border-radius: 40px 0 40px 10px; }
```

La abreviatura con uno, dos, tres y cuatro valores

Igual que margin y padding (apartado 8.10), la abreviatura admite de uno a cuatro valores, pero aquí se reparten por *esquinas* y no por lados. Con dos valores, el primero afecta a la diagonal que une la esquina superior izquierda con la inferior derecha y el segundo a la otra diagonal:

Valores	Reparto
1 valor: A	Las cuatro esquinas.
2 valores: A B	A: superior izquierda e inferior derecha. B: superior derecha e inferior izquierda.
3 valores: A B C	A: superior izquierda. B: superior derecha e inferior izquierda. C: inferior derecha.
4 valores: A B C D	Superior izquierda, superior derecha, inferior derecha, inferior izquierda (sentido de las agujas del reloj).

Redondear una caja que contiene una imagen
El borde se redondea, pero el **contenido** no se recorta por sí solo: una imagen colocada dentro seguirá teniendo las esquinas cuadradas y sobresaldrá por las esquinas redondeadas de su contenedor. La solución es overflow: hidden en el contenedor:

```
<div class="tarjeta">
  <img src="paisaje.svg" alt="Paisaje">
  <p>Imagen dentro de una tarjeta con esquinas redondeadas.</p>
</div>
<div class="tarjeta recortada">
  <img src="paisaje.svg" alt="Paisaje">
  <p>La misma tarjeta con overflow: hidden.</p>
</div>
```

```
.tarjeta { border: 2px solid #4B3563; border-radius: 20px; background: #fff; }
.tarjeta img { display: block; width: 100%; }
.tarjeta p { margin: 0; padding: 10px 14px; }

.recortada { overflow: hidden; }    /* recorta lo que sobresale de las esquinas redondeadas */
```



A la izquierda, las esquinas superiores de la imagen sobresalen del borde redondeado. A la derecha, `overflow: hidden` recorta la imagen siguiendo la curva.

3.16 Fondos

Propiedad	Qué controla
background-color	Color de fondo sólido.
background-image	Una imagen (o un degradado, con linear-gradient()/radial-gradient()) como fondo.
background-size	cover (rellena todo el espacio, recortando si hace falta) o contain (se ve entera, puede dejar huecos).
background-position	Dónde se ancla la imagen (center, top left...).
background-repeat	no-repeat, repeat, repeat-x/repeat-y.

```
.card img {
  background-image: linear-gradient(135deg, #4B3563, #8B5FA6);
  background-size: cover;
<!—Para pode poner una imagen de fondo-->
  background-image:url(imagenes/fondo.gif);
}
```



background-size: cover background-size: contain

Es la misma imagen (400×300, proporción 4:3) dentro de una caja mucho más ancha y baja. Con `cover` (izquierda) se recorta el sol y parte de las montañas para rellenar todo el hueco sin dejar huecos vacíos. Con `contain` (derecha) se ve la imagen completa, a costa de dejar franjas del color de fondo a los lados.

background-attachment: el fondo y el scroll

Las propiedades anteriores deciden qué imagen se ve (background-image), cuánto mide (background-size), dónde se coloca (background-position) y si se repite (background-repeat). Pero todas suponen un fondo quieto. Cuando alguien hace scroll, ¿qué debe hacer el fondo? ¿Moverse con el texto, quedarse pegado a su caja o quedarse inmóvil mientras todo pasa por delante? Esa decisión es de background-attachment, que significa literalmente "a qué se sujeta el fondo".

Valor	El fondo está sujeto a...	Qué se nota al hacer scroll

scroll (por defecto)	El propio elemento (su caja).	Se mueve con el elemento cuando la página se desplaza, pero **no** se mueve con el contenido interior del elemento si este tiene su propio scroll.
local	El contenido del elemento.	Si el elemento tiene su propio scroll, el fondo se desplaza junto con el texto, como si estuviera impreso en el mismo papel.
fixed	La ventana del navegador (el *viewport*).	No se mueve nunca: el elemento es solo un hueco por el que se ve una parte del fondo.

scroll frente a local: el scroll interior de una caja

Estos dos valores se comportan igual en la mayoría de los casos, y por eso scroll es el valor por defecto y casi nadie lo escribe. Solo se distinguen cuando **el propio elemento tiene scroll**, es decir, cuando su contenido es más alto que él y tiene overflow: auto (o scroll), como la lista de un chat o un bloque de código con altura limitada. Para verlo hace falta un fondo en el que se note el movimiento: unas franjas horizontales de 30 px que se repiten.

Este es el HTML de las dos cajas. Son idénticas salvo la clase que lleva cada una:

```
<div class="caja caja-scroll">
  <p>Línea 1 del contenido</p>
  <p>Línea 2 del contenido</p>
  <p>Línea 3 del contenido</p>
  <p>Línea 4 del contenido</p>
  <p>Línea 5 del contenido</p>
  <p>Línea 6 del contenido</p>
  <p>Línea 7 del contenido</p>
  <p>Línea 8 del contenido</p>
  <p>Línea 9 del contenido</p>
  <p>Línea 10 del contenido</p>
  <p>Línea 11 del contenido</p>
  <p>Línea 12 del contenido</p>
</div>

<div class="caja caja-local">
  <p>Línea 1 del contenido</p>
  <p>Línea 2 del contenido</p>
  <p>Línea 3 del contenido</p>
  <p>Línea 4 del contenido</p>
  <p>Línea 5 del contenido</p>
  <p>Línea 6 del contenido</p>
  <p>Línea 7 del contenido</p>
  <p>Línea 8 del contenido</p>
```

```
  <p>Línea 9 del contenido</p>
  <p>Línea 10 del contenido</p>
  <p>Línea 11 del contenido</p>
  <p>Línea 12 del contenido</p>
</div>
```

Y este es el CSS. Fíjate en que la única diferencia entre las dos cajas está en las dos últimas reglas:

```
.caja {
  height: 180px;
  overflow: auto;           /* la caja tiene su propio scroll */
  padding: 12px;
  background-image: linear-gradient(#EFE7F2 50%, #D9C8E3 50%);
  background-size: 100% 60px; /* franjas de 30px que se repiten */
}
.caja p { margin: 0 0 8px; }

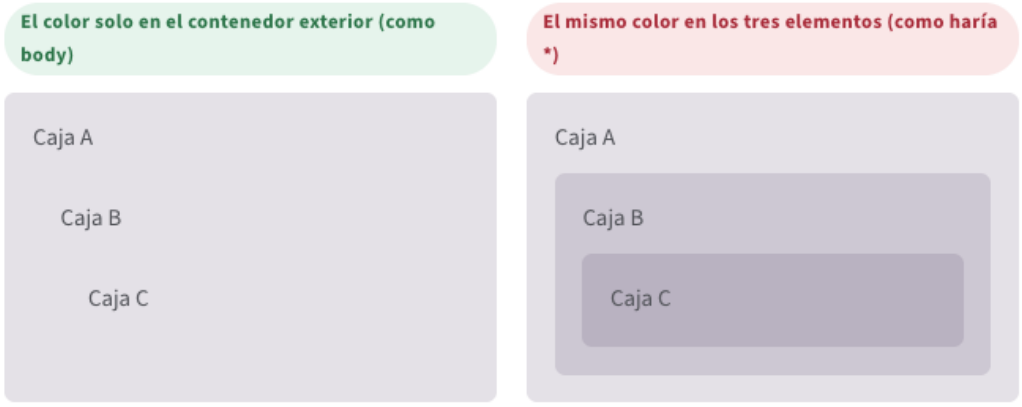
.caja-scroll { background-attachment: scroll; } /* valor por defecto */
.caja-local  { background-attachment: local; }
```

fixed: el elemento es una ventana
Con fixed el fondo deja de pertenecer al elemento y pasa a pertenecer a la **ventana del navegador**. Piensa en esa imagen como en un cuadro colgado en la pared, y en cada elemento con fixed como en un agujero recortado en un papel que deslizas por delante: según dónde esté el agujero en ese momento, ves una parte u otra del cuadro, pero el cuadro nunca se mueve. Es el efecto típico de las páginas de presentación, en las que el texto "pasa por encima" de una imagen de fondo que parece permanecer quieta. También puedes usarlo en el body para que el fondo de toda la página no se desplace. Vamos a ver una página completa con dos bandas que comparten la misma imagen de fondo, descarga el ejemplo AQUÍ.

Fondo de página: body, nunca *
Para poner un color de fondo a toda la página, la tentación es escribir * { background-color: ...; } para "asegurarse" de que llega a todas partes. Es un error: * selecciona todos los elementos del documento, así que cada uno (cada div, cada párrafo, cada botón) recibe su propia capa de fondo, no una sola compartida. body selecciona un único elemento, y es la forma correcta de hacerlo.

Con un color sólido y opaco, la diferencia apenas se nota a simple vista, porque todas esas capas son del mismo color exacto y se tapan limpiamente unas a otras. Pero en cuanto el color tiene algo de transparencia (muy habitual: superposiciones suaves, overlays), cada elemento anidado suma su propia capa sobre la de su padre:

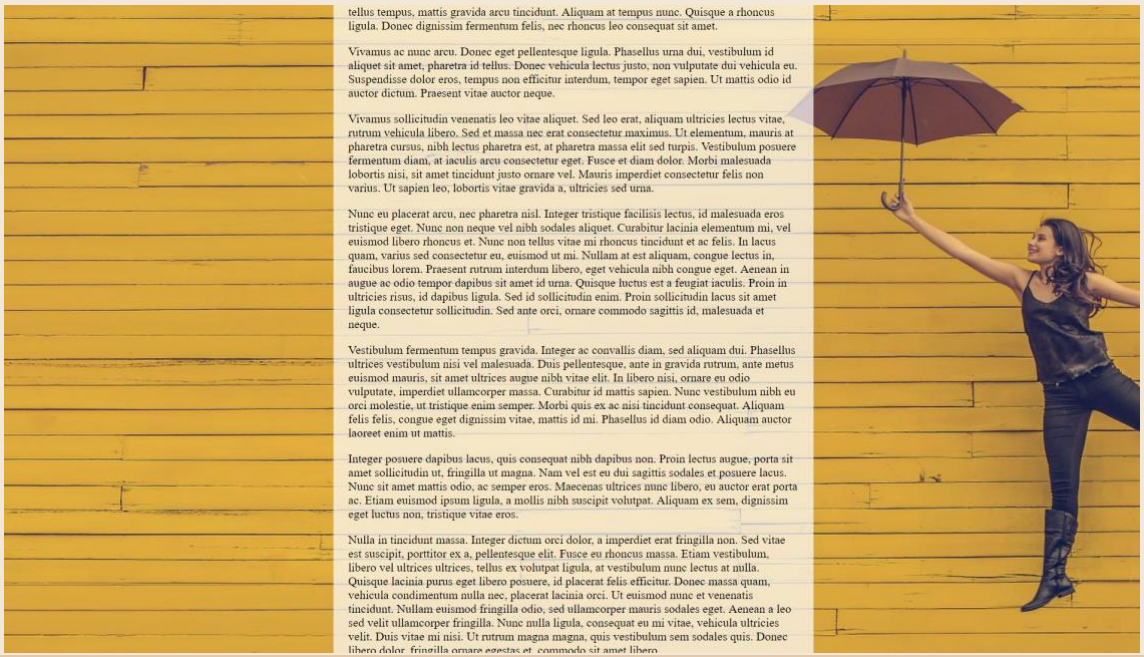
```
.a { background-color: rgba(75, 53, 99, .14); }
/* .b y .c están dentro de .a, en el HTML */
```



Mismas tres cajas anidadas, mismo color, misma opacidad. A la izquierda, solo la caja exterior tiene fondo: un tono uniforme, porque B y C son transparentes y dejan verlo a través. A la derecha, las tres tienen su propio fondo (lo que ocurriría de verdad con `*`): cada capa se suma a la anterior, y el resultado es una escalera cada vez más oscura que casi nunca es el efecto que buscabas.

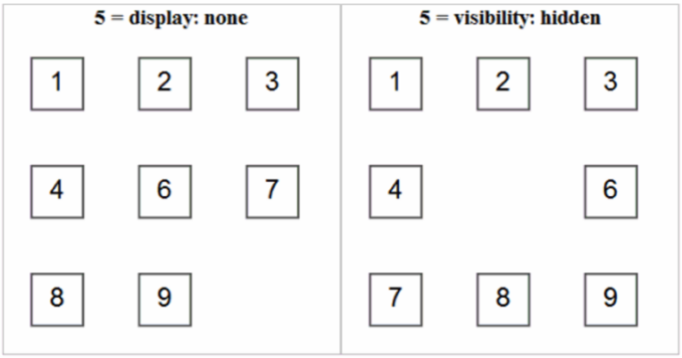
Actividad · Posicionamiento fondos

Esta página ya tiene todo el contenido del HTML con la estructura.
Añadir al contenedor los estilos necesarios para que tenga un ancho de 600px, fondo blanco (con opacidad al 70%) y padding de 20px;
Además, hay que poner los estilos necesarios para que aparezca el fondo tal y como aparece en la siguiente imagen y, además, hay que configurarlo para que al hacer scroll, el fondo no se mueva.



3.17 Visibility, overflow, z-index y cursor

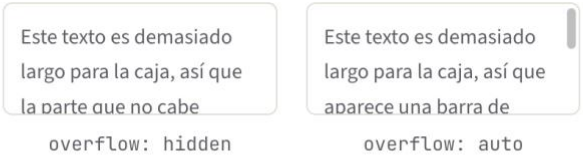
visibility: hidden se diferencia de display: none en algo importante: oculta el elemento visualmente, pero sigue ocupando su espacio en la maquetación (útil, por ejemplo, para reservar el hueco de un tooltip que aún no debe verse).



overflow decide qué pasa cuando el contenido no cabe en su caja: visible (por defecto, se desborda), hidden (se recorta), scroll (siempre muestra barra de desplazamiento) o auto (solo la muestra si hace falta).

```
<div class="caja-hidden">Este texto es demasiado largo para la caja, así que la parte que no cabe simplemente se recorta y desaparece, sin avisar.</div>
<div class="caja-auto">Este texto es demasiado largo para la caja, así que aparece una barra de desplazamiento y se puede hacer scroll para leer el resto.</div>
```

```
.caja-hidden,
.caja-auto {
  width: 170px;
  height: 70px;
  border: 1px solid #E2DDD2;
  padding: 10px;
}
.caja-hidden { overflow: hidden; }
.caja-auto { overflow: auto; }
```

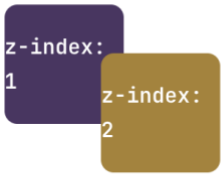


Mismo texto, misma altura de caja fija. Con `hidden` el contenido sobrante desaparece sin rastro; con `auto`, el navegador añade una barra de desplazamiento solo porque hace falta (con `scroll` esa barra aparecería siempre, incluso si el contenido sí cupiera entero).

z-index controla qué elemento queda por encima cuando varios se superponen, pero solo funciona sobre elementos posicionados (cualquier valor de position distinto de static). Un z-index más alto queda más cerca de quien mira la pantalla.

```
<div class="zona">
  <div class="caja dorada">z-index: 2</div>
  <div class="caja morada">z-index: 1</div>
</div>
```

```css
.zona { position: relative; height: 110px; }
.caja {
  position: absolute;
  width: 74px;
  height: 74px;
}
.dorada { top: 40px; left: 70px; z-index: 2; background: #A9812E; }
.morada { top: 10px; left: 10px; z-index: 1; background: #4B3563; }
```



La caja dorada (`z-index: 2`) se dibuja por encima de la morada (`z-index: 1`), aunque en el HTML la morada aparezca en segundo lugar: gana el `z-index` más alto, no el orden del código.

El cursor del ratón: cursor

El puntero del ratón cambia de forma según lo que tiene debajo, y esa forma es información: una mano dice "puedes hacer clic aquí", un cursor de texto dice "puedes escribir o seleccionar", un círculo tachado dice "esto no está disponible". El navegador ya lo hace por su cuenta con los elementos nativos (mano sobre los enlaces, cursor de texto sobre los textos). La propiedad cursor te permite elegirlo tú.

Valor	Para qué sirve
auto	El navegador decide según el contexto (valor inicial).
default	Flecha normal.
pointer	Mano: algo en lo que se puede hacer clic.
text	Cursor de texto (I): texto que se puede seleccionar o editar.
not-allowed	Círculo tachado: acción no permitida (control deshabilitado).
wait	Espera: la aplicación está ocupada y no responde.
progress	Trabajando en segundo plano; todavía se puede interactuar.
help	Interrogación: hay ayuda disponible.
grab	Mano abierta: se puede arrastrar el elemento.
grabbing	Mano cerrada: se está arrastrando.

move	Flechas en cruz: se puede mover.
crosshair	Cruz de precisión.
zoom-in	Lupa con +: al hacer clic se amplía.
copy	Flecha con +: se copiará al soltar.
col-resize	Flechas horizontales: se puede ajustar el ancho (columna).
none	Oculta el cursor.

3.18 Texto y tipografía

Propiedad	Controla
font-family	La familia tipográfica, admite una lista de alternativas de reserva.
font-size	El tamaño del texto.
color	Establece el color de letra utilizado para el texto.
font-weight	El grosor (400 normal, 600/700 semibold/bold).
font-style	normal o italic.
line-height	El interlineado, mejor sin unidad (1.5) que en píxeles, para que escale con el tamaño de fuente.
letter-spacing	El espaciado entre letras.
text-align	Alineación horizontal del texto dentro de su caja.
text-decoration	Subrayado, tachado, etc.
text-transform	uppercase, lowercase, capitalize.

Para usar una tipografía que no está preinstalada, se declara con @font-face antes de usarla (la alternativa a cargarla desde un servicio como Google Fonts.

```
@font-face {
  font-family: "Fraunces";
  src: url("/fuentes/fraunces.woff2") format("woff2");
}

h1 {
  font-family: "Fraunces", serif;
```

```
}
```

3.19 Listas, tablas y formularios

Listas

```
ul {
  list-style: none; /* quita las viñetas por defecto */
  padding-left: 0;
}
```

list-style es la forma abreviada (igual que margin o border) de tres propiedades independientes:

Propiedad	Controla	Valores habituales
list-style-type	La forma de la viñeta.	disc, decimal, square, none...
list-style-position	Si la viñeta va fuera o dentro de la caja del elemento de lista.	outside (por defecto), inside
list-style-image	Sustituye la viñeta por una imagen.	url("viñeta.svg")

Cuando solo necesitas quitar las viñetas, list-style: none; y list-style-type: none; hacen exactamente lo mismo, la forma abreviada es simplemente la más habitual de escribir porque es más corta.

Tablas

```
table {
  border-collapse: collapse; /* une los bordes de celdas contiguas */
  width: 100%;
}
th, td {
  border: 1px solid #E2DDD2;
  padding: 8px 12px;
}
```

Formularios

```
input, textarea {
  border: 1px solid #E2DDD2;
  border-radius: 6px;
  padding: 8px 12px;
  font: inherit; /* los formularios no heredan tipografía por defecto */
}
input:focus {
```

```
  outline: 2px solid #4B3563;
}
```

Nota: Los elementos de formulario son, por herencia histórica del navegador, de los pocos que no heredan la tipografía del resto de la página. font: inherit; es la forma habitual de corregirlo.

3.20 Variables CSS

Las variables CSS (o propiedades personalizadas) guardan un valor reutilizable en cualquier punto del documento. Se declaran con doble guion y se leen con var():

Sin variables

```
.btn-primario { background: #4B3563; }
.link { color: #4B3563; }
.borde-activo { border-color: #4B3563;
```

Con variables

```
:root {
  --color-acento: #4B3563;
}
.btn-primario { background: var(--colo
.link { color: var(--color-acento); }
```

Cambiar el acento de toda la web pasa de "buscar y reemplazar en 40 archivos" a "cambiar una línea en `:root`".

Esto es, literalmente, lo mismo que ya hiciste en Lunacy al crear variables de color: allí las creabas en el panel de Lunacy; aquí las declaras en :root (el elemento raíz del documento, equivalente a html) y funcionan exactamente igual: cámbialas una vez y se actualiza cada lugar que las usa.

Las variables CSS existen en el navegador en tiempo real: se pueden leer y modificar con JavaScript, y su valor puede cambiar dentro de una media query (por ejemplo, para implementar un tema oscuro).

3.21 Transform y transición

transform cambia cómo se dibuja un elemento: lo desplaza, lo agranda o lo encoge, lo gira o lo inclina. Lo más importante que hay que entender de ella es que no altera el diseño de la página: la caja sigue ocupando el mismo sitio en el flujo, los elementos vecinos no se mueven, y solo su aspecto se transforma, como si pusieras la caja sobre una hoja transparente y movieras la hoja. Por eso se usa para efectos y animaciones, y no para colocar elementos en el diseño.

Para ver bien esa diferencia, en todos los ejemplos siguientes hay una caja con una "A" dentro de un hueco punteado que marca el sitio original. Este es el HTML de una de las cajas y el CSS común a todas:

```
<div class="item">
  <div class="hueco"><div class="pieza t1">A</div></div>
  <code>translate(30px, 16px)</code>
```

```
</div>
```

```
.item { display: inline-block; text-align: center; margin: 0 22px 8px 0; vertical-align: top; }

/* El hueco punteado marca dónde estaba la caja antes de transformarla */
.hueco { width: 70px; height: 70px; margin: 22px auto 60px; border: 1px dashed #A9812E; }

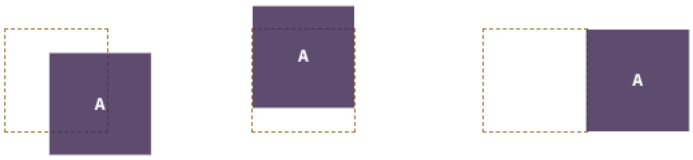
.pieza {
  width: 70px; height: 70px;
  background: rgba(75, 53, 99, .85);
  color: #fff; font-weight: 700; line-height: 70px; text-align: center;
}
```

Desplazar: translate()

translate(x, y) mueve el elemento: x positivo a la derecha, y positivo hacia abajo (en pantalla, el eje vertical crece hacia abajo). Hay atajos para un solo eje: translateX() y translateY(). Los porcentajes se calculan sobre el tamaño del propio elemento (no sobre el contenedor, como en top o margin): translate(100%, 0) mueve la caja exactamente su propio ancho. Esa es la razón de que translate(-50%, -50%) funcione para centrar.

```
<div class="item">
  <div class="hueco"><div class="pieza t1">A</div></div>
  <code>translate(30px, 16px)</code>
</div>
<div class="item">
  <div class="hueco"><div class="pieza t2">A</div></div>
  <code>translateY(-16px)</code>
</div>
<div class="item">
  <div class="hueco"><div class="pieza t3">A</div></div>
  <code>translate(100%, 0)</code>
</div>
```

```
.t1 { transform: translate(30px, 16px); }   /* 30px a la derecha y 16px hacia abajo */
.t2 { transform: translateY(-16px); }       /* solo vertical: negativo = hacia arriba */
.t3 { transform: translate(100%, 0); }      /* 100% = el ancho de la propia caja */
```



`translate(30px, 16px)` `translateY(-16px)` `translate(100%, 0)`

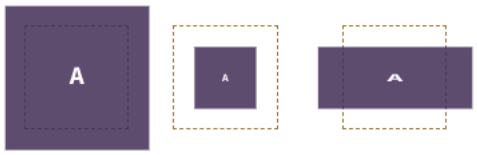
El hueco punteado no se mueve: es el sitio que la caja sigue ocupando. Los vecinos tampoco se mueven. La caja se dibuja desplazada, 30 px a la derecha y 16 px abajo; 16 px arriba; y exactamente su ancho a la derecha.

Escalar: scale()

scale(n) multiplica el tamaño: 1 es el tamaño original, 1.4 un 40 % más grande, .6 un 60 %. Escala todo (fondo, borde y contenido, texto incluido). Con dos valores, scale(x, y), el primero es el factor horizontal y el segundo el vertical, y la caja se deforma. También existen scaleX() y scaleY().

```
<div class="item">
  <div class="hueco"><div class="pieza s1">A</div></div>
  <code>scale(1.4)</code>
</div>
<div class="item">
  <div class="hueco"><div class="pieza s2">A</div></div>
  <code>scale(.6)</code>
</div>
<div class="item">
  <div class="hueco"><div class="pieza s3">A</div></div>
  <code>scale(1.5, .6)</code>
</div>
```

```
.s1 { transform: scale(1.4); }      /* 140% en los dos ejes */
.s2 { transform: scale(.6); }       /* 60%: se encoge */
.s3 { transform: scale(1.5, .6); }  /* ancho x1,5 y alto x0,6: se deforma */
```



`scale(1.4)` `scale(.6)` `scale(1.5, .6)`

La caja crece o se encoge alrededor de su centro. Observa que el hueco no cambia de tamaño: la maquetación no se entera de la transformación.

Girar: rotate()

rotate(ángulo) gira el elemento. El ángulo se escribe con unidad: grados (deg, una vuelta completa son 360), o vueltas (turn: .25turn es un cuarto de vuelta, igual que 90 grados). Un ángulo positivo gira en el sentido de las agujas del reloj; uno negativo, al revés.

```
<div class="item">
  <div class="hueco"><div class="pieza r1">A</div></div>
  <code>rotate(15deg)</code>
</div>
<div class="item">
  <div class="hueco"><div class="pieza r2">A</div></div>
  <code>rotate(-45deg)</code>
</div>
<div class="item">
  <div class="hueco"><div class="pieza r3">A</div></div>
  <code>rotate(.25turn)</code>
</div>
```

```
.r1 { transform: rotate(15deg); }      /* positivo: sentido de las agujas del reloj */
.r2 { transform: rotate(-45deg); }     /* negativo: en sentido contrario */
.r3 { transform: rotate(.25turn); }    /* un cuarto de vuelta, igual que 90deg */
```



`rotate(15deg)` `rotate(-45deg)` `rotate(.25turn)`

Giro de 15 grados, giro de 45 grados en sentido contrario y un cuarto de vuelta. La caja gira alrededor de su centro, y por eso sus esquinas pueden salirse del hueco.

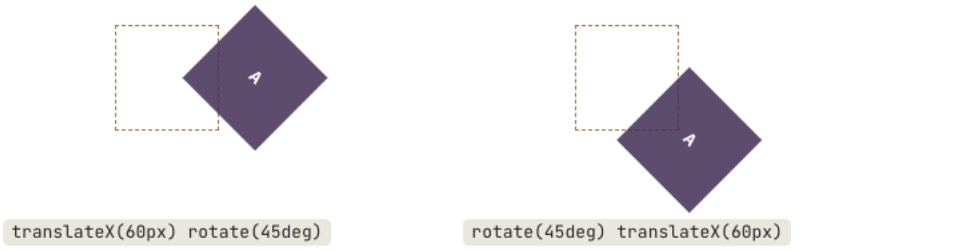
Varias transformaciones a la vez, y el orden

Para combinar transformaciones, se escriben todas en **una sola declaración**, separadas por espacios: transform: translateX(60px) rotate(45deg). El **orden importa**. Cada función cambia el "sistema de coordenadas" para las que vienen detrás: si primero giras, los ejes quedan girados y la siguiente traslación avanza en diagonal; si primero trasladas, los ejes siguen como estaban y el giro se hace ya en el nuevo sitio.

```
<div class="item">
  <div class="hueco"><div class="pieza o1">A</div></div>
  <code>translateX(60px) rotate(45deg)</code>
</div>
<div class="item">
```

```
<div class="hueco"><div class="pieza o2">A</div></div>
<code>rotate(45deg) translateX(60px)</code>
</div>
```

```
.o1 { transform: translateX(60px) rotate(45deg); }
.o2 { transform: rotate(45deg) translateX(60px); }
```



Las mismas dos funciones en distinto orden. En la primera, la caja se desplaza 60 px a la derecha y gira en ese sitio. En la segunda, gira primero y entonces avanza 60 px a lo largo de su eje ya girado, de modo que acaba más abajo.

Transiciones: animar el cambio

Sin más, un cambio de estilo ocurre de golpe: al pasar el cursor, la caja salta al nuevo valor. transition pide al navegador que, en lugar de saltar, recorra los valores intermedios durante un tiempo. Se define en cuatro partes:

Propiedad	Qué indica	Valor inicial
transition-property	Qué propiedad(es) se animan: transform, background-color, opacity... o all para todas.	all
transition-duration	Cuánto dura: .3s o 300ms.	0s (no hay animación)
transition-timing-function	El ritmo: linear (velocidad constante), ease (empieza despacio, acelera y frena), ease-in (arranca despacio), ease-out (frena al final), ease-in-out, steps(n) (a saltos) o una curva propia con cubic-bezier().	ease
transition-delay	Cuánto se espera antes de empezar.	0s

Lo normal es usar la abreviatura transition: propiedad duración ritmo retraso;. En ella, el primer tiempo es la duración y el segundo, el retraso. Se pueden animar varias propiedades separándolas por comas: transition: transform .25s ease-out, opacity .25s;. El ritmo es lo que más influye en la sensación de la animación. Cinco bolas con el mismo recorrido y el mismo tiempo, y distinto ritmo:

```
<div class="pistas">
<div class="pista"><span class="bola b-linear"></span><code>linear</code></div>
```

```
  <div class="pista"><span class="bola b-ease"></span><code>ease</code></div>
  <div class="pista"><span class="bola b-in"></span><code>ease-in</code></div>
  <div class="pista"><span class="bola b-out"></span><code>ease-out</code></div>
  <div class="pista"><span class="bola b-steps"></span><code>steps(5)</code></div>
</div>
```

```
.pista { position: relative; height: 30px; margin-bottom: 8px; border-bottom: 1px solid #E2DDD2; }
.bola { position: absolute; left: 0; top: 2px; width: 22px; height: 22px; border-radius: 50%; background:
#4B3563; }
.pista code { position: absolute; right: 0; bottom: 4px; }

/* La transición va en el estado base: define cuánto dura y con qué ritmo */
.b-linear { transition: left 1.6s linear; }
.b-ease   { transition: left 1.6s ease; }
.b-in     { transition: left 1.6s ease-in; }
.b-out    { transition: left 1.6s ease-out; }
.b-steps  { transition: left 1.6s steps(5); }

/* El cambio se dispara al pasar el cursor por el grupo */
.pistas:hover .bola { left: calc(100% - 22px); }
```

Todo junto: una tarjeta que se eleva

Unimos las dos ideas: transform eleva la tarjeta al pasar el cursor y transition hace que el movimiento sea suave, tanto al entrar como al salir (porque la transición está en el estado base):

```
.card {
  transition: transform .25s ease, box-shadow .25s ease;
}
.card:hover {
  transform: translateY(-6px);
  box-shadow: 0 14px 24px rgba(75,53,99,.22);
}
```

Actividad · WebCompleta 2

El HTML es el de la solución del Paso 1. Ahora dale estilo, siguiendo el orden marcado en los comentarios del propio archivo. Todavía no uses display: flex ni display: grid:, así que los elementos seguirán apilados uno debajo de otro.

1. En :root, declara: --brand: #4F46E5, --brand-dark: #4338CA, --brand-soft: #EEF2FF, --accent: #F59E0B, --ink: #111827, --ink-soft: #6B7280, --bg: #FFFFFF, --bg-alt: #F9FAFB, --line: #E5E7EB.
2. Cabecera: fondo blanco y borde inferior de 1px en --line. El .logo en --brand, negrita, sin subrayado. Los enlaces de nav en --ink-soft, sin subrayado, con un :hover que los cambie a --brand.

3. Botones .btn: display: inline-block, relleno, border-radius, sin subrayado. .btn-primary con fondo --brand y texto blanco; en :hover, fondo --brand-dark y transform: translateY(-2px) con transition. .btn-secondary con borde de 1.5px en --brand y fondo transparente.
4. Hero: fondo con un linear-gradient suave de --brand-soft a --bg, texto centrado, mucho relleno vertical. El h1 grande, con un max-width para que no ocupe todo el ancho.
5. Tarjetas de características: fondo blanco, borde --line, border-radius, box-shadow sutil, y en :hover que se eleven con transform + transition (mismo patrón que los botones).
6. Tarjetas de precio: igual que las de características, y además: borde de 2px en --brand para .price-card--featured, con position: relative en la tarjeta y un ::before con content: "Más popular" y position: absolute en la esquina superior.
7. Testimonios: borde izquierdo grueso en --brand, fondo --bg-alt, texto en cursiva, y una comilla grande decorativa con ::before y content: "\201C".
8. Pie de página: fondo --ink, texto claro, enlaces que cambien de color en :hover.

3.22 Introducción a media queries

Una media query aplica un bloque de reglas CSS solo cuando se cumple una condición sobre el dispositivo, casi siempre el ancho de la ventana:

```
.card {
  max-width: 280px;
}

/* Solo en pantallas de 600px de ancho o menos */
@media (max-width: 600px) {
  .card {
    max-width: 100%;
  }
}
```

3.23 Validación de hojas de estilo

Igual que el HTML se puede validar contra el estándar del W3C, existe un validador de CSS oficial (jigsaw.w3.org/css-validator) que revisa un archivo, una URL o un fragmento de código en busca de errores de sintaxis, propiedades mal escritas o valores no válidos.

En el día a día, las herramientas de desarrollador del navegador (pestaña Elements/Inspector, panel de estilos calculados) cumplen una función complementaria: no validan contra el estándar, pero muestran exactamente qué regla está ganando la cascada para cada propiedad, algo imprescindible para depurar problemas de especificidad.

Nota: Un CSS "inválido" no siempre falla de forma visible: un navegador simplemente ignora la declaración que no entiende y sigue con la siguiente. Eso hace que los errores

de CSS sean, a menudo, silenciosos, y es la razón principal para validar antes de dar por terminado un archivo de estilos.

4. Flexbox

Esta sección resuelve el problema siguiente, mucho más frecuente en el día a día: cómo organizar varios elementos entre sí dentro de un contenedor. Flexbox es la primera de las dos herramientas modernas de CSS pensadas exactamente para esto (la segunda, CSS Grid, la veremos más adelante).

4.1 Qué es Flexbox y qué problema resuelve

Flexbox (Flexible Box Layout) es un modelo de maquetación de CSS pensado para distribuir el espacio entre los elementos de un contenedor en una sola dimensión: una fila o una columna. Antes de que existiera (se estandarizó a principios de la década de 2010), tareas hoy triviales exigían soluciones incómodas: centrar algo verticalmente, repartir varias columnas de igual altura, o cambiar el orden visual de los elementos sin tocar el HTML eran problemas genuinamente difíciles de resolver.

Con Flexbox, esos problemas se resuelven con una o dos propiedades. Algunos de los casos de uso más habituales:

- Distribuir los enlaces de un menú de navegación con espacio uniforme entre ellos.
- Centrar cualquier elemento, tanto horizontal como verticalmente, dentro de su contenedor.
- Construir una fila de tarjetas que mantengan la misma altura, aunque su contenido varíe.
- Reordenar visualmente elementos (por ejemplo, para pantallas pequeñas) sin alterar el orden del HTML.

Nota: "Una sola dimensión" es la clave para saber cuándo usar Flexbox y cuándo CSS Grid: si estás organizando elementos en fila **o** en columna, Flexbox es la herramienta natural. Si necesitas controlar filas **y** columnas a la vez, como una retícula real, Grid encaja mejor. En la práctica, la mayoría de interfaces usan ambas a la vez, cada una donde corresponde.

4.2 Activar Flexbox: contenedor y elementos

Flexbox se activa con una única propiedad, aplicada al elemento que envuelve a los que quieres organizar:

```
.contenedor {
  display: flex;
}
```

En cuanto un elemento tiene display: flex, pasa a llamarse contenedor flex (flex container), y todos sus hijos directos (solo los directos, no los nietos) se convierten automáticamente en elementos flex (flex items), sin que tengas que hacer nada más sobre ellos. Por defecto, esos elementos se colocan en una fila, uno junto a otro.



Tres `<div>` normales (que por defecto ocuparían todo el ancho, uno debajo de otro) pasan a colocarse en fila en cuanto su padre recibe `display: flex`. No hace falta ninguna otra propiedad en los hijos.

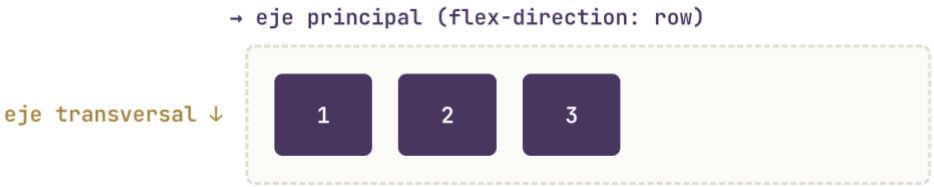
Existe también display: inline-flex, que hace exactamente lo mismo pero tratando al propio contenedor como un elemento en línea (útil cuando quieres que el contenedor flex conviva en la misma línea que otro texto, en lugar de ocupar todo el ancho disponible).

4.3 Los dos ejes: principal y transversal

Todo el vocabulario de Flexbox gira en torno a un único concepto, y es, con diferencia, lo más importante de todo el tema: cada contenedor flex tiene un eje principal (main axis) y un eje transversal (cross axis), perpendicular al anterior. Qué dirección tiene cada uno lo decide la propiedad flex-direction:

Valor	Eje principal
row (por defecto)	Horizontal, de izquierda a derecha.
row-reverse	Horizontal, de derecha a izquierda.
column	Vertical, de arriba a abajo.
column-reverse	Vertical, de abajo a arriba.

El eje transversal es siempre el perpendicular al principal: si flex-direction es row, el eje transversal es vertical; si es column, el eje transversal es horizontal.

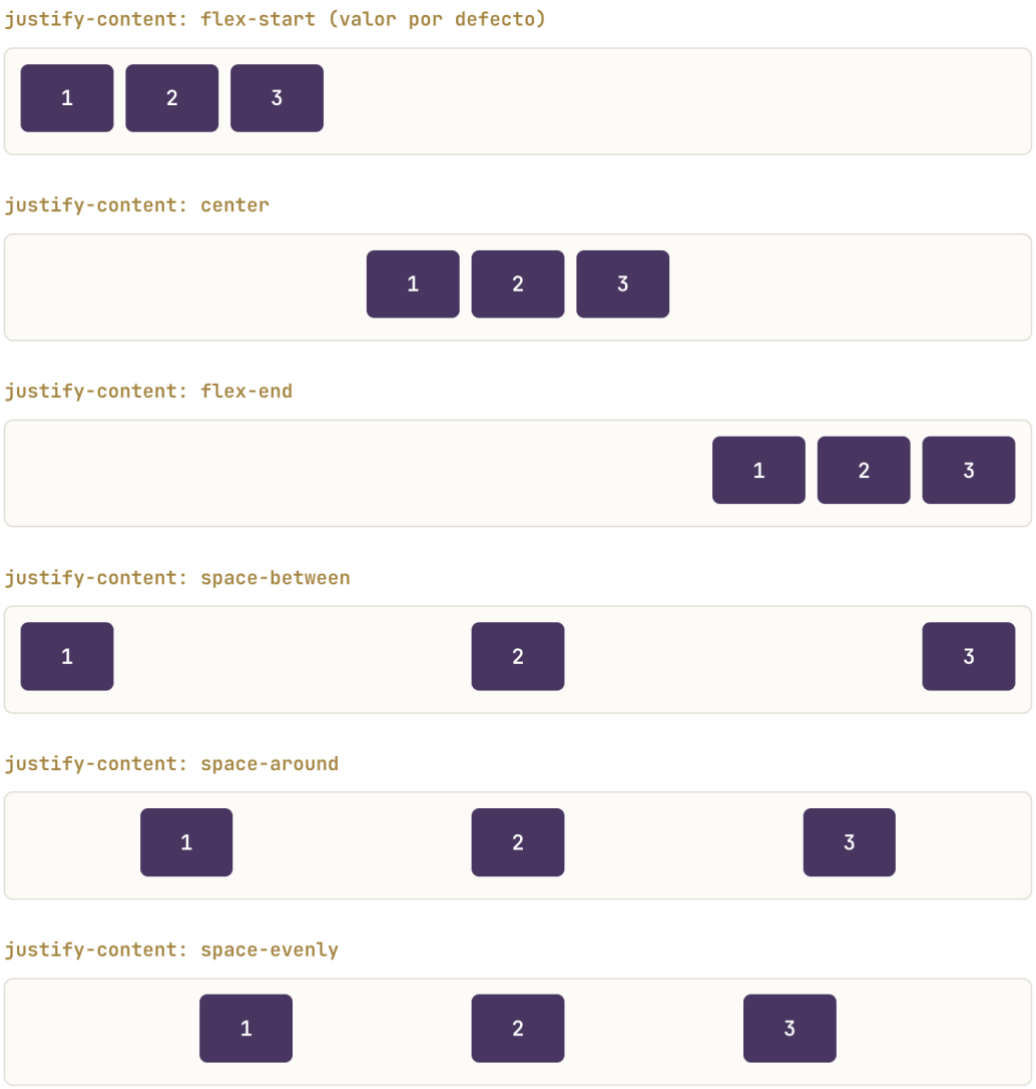


Con flex-direction: row (el valor por defecto), el eje principal es horizontal y el transversal es vertical. Las propiedades de alineación siempre se definen en relación a estos dos ejes, no en términos absolutos de "horizontal" y "vertical".

Importante: Esta es la razón por la que justify-content reparte el espacio horizontalmente cuando flex-direction es row, pero verticalmente en cuanto cambias a flex-direction: column. No es una excepción rara: justify-content siempre actúa sobre el eje principal, sea cual sea su dirección.

4.4 Alineación en el eje principal: justify-content

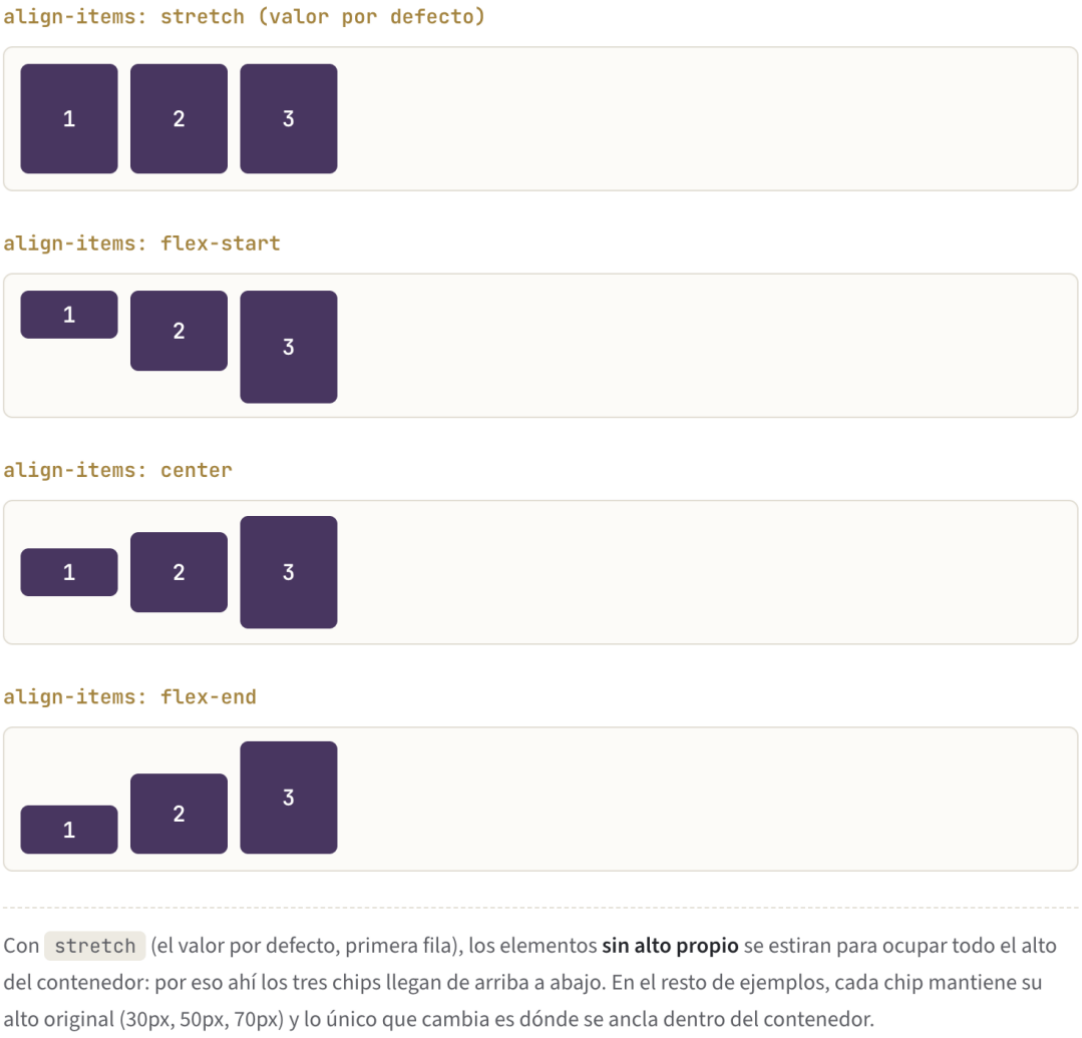
justify-content decide cómo se reparte el espacio sobrante del contenedor **a lo largo del eje principal**, una vez colocados todos los elementos con su tamaño natural.



Los tres mismos elementos, sin cambiar nada más que `justify-content` en el contenedor. Fíjate en la diferencia entre los tres últimos: `space-between` no deja hueco en los extremos; `space-around` reparte hueco alrededor de cada elemento (el hueco entre elementos es el doble que el de los extremos); `space-evenly` deja exactamente el mismo hueco en todos los huecos, extremos incluidos.

4.5 **Alineación en el eje transversal: align-items**

align-items hace, sobre el eje transversal, el trabajo equivalente al de justify-content sobre el principal: decide cómo se alinean los elementos en la dimensión perpendicular a la fila (o columna).



Con `stretch` (el valor por defecto, primera fila), los elementos **sin alto propio** se estiran para ocupar todo el alto del contenedor: por eso ahí los tres chips llegan de arriba a abajo. En el resto de ejemplos, cada chip mantiene su alto original (30px, 50px, 70px) y lo único que cambia es dónde se ancla dentro del contenedor.

Un quinto valor, baseline, alinea los elementos por la línea base de su texto (útil cuando los elementos contienen texto de distinto tamaño y quieres que "se lean" alineados, en lugar de alinear sus cajas).

Nota: align-items se define en el contenedor y afecta a todos los elementos a la vez. Si necesitas que uno solo de ellos se comporte de forma distinta al resto, no cambies align-items: usa align-self sobre ese elemento en concreto.

4.6 Envolver elementos: flex-wrap y align-content

Por defecto, Flexbox intenta colocar todos los elementos en una única línea, encogiéndolos si hace falta para que quepan. Cuando eso no es lo que quieres (por ejemplo, una fila de tarjetas que debe pasar a la siguiente línea en lugar de aplastarse), se lo indicas con flex-wrap:

Valor	Comportamiento
nowrap (por defecto)	Todos los elementos en una sola línea, encogiéndose si no caben.
wrap	Los elementos que no quepan pasan a una nueva línea, de arriba a abajo.

wrap-reverse	Igual que wrap, pero las nuevas líneas se añaden de abajo a arriba.



Cinco cajas de 80px (400px en total) dentro de un contenedor de solo 360px. Con `nowrap` se encogen todas un poco para caber en una única línea; con `wrap`, en cambio, mantienen su ancho de 80px y las que no caben pasan a una segunda línea.

Cuando flex-wrap: wrap genera varias líneas, aparece un nuevo problema: ¿cómo se reparten esas líneas entre sí en el eje transversal, si sobra espacio? Eso es exactamente lo que resuelve align-content (con las mismas opciones que justify-content: flex-start, center, space-between...). Es fácil confundirla con align-items, así que conviene distinguirlas bien:

Propiedad	Actúa sobre
align-items	Cómo se alinea cada elemento **dentro de su línea**.
align-content	Cómo se reparten **las líneas completas** entre sí. Solo tiene efecto visible si hay más de una línea (con flex-wrap: wrap) y sobra espacio en el eje transversal.

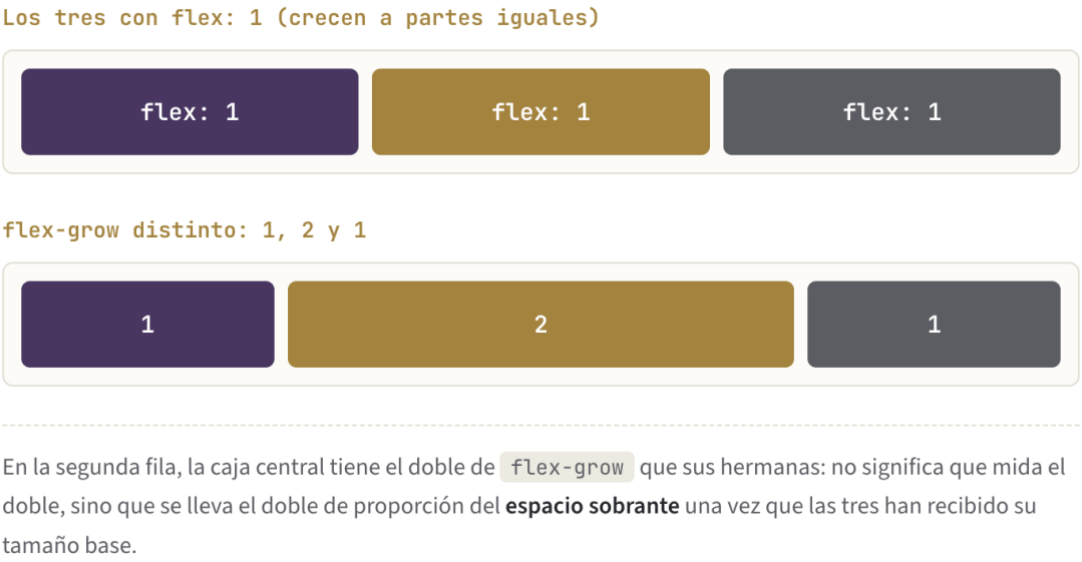
4.7 Tamaño de los elementos: flex-grow, flex-shrink, flex-basis

Hasta ahora hemos alineado elementos con un tamaño fijo. Flexbox también controla, de forma muy precisa, cómo crecen o se encogen los elementos para repartirse el espacio disponible. Son tres propiedades que se definen en los elementos (no en el contenedor) y casi siempre se escriben juntas con el atajo flex:

Propiedad	Controla
flex-grow	Cuánto **crece** un elemento, en proporción a sus hermanos, para ocupar el espacio sobrante. 0 (por defecto) significa que no crece nada.
flex-shrink	Cuánto se **encoge** un elemento, en proporción a sus hermanos, cuando no hay espacio suficiente. 1 (por defecto) significa que sí se encoge.
flex-basis	El tamaño de partida del elemento, antes de aplicar el crecimiento o el encogimiento. Admite cualquier unidad, o auto (el tamaño natural del contenido).

```css
/* La forma abreviada, en el orden grow shrink basis */
.item {
  flex: 1 1 200px;
}

/* El atajo más habitual en la práctica: "ocupa todo el espacio que puedas" */
.item {
  flex: 1; /* equivale a flex: 1 1 0 */
}
```



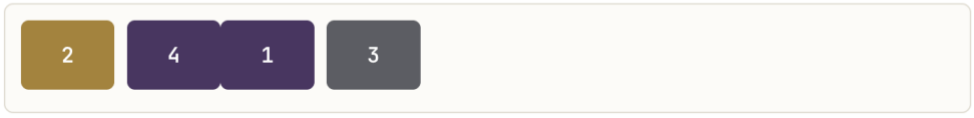
Importante: Un error de principiante muy común es escribir flex-grow: 1; esperando que el elemento ocupe "todo el ancho posible" y que no funcione como se espera. Repasa flex-basis: si no lo defines, su valor por defecto dentro del atajo flex: 1 es 0, así que el elemento parte de tamaño cero y crece desde ahí. Si necesitas que además respete un tamaño mínimo razonable antes de crecer, usa algo como flex: 1 1 150px;.

4.8 Reordenar y alinear individualmente: order y align-self

order

Por defecto, todos los elementos flex tienen order: 0 y se muestran en el mismo orden en que aparecen en el HTML. Asignar un valor distinto de order a un elemento cambia su posición visual: los elementos se muestran ordenados de menor a mayor valor de order, sin importar su orden real en el código.

HTML en orden 1, 2, 3, 4 — visualmente reordenado con CSS



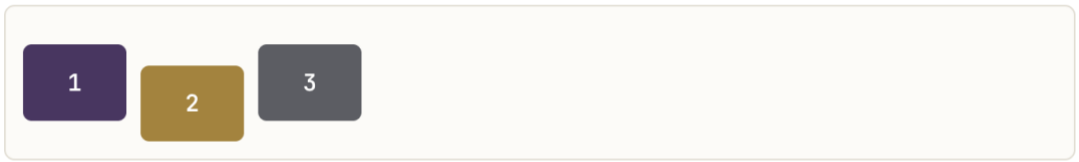
El HTML declara los elementos en el orden 1, 2, 3, 4. Con `order: 3`, `order: 1`, `order: 4` y `order: 2` respectivamente, se muestran como 2, 4, 1, 3.

Importante: order cambia el orden visual, pero no el orden real del documento: la navegación por teclado (Tab) y los lectores de pantalla siguen el orden del HTML, no el visual. Usar order para maquetar sin más suele estar bien, pero abusar de él para reorganizar el contenido de forma muy distinta a su lectura lógica es una fuente frecuente de problemas de accesibilidad.

align-self

align-self se aplica sobre un elemento individual y admite los mismos valores que align-items (flex-start, center, flex-end, stretch, baseline). Cuando se define, **sobrescribe** para ese elemento concreto el valor de align-items heredado del contenedor.

align-items: center en el contenedor, align-self: flex-end en el elemento 2



Las cajas 1 y 3 heredan `align-items: center` del contenedor. La caja 2 lo ignora gracias a `align-self: flex-end`, propio y exclusivo de ese elemento.

4.9 Espaciado entre elementos: gap

Antes de gap, separar elementos flex obligaba a añadir margin a cada uno y luego corregir a mano el margen sobrante en el primero o el último, para que no quedara un hueco extra en el borde del contenedor. gap resuelve esto de forma directa: define el espacio entre elementos, sin afectar nunca al borde exterior del contenedor.

```
.contenedor {
  display: flex;
  gap: 16px; /* separación en ambos ejes */
}

/* También se puede controlar cada eje por separado */
.contenedor {
  row-gap: 16px;
  column-gap: 24px;
}
```

Nota: gap funciona igual de bien con flex-wrap: wrap: separa tanto los elementos dentro de cada línea como las líneas entre sí, con un único valor y sin ningún cálculo manual.

4.10 Patrones habituales con Flexbox

Con lo visto hasta ahora ya puedes resolver, en una o dos líneas de CSS, varios de los problemas de maquetación más frecuentes.

Barra de navegación

```css
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 24px;
}
```



El logotipo a un extremo y los enlaces al otro, centrados verticalmente entre sí, sin una sola regla de posicionamiento (Tema 8.12).

Centrado perfecto

```css
.contenedor {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 300px;
}
```



El centrado vertical, históricamente uno de los problemas más incómodos de resolver en CSS antes de Flexbox, se reduce a estas dos propiedades.

Fila de tarjetas de igual altura

```css
.tarjetas {
  display: flex;
  gap: 16px;
}
```

```
.tarjeta {
  flex: 1; /* todas ocupan el mismo ancho */
}
```

Gracias a que align-items vale stretch por defecto, todas las tarjetas alcanzan la altura de la más alta del grupo de forma automática, sin fijar ninguna altura a mano.

Layout de página completo

```
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
main {
  flex: 1; /* empuja el footer hasta el fondo, aunque haya poco contenido */
}
```

Este patrón (conocido como *sticky footer*) resuelve un problema clásico: que el pie de página se quede "flotando" a media pantalla cuando el contenido es corto, en lugar de asentarse en la parte inferior. Con main creciendo para ocupar todo el espacio sobrante (flex: 1), el footer queda siempre pegado abajo, tenga el contenido la altura que tenga.

Nota: El patrón "holy grail" (header + nav/main/aside + footer) se puede construir combinando dos contenedores flex anidados: uno en column para las tres franjas verticales (encabezado, cuerpo, pie), y dentro del cuerpo, otro en row para la navegación, el contenido principal y la barra lateral.

Actividad · WebCompleta 3

El HTML es el de la solución del Paso 2. Ahora completa lo siguiente.

1. **.nav-wrap:** añade display: flex, align-items: center y justify-content: space-between, para que el logo quede a un lado y el grupo de enlaces + el botón al otro.
2. **nav:** sustituye margin: 14px 0; por display: flex, align-items: center y gap: 24px (el hueco entre enlaces pasa a controlarlo el gap, no un margen).
3. **nav a:** borra la línea margin-right: 20px;, ya no hace falta.
4. **.testimonials-grid:** no existe todavía como regla. Créala con display: flex, flex-wrap: wrap y gap: 24px, para que los tres testimonios se coloquen en fila y bajen a una segunda línea si no caben.
5. **.quote:** añade flex: 1 1 280px; (así cada testimonio ocupa como mínimo 280px y se reparte el resto), y borra margin: 0 0 20px;: el espacio entre tarjetas ya lo da el gap del contenedor.

El resto del archivo (cabecera, botones, hero, tarjetas...) no hace falta tocarlo.

5. CSS Grid

Flexbox organiza elementos en una sola dimensión. CSS Grid controla filas y columnas a la vez.

5.1 Qué es CSS Grid y en qué se diferencia de Flexbox

CSS Grid es un modelo de maquetación pensado para controlar, a la vez, filas y columnas: una auténtica rejilla bidimensional, en lugar de la única fila o columna que organiza Flexbox. Es la herramienta natural para la estructura general de una página, mientras que Flexbox suele encajar mejor dentro de los componentes que viven sobre esa retícula (una barra de navegación, los botones de una tarjeta).

	Flexbox	CSS **Grid**
Dimensiones	Una (fila *o* columna)	Dos (filas *y* columnas a la vez)
Punto de partida	El tamaño del contenido decide el espacio	La rejilla se define primero; el contenido se coloca dentro
Caso de uso típico	Un menú, una fila de botones, una tarjeta	La estructura general de una página completa

Nota: No son herramientas rivales: en un proyecto real conviven todo el tiempo. Es habitual usar Grid para la retícula general de una página y, dentro de cada celda de esa retícula, Flexbox para organizar los elementos internos de un componente.

5.2 Activar Grid: contenedor y elementos

Igual que con Flexbox, activar Grid es cuestión de una única propiedad en el contenedor:

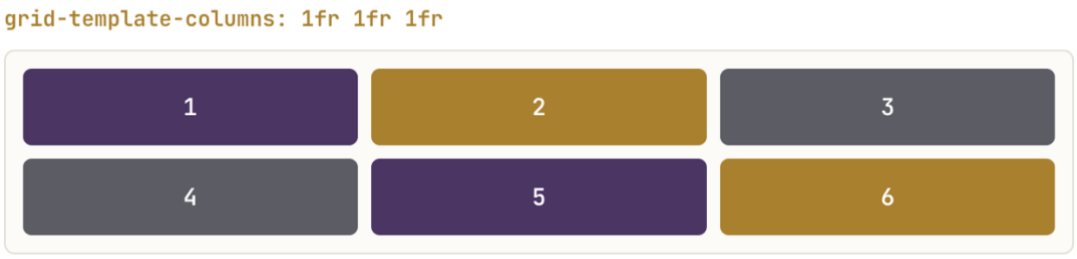
```
.contenedor {
  display: grid;
}
```

Sus hijos directos se convierten automáticamente en elementos de la rejilla (grid items). La diferencia con Flexbox es que, sin ninguna propiedad adicional, display: grid por sí solo no hace demasiado visible: coloca los elementos en una única columna, uno debajo de otro, porque todavía no le hemos dicho a la rejilla cuántas columnas o filas tiene. Eso es exactamente lo que resuelve el siguiente apartado.

5.3 Definir columnas y filas

grid-template-columns y **grid-template-rows** definen cuántas columnas y filas tiene la rejilla, y qué tamaño tiene cada una. Cada valor de la lista es una pista (track):

```
.contenedor {
  display: grid;
  grid-template-columns: 200px 200px 200px; /* 3 columnas de 200px */
  grid-template-rows: 100px 100px; /* 2 filas de 100px */
}
```



Con tres columnas definidas, los seis elementos se reparten automáticamente en dos filas de tres, sin que hayamos definido grid-template-rows: Grid crea las filas que hagan falta.

Puedes mezclar unidades libremente: grid-template-columns: 200px 1fr 2fr; es perfectamente válido, y es habitual combinar una columna de ancho fijo (una barra lateral) con columnas flexibles (el contenido principal). La unidad fr merece su propio apartado, porque es el verdadero motor de Grid.

5.4 La función repeat() y la unidad fr

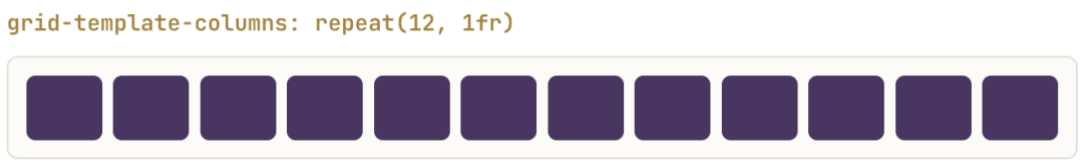
La unidad fr (fracción) representa una porción del espacio disponible en el contenedor, repartida en proporción al resto de columnas en fr. Es, para las columnas de Grid, el equivalente casi exacto de flex-grow en Flexbox.

```
/* Tres columnas iguales, cada una un tercio del espacio disponible */
.contenedor {
  grid-template-columns: 1fr 1fr 1fr;
}

/* Una barra lateral fija + el resto del espacio para el contenido */
.contenedor {
  grid-template-columns: 240px 1fr;
}
```

Escribir 1fr repetido muchas veces es tedioso (y propenso a errores de conteo) en rejillas con muchas columnas, así que Grid ofrece la función repeat():

```
/* Equivalen exactamente a lo mismo */
.doce-columnas { grid-template-columns: 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr; }
.doce-columnas { grid-template-columns: repeat(12, 1fr); }
```



repeat() también admite patrones más complejos, repitiendo varios valores a la vez: repeat(4, 1fr 2fr) genera ocho columnas, alternando el patrón "estrecha, ancha" cuatro veces.

5.5 Espaciado: gap

gap funciona en Grid exactamente igual que en Flexbox: define la separación entre columnas y filas, sin afectar al borde exterior del contenedor.

```
.contenedor {
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  gap: 24px; /* misma separación en filas y columnas */
}

/* O por separado */
.contenedor {
  row-gap: 32px;
  column-gap: 24px;
}
```

5.6 Posicionar elementos con líneas de la rejilla

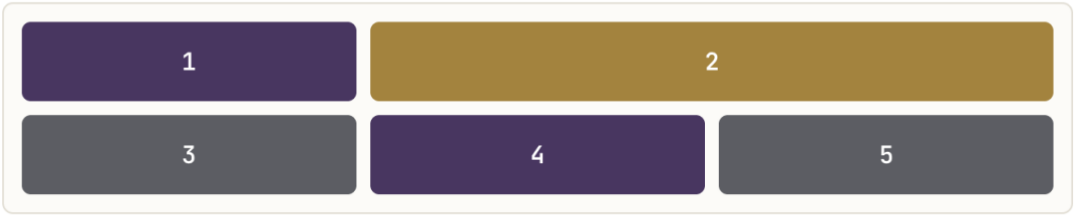
Al crear columnas y filas, Grid numera automáticamente las líneas que quedan entre ellas, empezando en 1. Una rejilla de 3 columnas tiene 4 líneas verticales (antes de la primera, entre cada par, y después de la última). Puedes decirle a un elemento que ocupe desde una línea hasta otra, saltándose la colocación automática:

```
.destacado {
  grid-column: 1 / 3; /* desde la línea 1 hasta la línea 3: ocupa 2 columnas */
}
```

También puedes indicar cuántas pistas quieres ocupar, sin calcular en qué línea termina, con la palabra clave span:

```
.destacado {
  grid-column: span 2; /* ocupa 2 columnas a partir de donde le toque */
}
```

El elemento 2 tiene grid-column: span 2



El elemento 2 ocupa el espacio de dos columnas; el resto sigue colocándose automáticamente en el hueco que queda, fila a fila.

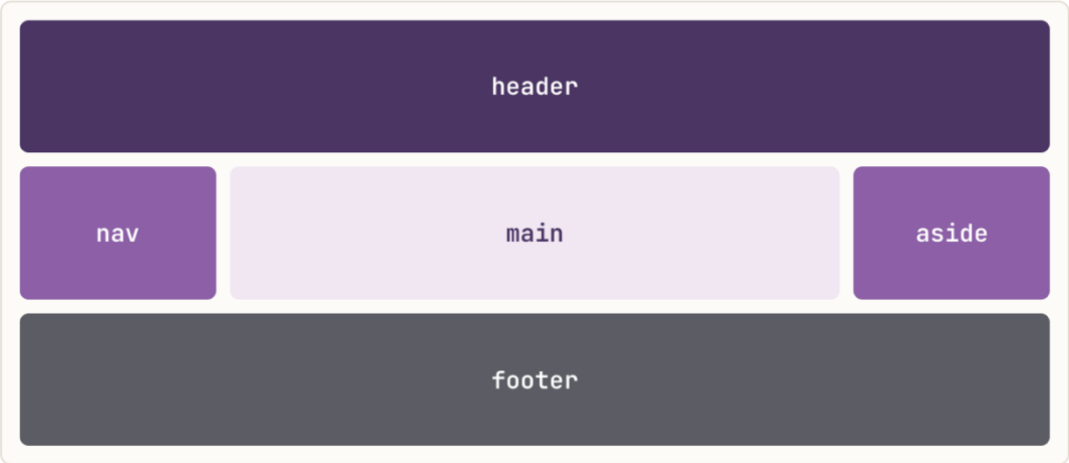
grid-row funciona exactamente igual que grid-column, pero sobre las líneas horizontales, para hacer que un elemento ocupe varias filas en lugar de varias columnas.

5.7 **Áreas con nombre: grid-template-areas**

Contar líneas funciona, pero para una estructura de página completa existe una forma mucho más legible: nombrar las regiones de la rejilla y "dibujarlas" directamente en el CSS, casi como un boceto en ASCII.

```
.pagina {
  display: grid;
  grid-template-columns: 200px 1fr 200px;
  grid-template-areas:
    "header header header"
    "nav    main   aside"
    "footer footer footer";
}

header { grid-area: header; }
nav    { grid-area: nav; }
main   { grid-area: main; }
aside  { grid-area: aside; }
footer { grid-area: footer; }
```



Cada nombre del grid-template-areas se corresponde con el nombre usado en grid-area de cada elemento. La forma del CSS literalmente dibuja la forma de la página.

Importante: Cada fila de grid-template-areas debe tener el mismo número de nombres que columnas tenga la rejilla, y las celdas de un mismo nombre deben formar siempre un rectángulo. Si quieres dejar una celda vacía (sin ningún elemento asignado), usa un punto (.) en su lugar.

5.8 Alineación en Grid

Grid separa la alineación en dos parejas de propiedades, según si actúan sobre el contenido de cada celda o sobre la rejilla completa dentro de su contenedor:

Propiedad	Se define en	Alinea
justify-items	Contenedor	Todos los elementos, horizontalmente, dentro de su propia celda.
align-items	Contenedor	Todos los elementos, verticalmente, dentro de su propia celda.
justify-self / align-self	Elemento	Ese elemento en concreto, sobrescribiendo lo anterior.
justify-content	Contenedor	La rejilla entera, horizontalmente, si sobra espacio en el contenedor.
align-content	Contenedor	La rejilla entera, verticalmente, si sobra espacio en el contenedor.

justify-items: center; align-items: center;



Sin `justify-items` / `align-items`, cada elemento se estira por defecto para llenar toda su celda (igual que `stretch` en Flexbox). Aquí, con ambos en `center`, cada chip se centra dentro del espacio de su celda, que sigue siendo mucho más grande que el chip.

Nota: Los valores modernos de Grid son start, end, center y stretch (sin el prefijo flex- que usa Flexbox). La mayoría de navegadores también aceptan flex-start/flex-end como alias por compatibilidad, pero lo correcto en Grid es escribir start y end a secas.

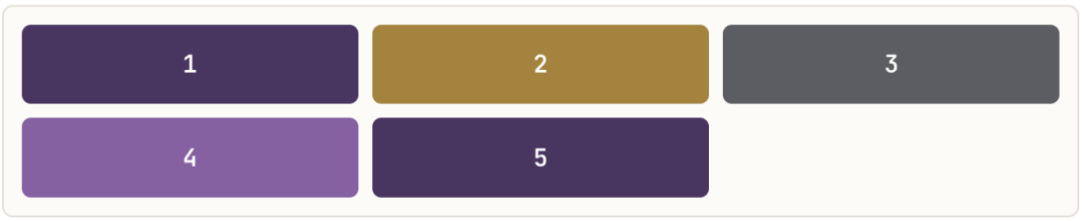
5.9 Rejillas responsive: auto-fit, auto-fill y minmax()

Uno de los trucos más apreciados de Grid es construir una retícula de tarjetas que se adapta sola al ancho disponible, sin escribir ni una sola media query.

```
.tarjetas {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
  gap: 12px;
}
```

minmax(160px, 1fr) le dice a cada columna: "no bajes nunca de 160px, pero si sobra espacio, repártetelo como un fr normal". auto-fit le dice a Grid: "calcula tú misma cuántas columnas de al menos 160px caben en el ancho disponible, y crea esas columnas automáticamente".

grid-template-columns: repeat(auto-fit, minmax(140px, 1fr))



El número de columnas de esta rejilla cambia solo, sin ninguna media query detrás. Al estrechar la ventana, algunas tarjetas bajan a la siguiente fila; al ensancharla, vuelven a subir.

La diferencia entre auto-fit y auto-fill es sutil pero importante: si hay menos elementos de los que caben columnas, auto-fit estira las columnas existentes para ocupar todo el ancho (colapsando las columnas vacías a 0), mientras que auto-fill mantiene el hueco de esas columnas vacías, sin estirar las que sí tienen contenido. En la práctica, auto-fit es el que casi siempre se quiere para una retícula de tarjetas.

5.10 CSS Grid frente a Flexbox: cuándo usar cada uno

Situación	Herramienta
La estructura general de una página (encabezado, navegación, contenido, barra lateral, pie)	CSS Grid
Una retícula de tarjetas de producto o de artículos	CSS Grid
Una barra de navegación (logotipo + enlaces)	Flexbox
Centrar un elemento dentro de otro	Cualquiera de las dos
Los botones de una tarjeta, o una lista de etiquetas	Flexbox
Un formulario con etiquetas y campos alineados en columnas	CSS Grid

La pregunta rápida para decidir: si al describir el problema usas la palabra "y" ("necesito controlar las columnas y las filas"), es Grid. Si solo necesitas ordenar cosas en una fila o en una columna, es Flexbox.

Actividad · WebCompleta 4
El HTML es el de la solución del Paso 3. Ahora completa lo siguiente.
1. **.features-grid:** no existe todavía como regla. Créala con display: grid, grid-template-columns: repeat(2, 1fr) (dos columnas iguales) y gap: 24px.
2. **.feature-card:** borra la línea margin-bottom: 20px;, el hueco entre tarjetas ya lo da el gap del contenedor.
3. **.pricing-grid:** tampoco existe todavía. Créala con display: grid, grid-template-columns: repeat(3, 1fr) (tres columnas iguales, una por plan) y gap: 24px.
4. **.price-card:** borra la línea margin-bottom: 24px;, por la misma razón que en las tarjetas de características.
El resto del archivo (cabecera, botones, hero, testimonios...) ya está resuelto desde pasos anteriores: no hace falta tocarlos. Con poco ancho de ventana, estas dos rejillas se verán apretadas a propósito: eso es justo lo que arreglaremos en el Paso 5 con media queries.

6. Diseño responsive

Ya sabes construir la estructura de una página con Grid y organizar sus componentes con Flexbox. Falta la pieza que hace que esa misma página funcione igual de bien en un móvil, una tableta o un monitor de escritorio: el diseño responsive.

6.1 Por qué el diseño responsive

El término diseño responsive (responsive web design) lo acuñó el diseñador Ethan Marcotte en 2010, para describir una alternativa a lo que hasta entonces era habitual: mantener una web "de escritorio" y, en paralelo, una versión "móvil" completamente aparte (a menudo en un subdominio como m.ejemplo.com). Su propuesta combinaba tres ingredientes que ya conoces por separado: retículas fluidas, imágenes flexibles y

media queries, para que un único HTML y una única hoja de estilos se adaptaran a cualquier tamaño de pantalla.

Hoy es, sencillamente, la forma estándar de construir para la web: nadie mantiene dos sitios separados, y las herramientas que hemos visto en los tres temas anteriores (Flexbox, Grid, y ahora las media queries) son, precisamente, las que lo hacen posible.

6.2 El viewport y su meta etiqueta

El viewport es el área visible de la página dentro del navegador. En un móvil, sin ninguna instrucción adicional, los navegadores asumían históricamente que toda web estaba diseñada para escritorio: renderizaban la página a un ancho de unos 980px y luego la encogían para que "cupiera" en la pantalla, obligando a hacer zoom para leer cualquier cosa. La etiqueta <meta name="viewport">, en el <head> del documento, evita ese comportamiento:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Valor	Qué indica
width=device-width	El ancho del viewport debe coincidir con el ancho real del dispositivo, en lugar de un ancho de escritorio fijo.
initial-scale=1.0	El nivel de zoom inicial, sin ampliar ni reducir nada al cargar la página.

Importante: Sin esta etiqueta, ninguna media query basada en el ancho de pantalla funcionará correctamente en un móvil: el navegador seguirá creyendo que tiene 980px de ancho disponibles, y tus breakpoints pensados para pantallas pequeñas nunca llegarán a activarse. Es, con diferencia, el olvido más frecuente al empezar con diseño responsive.

6.3 Sintaxis completa de las media queries

Ya viste una primera introducción a las media queries. Ahora completamos su sintaxis: una regla @media aplica un bloque de CSS solo cuando se cumple una condición sobre el dispositivo o la ventana.

```
@media screen and (max-width: 768px) {
  .card {
    flex-direction: column;
  }
}
```

Parte	Función
screen	Tipo de medio: screen (pantallas), print (impresión) o all (por defecto, cualquiera). En la práctica, casi siempre se omite y se sobreentiende all.
and	Combina el tipo de medio con una o más condiciones (features).

(max-width: 768px)	La **media feature**: la condición real que se evalúa.

Las features más usadas son max-width (se activa cuando la ventana mide como mucho ese ancho) y min-width (se activa cuando mide, como mínimo, ese ancho). Se pueden combinar varias condiciones con and, o con una coma (equivalente a un "o" lógico):

```
/* Entre 600px y 900px (ambas condiciones a la vez) */
@media (min-width: 600px) and (max-width: 900px) { ... }

/* Menos de 600px, O más de 1200px (cualquiera de las dos) */
@media (max-width: 600px), (min-width: 1200px) { ... }
```

Nota: CSS moderno permite también una sintaxis de rango, más parecida a una desigualdad matemática: @media (600px <= width <= 900px) es equivalente a la primera media query del ejemplo anterior, y para muchas personas resulta más legible. Ambas sintaxis son válidas hoy en cualquier navegador actual; usa la que te resulte más clara.

6.4 Breakpoints: cuáles y por qué

Un breakpoint es el ancho de pantalla en el que el diseño cambia de estructura. No existe una lista oficial ni obligatoria, pero estos valores, populares gracias a frameworks como Bootstrap, son un punto de partida razonable y ampliamente reconocido:

Rango aproximado	Dispositivo típico
< 576px	Móvil, en vertical
576px – 768px	Móvil en horizontal, tableta pequeña
768px – 992px	Tableta
992px – 1200px	Portátil, escritorio pequeño
> 1200px	Escritorio grande

Importante: La recomendación profesional real no es memorizar estos valores y aplicarlos a ciegas, sino diseñar guiándote por el propio contenido: prueba tu diseño reduciendo el ancho del navegador poco a poco, y coloca un breakpoint exactamente donde el diseño empieza a romperse (un texto que se aplasta, una tarjeta demasiado estrecha para su contenido). A esos puntos se les llama, con acierto, breakpoints de contenido, y casi nunca coinciden exactamente con las dimensiones de un modelo de móvil concreto.

6.5 Mobile-first frente a desktop-first

Existen dos estrategias opuestas para organizar las media queries de un proyecto, según qué versión se escribe primero como base:

Desktop-first (con max-width)

```
.tarjetas {
  display: grid;
  grid-template-columns: repeat(3,
}

@media (max-width: 900px) {
  .tarjetas { grid-template-columns
}
@media (max-width: 600px) {
  .tarjetas { grid-template-columns
}
```

Mobile-first (con min-width)

```
.tarjetas {
  display: grid;
  grid-template-columns: 1fr;
}

@media (min-width: 600px) {
  .tarjetas { grid-template-columns
}
@media (min-width: 900px) {
  .tarjetas { grid-template-columns
}
```

Las dos versiones producen exactamente el mismo resultado visual en los tres rangos de ancho. La diferencia está en la base sin media queries: en la de la izquierda es la versión de escritorio (3 columnas); en la de la derecha, la versión de móvil (1 columna).

Mobile-first (con min-width) es hoy la estrategia recomendada de forma prácticamente unánime, por dos razones de peso:

- **Rendimiento**: un móvil con conexión limitada descarga y aplica solo el CSS base (pensado para él), sin tener que procesar primero un diseño de escritorio complejo que luego tiene que deshacer.
- **Disciplina de diseño**: obliga a decidir primero qué es realmente imprescindible en el espacio más reducido, añadiendo complejidad solo a medida que hay sitio de sobra, en lugar de partir de todo y recortar.

Nota: "Mobile-first" no significa que el móvil sea más importante que el escritorio como audiencia: es, únicamente, el orden en el que se escribe el CSS (de la versión más simple a la más compleja), independientemente de en qué dispositivo se vaya a usar más una web concreta.

6.6 Combinar media queries con Flexbox y Grid

En la práctica, una media query casi nunca actúa sola: su trabajo habitual es cambiar el valor de una propiedad de Flexbox o Grid para reorganizar un componente completo.

```
.resp-card {
  display: flex;
  gap: 18px;
}

@media (max-width: 700px) {
  .resp-card {
```

```
    flex-direction: column;
  }
}
```

Este patrón (fila en pantallas anchas, columna en pantallas estrechas) es, probablemente, el cambio responsive más habitual de todos: aparece en tarjetas, en cabeceras, y en el propio layout general de la página.

6.7 Unidades e imágenes responsive

Imágenes flexibles
La regla más antigua (y todavía la más usada) del diseño responsive es esta:

```
img {
  max-width: 100%;
  height: auto;
}
```

max-width: 100% impide que una imagen se salga del ancho de su contenedor, sea cual sea su tamaño original; height: auto asegura que, al reducirse el ancho, el alto se reduzca en la misma proporción, sin deformar la imagen. Profundizaremos en las técnicas de imágenes responsive (el atributo srcset, distintos tamaños según resolución de pantalla) más adelante.

Tipografía fluida con clamp()
Antes de clamp(), escalar un título entre un tamaño mínimo y uno máximo exigía dos o tres media queries distintas, cada una fijando un tamaño de fuente distinto. clamp(mínimo, preferido, máximo) resuelve esto en una sola línea:

```
h1 {
  /* nunca menos de 1rem, nunca más de 2.2rem, y en medio, un 4% del ancho del viewport */
  font-size: clamp(1rem, 4vw, 2.2rem);
}
```

Nota: clamp() no sustituye a las media queries (no puede cambiar flex-direction ni reorganizar una rejilla), pero es la herramienta más elegante disponible hoy específicamente para tamaños que deben variar de forma continua, como el de la tipografía o algunos espaciados.

6.8 Otras media features

Más allá del ancho, las media queries pueden consultar otras características del dispositivo o de las preferencias configuradas por la persona usuaria en su sistema operativo:

Media feature	Detecta
orientation: portrait / landscape	Si la pantalla está en vertical o en horizontal.

prefers-color-scheme: light / dark	Si la persona usuaria tiene activado el tema oscuro en su sistema.
prefers-reduced-motion: reduce	Si ha pedido minimizar animaciones (por mareo, por sensibilidad al movimiento...).
hover: hover / pointer: fine	Si el dispositivo principal admite realmente pasar el cursor por encima (un ratón) o es de precisión fina, frente a un dedo en una pantalla táctil.

```css
/* Solo aplica el efecto hover en dispositivos que realmente tienen "hover" */
@media (hover: hover) {
  .tarjeta:hover {
    transform: translateY(-4px);
  }
}
```

Importante: Un efecto :hover definido sin esta comprobación no falla en pantallas táctiles, pero se comporta de forma confusa: el primer toque activa el estado hover en lugar de la acción principal, y hace falta un segundo toque para "activarlo" de verdad. Envolver los efectos de hover puramente decorativos en @media (hover: hover) evita ese problema.

6.9 Container queries: responsive por componente

Todo lo visto hasta ahora responde al ancho de la ventana. Pero un mismo componente (una tarjeta, por ejemplo) puede vivir a veces en una columna estrecha de 250px y otras veces en un contenido principal de 800px, dentro de la misma página: una media query no puede distinguir esos dos casos, porque el ancho de la ventana es idéntico en ambos.

Las container queries resuelven exactamente esto: en lugar de preguntar por el ancho de la ventana, el componente pregunta por el ancho de su propio contenedor.

```css
/* 1. Declara el contenedor que se va a consultar */
.contenedor-tarjeta {
  container-type: inline-size;
  container-name: tarjeta;
}

/* 2. Consulta ese contenedor, no la ventana */
@container tarjeta (min-width: 300px) {
  .card {
    flex-direction: row;
  }
}
```



Nota: Las container queries no sustituyen a las media queries: se complementan. Las media queries siguen siendo la herramienta adecuada para el layout general de la página, las container queries brillan cuando construyes componentes verdaderamente reutilizables (como los componentes de Lunacy) que no controlan de antemano en qué parte de la página van a colocarlos.

6.10 Herramientas de prueba

Todos los navegadores modernos incluyen un modo de diseño responsive dentro de las herramientas de desarrollador (icono de móvil/tableta, o Ctrl+Shift+M en Chrome), que simula distintos anchos de pantalla sin necesidad de cambiar de dispositivo físico, y permite además fijar preferencias como prefers-color-scheme o prefers-reduced-motion para probar esas media features.

Importante: El modo responsive del navegador es un simulador, no el dispositivo real: no reproduce con exactitud el rendimiento, el tamaño de fuente por defecto del sistema, ni matices táctiles del hardware real. Es excelente para iterar rápido durante el desarrollo, pero antes de dar por terminado un proyecto conviene probarlo en, al menos, un móvil y una tableta reales.

Actividad · WebCompleta 5

La etiqueta <meta name="viewport"> ya está en el <head> desde el Paso 1: sin ella, nada de lo que viene ahora funcionaría en un móvil de verdad. Hacer estos cambios:

1. **Tipografía fluida**: en .hero h1, sustituye font-size: 2.6rem; por font-size: clamp(1.8rem, 5vw + 1rem, 2.6rem);. No hace falta ninguna media query para esto: el tamaño fluye solo entre esos dos extremos.
2. **Media query**: al final del archivo, crea un @media (max-width: 768px) { } con estas reglas dentro:
   .nav-wrap: flex-direction: column, align-items: flex-start, gap: 14px (para que el logo, los enlaces y el botón se apilen).
   nav: flex-wrap: wrap.
   .hero, .features, .pricing y .testimonios: reduce su padding a 56px 20px.
   .features-grid y .pricing-grid: grid-template-columns: 1fr, para que pasen a una sola columna.

Fíjate que no hace falta tocar .testimonials-grid: como ya usa flex-wrap: wrap desde el Paso 3, se adapta sola. Es un buen ejemplo de por qué Flexbox suele necesitar menos ayuda responsive que Grid con columnas fijas.

Prueba el resultado con el modo de diseño responsive del navegador. Con este ejercicio, la Web de Nimbus queda terminada.

7. Hojas de estilo alternativas y validación

Una misma página no tiene por qué verse igual siempre: puede verse de otra manera al imprimirla, en el tema oscuro del sistema, o cuando alguien elige un estilo distinto. En este tema aprenderás a definir esas hojas de estilo alternativas (de la técnica clásica de HTML a las herramientas actuales), y a comprobar que todo tu CSS es correcto antes de darlo por bueno.

7.1 Hojas de estilo alternativas

HTML lleva décadas ofreciendo un mecanismo para que una misma página tenga varias hojas de estilo entre las que la persona usuaria puede elegir: las hojas de estilo alternativas. Se declaran con la etiqueta <link> que ya conoces, añadiendo dos detalles: rel="alternate stylesheet" y un atributo title que da nombre al estilo.

```
<!-- Siempre se aplica -->
<link rel="stylesheet" href="base.css">

<!-- Estilo preferido: se aplica por defecto -->
<link rel="stylesheet" href="claro.css" title="Claro">

<!-- Estilos alternativos: desactivados hasta que alguien los elija -->
<link rel="alternate stylesheet" href="oscuro.css" title="Oscuro">
<link rel="alternate stylesheet" href="alto-contraste.css" title="Alto contraste">
```

Según su combinación de atributos, cada hoja de estilo de un documento cae en una de estas tres categorías:

Tipo	Cómo se declara	Comportamiento

Persistente	rel="stylesheet", sin title	Se aplica siempre, elija lo que elija la persona usuaria.
Preferida	rel="stylesheet" con title	Se aplica por defecto, pero se desactiva si se elige una alternativa.
Alternativa	rel="alternate stylesheet" con title	Desactivada por defecto; se activa al elegirla, y desactiva la preferida.

Importante: Aquí está la cara menos amable de esta técnica: el menú para elegir entre estilos alternativos (Ver > Estilo de página) solo existe de forma nativa en Firefox. Los demás navegadores no ofrecen ninguna interfaz para ello sin una extensión, de modo que, si quieres que una persona usuaria cualquiera pueda cambiar de estilo, tendrás que construir tú mismo el botón que lo haga. Por eso hoy se usa poco en producción, aunque sigue siendo útil para comparar dos diseños durante el desarrollo.

Por ello, para adaptar una página a las necesidades de quien la visita, hoy se recurre sobre todo a las herramientas de los apartados siguientes, que sí funcionan en todos los navegadores actuales.

7.2 Hojas de estilo por medio: pantalla e impresión

Hay un tipo de hoja alternativa que sí funciona desde siempre en todos los navegadores, porque no depende de que la persona usuaria elija nada: el navegador la activa solo, según el medio en el que se muestra la página. Se indica con el atributo media de <link> (o con @media dentro de una hoja):

```
<link rel="stylesheet" href="pantalla.css" media="screen">
<link rel="stylesheet" href="impresion.css" media="print">
```

El caso más útil en la práctica es el de impresión: nadie quiere gastar tinta en un menú de navegación, un fondo de color o un botón que, en papel, no sirven para nada. Una hoja de impresión típica hace cosas como estas:

```
@media print {
  /* Oculta lo que no tiene sentido en papel */
  nav, .banner, .btn { display: none; }

  /* Texto negro sobre blanco, sin fondos que gasten tinta */
  body { background: #fff; color: #000; }

  /* Muestra la dirección de cada enlace, porque en papel no se puede pulsar */
  a[href]::after { content: " (" attr(href) ")"; }

  /* Evita que una tarjeta se parta entre dos páginas */
  .card { break-inside: avoid; }
}
```

```css
/* Márgenes de la página impresa */
@page { margin: 2cm; }
```



El mismo HTML, dos presentaciones. En la de impresión desaparecen el menú y la imagen decorativa, el texto pasa a negro sobre blanco y cada enlace muestra su dirección entre paréntesis. Cuando lo apliques a tu propia página, pulsa Ctrl+P para ver la vista previa de impresión con tu hoja aplicada.

Nota: Las herramientas de desarrollador de los navegadores permiten emular el medio de impresión sin necesidad de imprimir nada (en Chrome, dentro del panel de renderizado, la opción "Emulate CSS media type: print"). Es la forma más rápida de depurar una hoja de impresión.

7.3 Tema claro y oscuro con prefers-color-scheme

La mayoría de sistemas operativos permiten elegir entre un tema claro y uno oscuro, y el navegador expone esa elección a las páginas web mediante la media feature prefers-color-scheme. Es el mecanismo que sí funciona en todos los navegadores actuales, y el punto de partida de cualquier tema oscuro moderno:

```css
body {
  background: #FFFFFF;
  color: #24262B;
}

@media (prefers-color-scheme: dark) {
  body {
    background: #1E1A26;
    color: #EDE8F2;
  }
}
```

Importante: Un tema oscuro no consiste en invertir los colores. Si te limitas a poner fondo oscuro y mantener el resto igual, los colores de acento pensados para fondo claro

se vuelven casi invisibles: sobre un fondo oscuro. Cada tema necesita su propia paleta, comprobada por separado.

7.4 Temas con variables, color-scheme y light-dark()

Reescribir cada regla dentro de una media query, como en el ejemplo anterior, escala mal: en un proyecto real habría que duplicar decenas de colores. La solución es guardar los colores en variables CSS y cambiar solo las variables, no las reglas que las usan.

```
:root {
  --bg: #FFFFFF;
  --fg: #24262B;
  --acento: #4B3563;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #1E1A26;
    --fg: #EDE8F2;
    --acento: #C9A6E8;
  }
}

body { background: var(--bg); color: var(--fg); }
```

Con esto, el resto de la hoja de estilos no necesita saber nada del tema oscuro: usa var(--bg) y ya está. Pero CSS moderno ofrece dos herramientas que simplifican todavía más este patrón:

La propiedad color-scheme

color-scheme le dice al navegador con qué esquemas de color está preparada la página (light, dark o light dark para ambos). Cuando declaras color-scheme: light dark, el navegador adapta por sí mismo los elementos que estiliza él: barras de desplazamiento, campos de formulario, casillas, y el color del texto y del fondo por defecto.

La función light-dark()

light-dark(valor-claro, valor-oscuro) devuelve uno de sus dos colores según el esquema que esté activo. Elimina la necesidad de la media query y de duplicar las variables:

```
:root {
  color-scheme: light dark;

  --bg:     light-dark(#FFFFFF, #1E1A26);
  --fg:     light-dark(#24262B, #EDE8F2);
  --acento: light-dark(#4B3563, #C9A6E8);
}
```

Nota: light-dark() está disponible en los navegadores principales desde 2024, así que en un navegador algo antiguo no funcionará. Para proyectos que deban soportarlos, el

patrón de variables con media query de la primera versión sigue siendo la alternativa segura. Y ten presente que light-dark() solo admite colores: no vale para tamaños, imágenes u otros valores.

7.5 Selector manual de tema y preferencia del sistema

Respetar la preferencia del sistema es lo correcto por defecto, pero muchas personas quieren poder cambiar de tema en una web concreta sin tocar la configuración de todo su dispositivo. El patrón profesional combina las dos cosas, con tres estados posibles: claro, oscuro y seguir al sistema.

La idea es sencilla: por defecto, la página deja que color-scheme: light dark siga al sistema; cuando la persona elige explícitamente un tema, un atributo data-theme en el elemento raíz fuerza light o dark. Puedes probarlo en el ejemplo de más abajo, que aplica ese mismo CSS a un contenedor de la página en lugar de a :root, para no cambiar el tema de todo este documento:

```
:root {
  color-scheme: light dark; /* sin elección manual: sigue al sistema */
}
:root[data-theme="light"] { color-scheme: light; }
:root[data-theme="dark"]  { color-scheme: dark; }
```

7.6 Validación de hojas de estilo

Existe un validador oficial de CSS. Lo primero que conviene entender es cómo reacciona el navegador ante un error: no se detiene ni avisa, sino que ignora la declaración incorrecta y continúa con la siguiente. Por eso los errores de CSS suelen ser silenciosos.

Errores típicos y su efecto

```
.card {
  colour: #4B3563;          /* 1. Propiedad inexistente (colour en vez de color) */
  padding: 16 px;           /* 2. Espacio entre número y unidad */
  width: 200;               /* 3. Falta la unidad (solo 0 puede ir sin unidad) */
  border-radius: 10px       /* 4. Falta el punto y coma */
  margin: 0 auto;
}

.card:hover {
  transform: translateY(-4px);
/* 5. Falta la llave de cierre de .card:hover */

.card p {
  margin: 0;
}
```

Error	Qué hace el navegador
1. Propiedad inexistente	Ignora esa línea, el texto conserva su color anterior.
2. 16 px	Valor no válido: ignora padding y la tarjeta se queda sin relleno.
3. 200 sin unidad	Valor no válido: ignora width.
4. Sin ; tras 10px	El navegador lee 10px margin: 0 auto como un único valor absurdo y descarta **las dos** declaraciones: border-radius y también margin, que sí estaba bien escrita.
5. Llave sin cerrar	La regla siguiente (.card p) se interpreta como parte de la que quedó abierta y deja de aplicarse como esperabas. Ojo: si la llave que falta es la de la **última** regla del archivo, el navegador la cierra por su cuenta y no notarás nada.

Importante: El error 4 es el más traicionero: una sola letra que falta (el punto y coma) invalida dos propiedades, incluida una que estaba perfectamente escrita. Cuando una propiedad "no hace nada" y su sintaxis parece correcta, revisa siempre la línea inmediatamente anterior.

Tres herramientas complementarias

Herramienta	Qué aporta	Cuándo usarla
Validador CSS del W3C jigsaw.w3.org/css-validator	Revisa un archivo, una URL o un fragmento pegado, y lista errores y avisos con su número de línea.	Antes de entregar un trabajo o dar por cerrada una hoja de estilos.
Herramientas de desarrollador	El panel de estilos marca en tachado (con un aviso) las declaraciones que el navegador ha descartado, y muestra qué regla gana la cascada.	Depurando en directo, elemento por elemento.
Linter (Stylelint)	Analiza el CSS mientras escribes y lo revisa contra un conjunto de reglas configurables: errores, estilo y buenas prácticas.	En cualquier proyecto real, integrado en el editor y en el flujo de trabajo del equipo.

8. CSS profesional actual

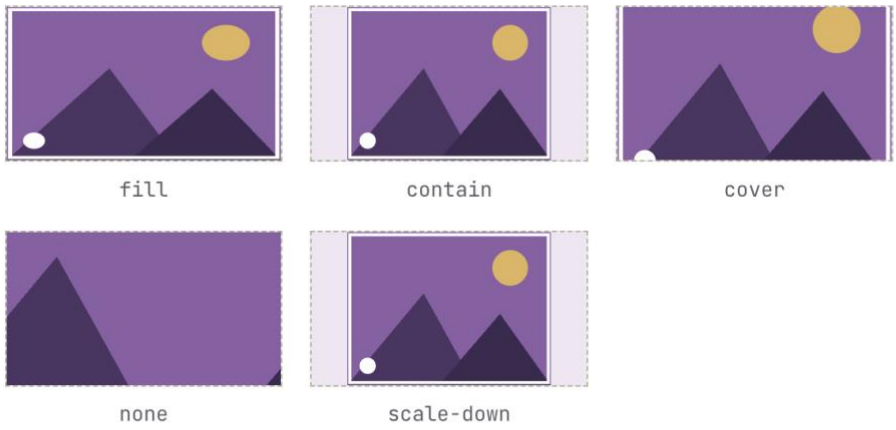
Este tema reúne lo que separa un CSS "que funciona" de un CSS que se escribe hoy en un proyecto profesional: propiedades que se usan a diario y que no habías visto todavía (object-fit, aspect-ratio), formas más robustas de dimensionar y organizar, y un puñado de herramientas modernas de color, texto, efectos y scroll. Al final de cada apartado verás el estado de soporte en los navegadores actuales.

8.1 Imágenes y vídeo dentro de su caja

Hay un problema que aparece en cuanto una interfaz muestra fotos de verdad: las imágenes rara vez tienen la proporción de la caja donde las quieres colocar. En una galería de tarjetas, una foto vertical, una horizontal y una cuadrada deben ocupar el mismo hueco. Si solo fijas width y height en la etiqueta <img>, el navegador estira la imagen hasta deformarla.

object-fit
La propiedad object-fit decide cómo se ajusta el contenido de un elemento "reemplazado" (<img>, <video>, <canvas>...) a la caja que le has dado. Es el equivalente, para imágenes normales, de lo que background-size hacía con las imágenes de fondo.

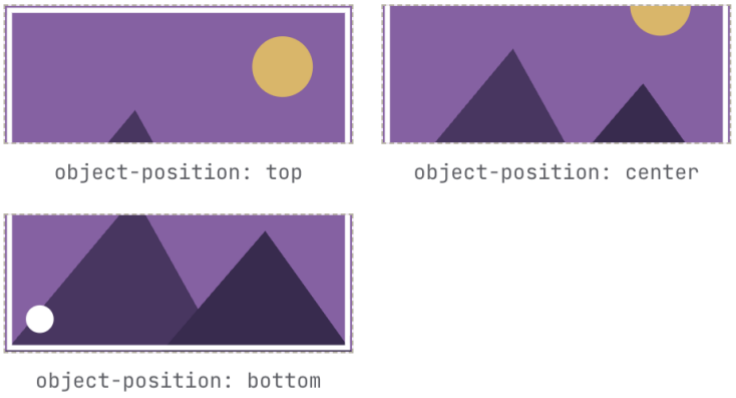
Valor	Comportamiento	Equivale en background-size
fill (por defecto)	Estira la imagen hasta llenar la caja, deformándola.	(sin equivalente directo)
contain	Escala la imagen para que se vea entera, respetando su proporción; puede dejar huecos.	contain
cover	Escala la imagen para llenar la caja, respetando su proporción; recorta lo que sobre.	cover
none	No escala: muestra la imagen a su tamaño original, recortada por la caja.	(sin equivalente directo)
scale-down	Elige entre none y contain el que dé la imagen más pequeña.	(sin equivalente directo)



La misma imagen de 400 × 300 px (proporción 4:3, con un marco blanco y un punto blanco en la esquina inferior izquierda para ver qué se recorta) dentro de una caja de 160 × 90 px (proporción 16:9). Con `fill` se deforma; con `contain` sobran franjas a los lados; con `cover` se pierden los bordes superior e inferior; con `none` solo se ve la parte central del tamaño original.

object-position

Con cover, el navegador centra la imagen y recorta por igual a ambos lados. object-position permite decidir qué parte se conserva (funciona como background-position, y admite palabras clave o porcentajes). Es imprescindible en retratos, donde recortar por el centro puede cortar la cabeza:



Las tres cajas usan `object-fit: cover` sobre una caja muy apaisada (200 × 80 px). Con `top` se conserva el sol; con `bottom`, las montañas y el punto blanco.

aspect-ratio

aspect-ratio fija la proporción entre el ancho y el alto de un elemento, sin necesidad de calcular la altura a mano ni de usar el viejo truco del padding-top: 56.25%. Se escribe como una razón (16 / 9) o como un número (1 equivale a 1 / 1): basta con dar el ancho y el alto se deduce.



Tres cajas con el mismo width: 150px y ninguna altura declarada. Cada una calcula la suya a partir de su aspect-ratio, y la mantiene si el ancho cambia.

Los tres juntos forman el patrón profesional más repetido con imágenes: una caja con proporción fija, y la imagen rellenándola sin deformarse.

```css
/* Imagen de tarjeta: siempre 16:9, se recorta si hace falta */
.tarjeta img {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
}

/* Avatar redondo: cuadrado perfecto + círculo + recorte centrado */
.avatar {
  width: 72px;
  aspect-ratio: 1;
  border-radius: 50%;
  object-fit: cover;
}
```



Los dos patrones del código anterior, aplicados sobre la misma imagen 4:3.

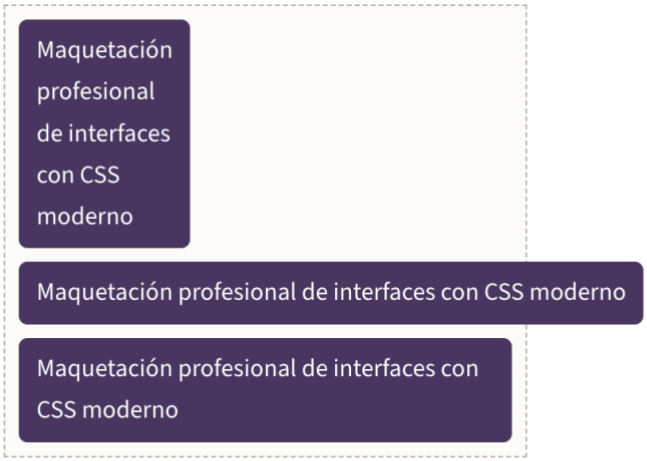
Importante: Aunque uses CSS para dimensionar, conviene seguir poniendo los atributos width y height en la etiqueta <img> (con los valores reales de la imagen) y height: auto en la hoja de estilos. Los navegadores los usan para calcular la proporción antes de descargar la imagen y reservar su hueco, evitando que el contenido "salte" cuando la imagen termina de cargarse. Ese salto, llamado desplazamiento de diseño, es una de las cosas que más penaliza la experiencia (y el posicionamiento en buscadores).

Nota: object-fit y aspect-ratio tienen soporte amplio en todos los navegadores actuales. Recuerda que object-fit solo actúa sobre elementos como <img> o <video>; para una imagen aplicada como background-image hay que usar background-size.

8.2 Tamaños intrínsecos y unidades modernas

min-content, max-content y fit-content
Además de píxeles y porcentajes, el ancho y el alto admiten palabras clave que dependen del contenido:

Valor	Significado
min-content	El mínimo posible sin desbordar: el texto se parte en cada palabra y la caja mide lo que la palabra más larga.
max-content	El máximo natural: el contenido en una sola línea, sin partir, aunque se salga de su contenedor.
fit-content	Se ajusta al contenido, pero sin pasar del espacio disponible: crece hasta max-content y, si no cabe, se parte.



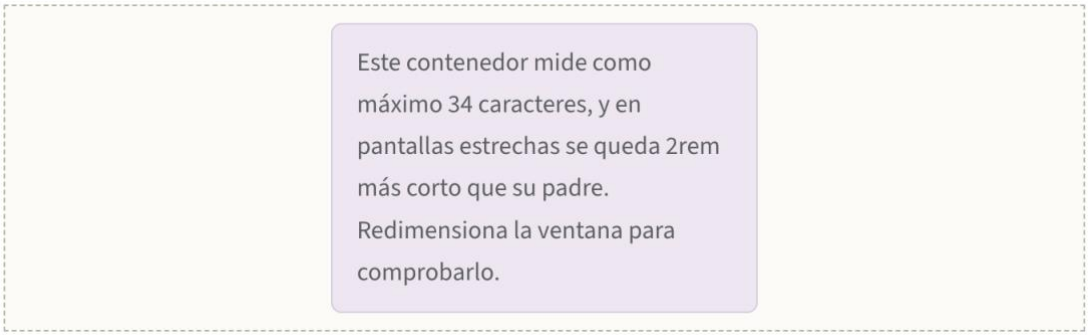
de arriba abajo: min-content · max-content · fit-content (contenedor de 300px)

El mismo texto en tres cajas dentro de un contenedor de 300 px. `max-content` desborda al no partirse nunca; `fit-content` es el término medio y el más útil: una etiqueta o un botón que se ajusta a su texto, pero que nunca se sale de su sitio.

Contenedores fluidos con min(), max() y calc()
Además de clamp(), CSS ofrece min() y max(), que eligen el menor o el mayor de varios valores, y calc(), que permite mezclar unidades distintas (calc(100% - 2rem)).

Combinadas resuelven, sin ninguna media query, el patrón de contenedor de página más habitual:

```css
.contenedor {
  /* el menor entre: todo el ancho menos 2rem, y 65 caracteres de ancho */
  width: min(100% - 2rem, 65ch);
  margin-inline: auto;
}
```



Un contenedor que se centra, no se pasa de un ancho cómodo de lectura y deja margen a los lados en móvil, con una sola declaración. La unidad ch equivale aproximadamente al ancho del carácter "0" de la fuente.

La altura de la ventana en móvil: dvh, svh, lvh

Ya hemos usado min-height: 100vh para que el pie de página quedase abajo. En un móvil hay un problema: la barra de direcciones del navegador aparece y desaparece al hacer scroll, y 100vh se calcula como si estuviera siempre oculta, así que la página resulta unos píxeles más alta que la pantalla visible. Las unidades modernas resuelven el dilema con tres variantes:

Unidad	Altura de la ventana...
svh (*small*)	...con las barras del navegador visibles (la más pequeña posible).
lvh (*large*)	...con las barras ocultas (la más grande posible; equivale a vh).
dvh (*dynamic*)	...la real en cada momento, actualizándose al aparecer o desaparecer las barras.

```css
body {
  min-height: 100vh;   /* alternativa antigua, por si el navegador no conoce dvh */
  min-height: 100dvh;  /* la que se aplica en navegadores modernos */
}
```

Declarar las dos líneas seguidas es un patrón deliberado: si un navegador no entiende 100dvh, ignora esa declaración y se queda con la anterior.

8.3 Propiedades lógicas

Propiedades como margin-left o text-align: left están pensadas para un idioma que se lee de izquierda a derecha. En árabe o en hebreo, que se leen de derecha a izquierda, "el inicio" del texto está a la derecha, y esos valores dejarían de tener sentido. Las propiedades lógicas se definen respecto a la dirección de lectura, no respecto a la pantalla, y se adaptan solas:

Física (antigua)	Lógica (moderna)
margin-left / margin-right	margin-inline-start / margin-inline-end
margin-top / margin-bottom	margin-block-start / margin-block-end
padding-left y padding-right	padding-inline (un solo valor para ambos lados)
width / height	inline-size / block-size
border-left	border-inline-start
text-align: left	text-align: start
top, right, bottom, left	inset-block-start, inset-inline-end... (o inset, apartado 13.4)

8.4 Atajos que ahorran líneas

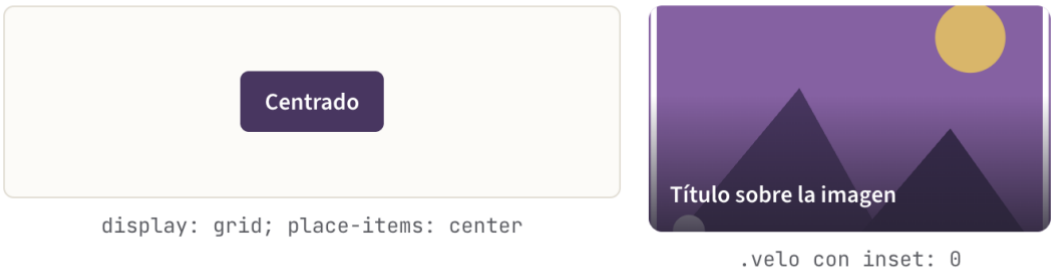
Atajo	Sustituye a
inset: 0;	top: 0; right: 0; bottom: 0; left: 0;
inset: 10px 20px;	Arriba y abajo 10px, izquierda y derecha 20px (mismo orden que margin)
margin-inline: auto;	margin-left: auto; margin-right: auto;
place-items: center;	align-items: center; justify-items: center; (pensado para Grid; en Flexbox, justify-items no tiene efecto)
place-content: center;	align-content: center; justify-content: center;
place-self: center;	align-self: center; justify-self: center; (en Grid)

Los dos que más se usan en la práctica son place-items, que reduce el centrado perfecto de un elemento a dos líneas, e inset: 0, que estira un elemento posicionado hasta cubrir por completo a su contenedor:

```
/* Centrado perfecto con Grid */
.centrado {
  display: grid;
  place-items: center;
```

```
}

/* Capa que cubre exactamente a su contenedor */
.velo {
  position: absolute;
  inset: 0;
}
```



A la izquierda, el centrado con Grid. A la derecha, un degradado oscuro colocado con inset: 0 sobre la imagen para que el texto blanco se lea bien.

8.5 CSS anidado nativo

Durante años, escribir las reglas de un componente en CSS obligaba a repetir su selector una y otra vez, y por eso existían preprocesadores como Sass. El CSS actual permite anidar reglas directamente, usando el símbolo & para referirse al selector padre:

Plano (repetitivo)

```
.tarjeta { padding: 14px; }
.tarjeta h4 { margin: 0; }
.tarjeta:hover {
  border-color: #4B3563;
}
@media (max-width: 600px) {
  .tarjeta { padding: 10px; }
}
```

Anidado

```
.tarjeta {
  padding: 14px;

  & h4 { margin: 0; }
  &:hover {
    border-color: #4B3563;
  }
  @media (max-width: 600px) {
    padding: 10px;
  }
}
```

Ambas versiones producen exactamente el mismo resultado. En la anidada, el `&` significa "este mismo selector" (`.tarjeta`), y hasta la media query queda dentro de la regla a la que afecta.

8.6 Organizar la cascada: @layer y arquitectura

Capas de la cascada

Vimos que cuando dos reglas chocan, gana la de mayor especificidad. En proyectos grandes eso lleva a "guerras de especificidad": para sobrescribir un estilo hay que escribir selectores cada vez más largos, o recurrir a !important. Las capas de la cascada (@layer) añaden un criterio previo a la especificidad: el orden de las capas.

```
/* 1. Declara el orden: las capas posteriores ganan a las anteriores */
@layer reset, base, componentes, utilidades;

@layer base {
  #pagina .titulo { color: red; }   /* especificidad alta */
}
@layer utilidades {
  .titulo { color: green; }     /* especificidad baja, pero capa posterior */
}
```

El título será verde: la capa utilidades se declaró después que base, así que gana sin importar la especificidad de los selectores de una y otra. Dentro de una misma capa, las reglas siguen resolviéndose por especificidad como siempre. Compruébalo en directo con la misma regla escrita con y sin capas:

Sin capas

#lyr-a .lyr-x (rojo, con id) frente a .lyr-g-a (verde, solo clase): gana el id → texto rojo

Con capas (@layer lyr-base, lyr-util)

#lyr-b .lyr-x (rojo, con id) en la capa base frente a .lyr-g-b (verde, solo clase) en la capa de utilidades: gana la capa posterior → texto verde

En las dos cajas compiten una regla roja con id y una regla verde con solo una clase. Solo cambia que, en la segunda, cada regla está dentro de su capa.

Importante: Los estilos que no están dentro de ninguna capa ganan a los que sí lo están, sea cual sea su especificidad o su orden. Por eso, al adoptar @layer, lo habitual es meter en capas todo el CSS propio del proyecto y de las librerías, y dejar fuera solo aquellos ajustes puntuales que deben ganar siempre. Las capas tienen soporte en todos los navegadores actuales desde 2022.

Un reset moderno

Los navegadores aplican estilos por defecto que varían ligeramente entre ellos. Casi todos los proyectos empiezan con un pequeño "reset" que los neutraliza. Este es un punto de partida actual, mínimo y razonado:

```
@layer reset {
```

```
  *, *::before, *::after { box-sizing: border-box; }
  * { margin: 0; }
  body { line-height: 1.6; -webkit-font-smoothing: antialiased; }
  img, picture, video, svg { display: block; max-width: 100%; }
  input, button, textarea, select { font: inherit; }
  h1, h2, h3 { text-wrap: balance; }
}
```

Cómo organizar y nombrar el CSS de un proyecto
A medida que crece un proyecto, hace falta una convención para nombrar clases y repartir el código. Hay dos enfoques dominantes:

Enfoque	Idea	Ejemplo
Componentes con nombre (BEM)	Cada componente tiene su bloque, sus elementos y sus modificadores, con nombres descriptivos.	.tarjeta, .tarjeta__titulo, .tarjeta--destacada
Clases de utilidad	Muchas clases pequeñas de un solo propósito, combinadas directamente en el HTML.	class="flex gap-4 p-4 rounded"

El segundo enfoque es el que sigue Tailwind CSS, al que dedicamos el siguiente tema.

8.7 Selectores de estado y accesibilidad

:focus-visible
La pseudo-clase :focus se activa siempre que un elemento recibe el foco, tanto al pulsarlo con el ratón como al llegar a él con el teclado. El resultado suele ser molesto: el anillo de foco aparece en cada clic sobre un botón, y por eso muchos diseñadores caían en la tentación de escribir outline: none, dejando sin ninguna señal de foco a las personas que navegan con teclado. :focus-visible resuelve el dilema: solo se activa cuando el navegador considera que la persona necesita ver el foco (típicamente, al navegar con teclado).

:focus-within
:focus-within selecciona a un **contenedor** cuando cualquiera de sus descendientes tiene el foco. Permite resaltar el grupo entero de un campo de formulario (etiqueta, campo y ayudas) en lugar de solo el campo:

:user-invalid y :user-valid
La pseudo-clase clásica :invalid se activa desde el primer instante: un campo obligatorio y vacío aparece en rojo antes de que la persona haya escrito nada. :user-invalid espera a que la persona haya interactuado con el campo (normalmente, hasta que lo abandona) para mostrar el error, y :user-valid hace lo mismo para confirmar que es correcto.

:not() y :target
:not() excluye elementos de una selección, y evita tener que sobrescribir un estilo para deshacerlo: li:not(:last-child) { border-bottom: 1px solid #ccc; } pone un separador

entre elementos de una lista sin ponerlo tras el último. :target selecciona el elemento cuyo id coincide con el fragmento de la URL (#seccion), lo que permite resaltar la sección a la que acaba de saltar un enlace interno sin JavaScript:

```
:target {
  outline: 3px solid #A9812E;
  outline-offset: 4px;
}
```

accent-color
Los controles nativos de formulario (casillas, botones de radio, barras deslizantes, barras de progreso) suelen verse con el color azul del navegador. accent-color los recolorea con el color de tu marca con una sola línea, sin reconstruirlos.



Regla usada: `input { accent-color: var(--accent); }`

8.8 Texto profesional

text-wrap: balance y pretty
Los títulos que se parten en dos líneas dejan a menudo una segunda línea con una o dos palabras sueltas. text-wrap: balance reparte el texto de forma equilibrada entre las líneas; text-wrap: pretty está pensado para párrafos largos y evita las líneas finales con una sola palabra (las "viudas"). Ambos son solo una mejora tipográfica: un navegador que no los conozca los ignora y el texto se ve igual que siempre.

Limitar líneas y evitar desbordamientos

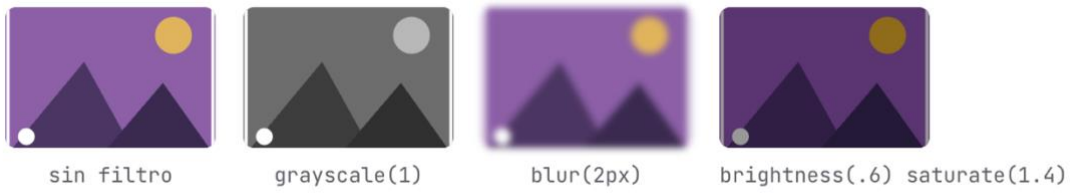
Necesidad	Técnica
Recortar un texto largo a N líneas con puntos suspensivos (resúmenes de tarjeta)	display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 3; overflow: hidden;
Recortar a una sola línea con puntos suspensivos	white-space: nowrap; overflow: hidden; text-overflow: ellipsis;
Que una palabra o URL larguísima no rompa la maquetación	overflow-wrap: anywhere; (o break-word)



Sin `overflow-wrap`, la dirección larga se sale de su caja. Con `anywhere`, el navegador la parte donde haga falta.

8.9 Efectos visuales

filter
filter aplica efectos gráficos a cualquier elemento: blur(), grayscale(), brightness(), contrast(), sepia(), saturate(), hue-rotate() y drop-shadow(). Se pueden combinar separándolos con espacios.
backdrop-filter
Mientras filter afecta al propio elemento, backdrop-filter afecta a lo que hay detrás de él, y produce el efecto de "cristal esmerilado" tan habitual en interfaces modernas. Necesita un fondo total o parcialmente transparente para que el efecto se vea:



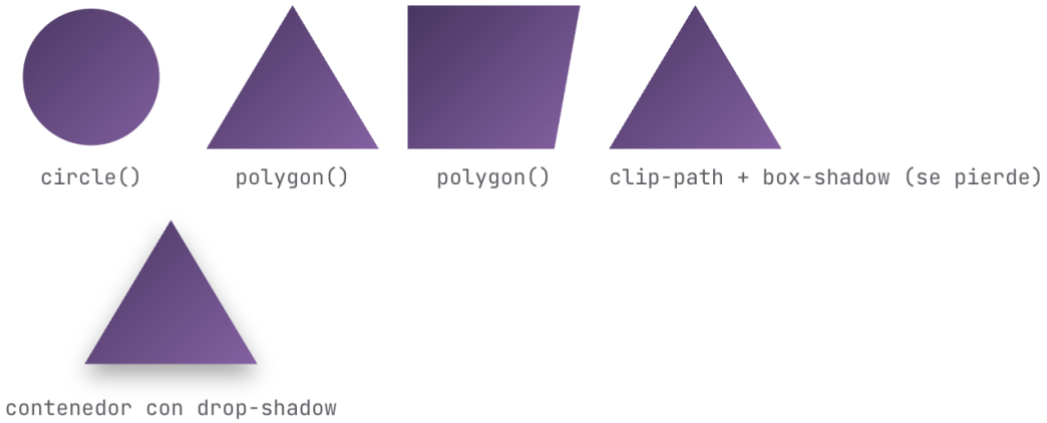
backdrop-filter
Mientras filter afecta al propio elemento, backdrop-filter afecta a lo que hay detrás de él, y produce el efecto de "cristal esmerilado" tan habitual en interfaces modernas. Necesita un fondo total o parcialmente transparente para que el efecto se vea:



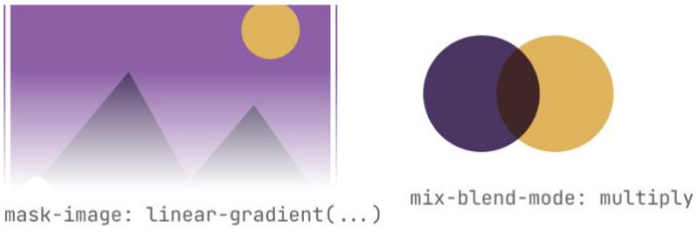
Regla usada: `background: rgba(255,255,255,.28); backdrop-filter: blur(8px);`. El paisaje se difumina solo en la zona que queda detrás de la tarjeta.

clip-path y drop-shadow
clip-path recorta un elemento con una forma: circle(), ellipse(), inset() o polygon() con las coordenadas que quieras. Tiene una consecuencia que sorprende: como recorta todo lo que hay fuera de la forma, **también recorta la sombra** de box-shadow. La solución

es envolver el elemento en otro y aplicar allí filter: drop-shadow(), que sigue el contorno visible:



mask y mix-blend-mode
mask-image hace transparente una parte del elemento según una imagen o un degradado (donde la máscara es opaca se ve el elemento; donde es transparente, desaparece), lo que permite difuminar el borde de una imagen. mix-blend-mode controla cómo se mezcla un elemento con los que tiene debajo.



8.10 Scroll

Propiedad	Para qué sirve
scroll-behavior: smooth	Los saltos a enlaces internos (#seccion) se animan en lugar de saltar de golpe.
scroll-snap-type / scroll-snap-align	El desplazamiento "se imanta" a puntos concretos: carruseles, galerías, pantallas completas.
scroll-margin-top / scroll-padding-top	Deja un hueco al saltar a un ancla, para que una cabecera fija no tape el título de destino.
overscroll-behavior: contain	Impide que, al llegar al final de un panel con scroll propio (un menú, un modal), el scroll "se escape" a la página de detrás.
scrollbar-gutter: stable	Reserva siempre el hueco de la barra de desplazamiento, para que el contenido no se mueva cuando la barra aparece.

Carrusel con scroll-snap

Un carrusel horizontal sin ninguna línea de JavaScript:

```css
.carrusel {
  display: flex;
  overflow-x: auto;
  scroll-snap-type: x mandatory;   /* eje horizontal, obligatorio */
}
.carrusel > * {
  flex: 0 0 70%;
  scroll-snap-align: start;        /* cada elemento se ancla por su borde inicial */
}
```